TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG QUẢN LÝ NÔNG TRẠI NHÀ KÍNH

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **PHẠM MINH TIẾN**
MSSV: 21119140
TRẦN TRUNG HIẾU
MSSV: 21119067

TP. HỒ CHÍ MINH – 07/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ – THÔNG TIN



ĐỒ ÁN TỐT NGHIỆP

HỆ THỐNG QUẢN LÝ NÔNG TRẠI NHÀ KÍNH

NGÀNH CÔNG NGHỆ KỸ THUẬT MÁY TÍNH

Sinh viên: **PHẠM MINH TIẾN**
MSSV: 21119140

TRẦN TRUNG HIẾU
MSSV: 21119067

Hướng dẫn**: PGS. TS. PHAN VĂN CA**

TP. HỒ CHÍ MINH – 06/2026

Trường Đại học CNKT TP.HCM
Khoa Điện – Điện tử
BM KT ĐT-TT

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

THÔNG TIN ĐỒ ÁN TỐT NGHIỆP

1. THÔNG TIN SINH VIÊN

- Họ và tên sinh viên: Phạm Minh Tiến
- Mã số sinh viên: 21119140
- Lớp: 21119CL1B
- Email: 21119140@student.hcmute.edu.vn
- Số điện thoại: 0941085565
- Họ và tên sinh viên: Trần Trung Hiếu
- Mã số sinh viên: 21119067
- Lớp: 21119CL3B
- Email: 21119067@student.hcmute.edu.vn
- Số điện thoại: 0948478669
- Ngành: Công nghệ Kỹ thuật Máy tính
- Khoa: Điện – Điện tử

2. THÔNG TIN ĐỀ TÀI

- Tên đề tài (Tiếng Việt): Hệ thống quản lý nông trại nhà kính
- Tên đề tài (Tiếng Anh): Greenhouse Farm Management System
- Loại hình:

  - ☐ Nghiên cứu
  - ☑ Thiết kế
  - ☑ Chế tạo
  - ☑ Mô phỏng

- ☐ Khác: .....

- Thời gian thực hiện:

  - Ngày giao đề tài: 26/03/2026
  - Ngày hoàn thành: 21/06/2026

3. GIẢNG VIÊN HƯỚNG DẪN

- Họ và tên: Phan Văn Ca
- Học hàm/Học vị: PGS. TS
- Đơn vị công tác: Trường Đại học Công nghệ Kỹ thuật TP.HCM
- Email: capv@hcmute.edu.vn

4. TÓM TẮT NỘI DUNG ĐỒ ÁN

Đồ án xây dựng hệ thống quản lý nông trại nhà kính thông minh cho ba nhà kính trồng xà lách, củ cải đỏ và củ cải trắng. Mạng lưới cảm biến thu thập liên tục dữ liệu về nhiệt độ, độ ẩm và ánh sáng, truyền về trung tâm rồi đưa lên giao diện web và ứng dụng điện thoại để người dùng theo dõi và điều khiển quạt, tưới, đèn từ xa. Hệ thống có khả năng tự vận hành theo ngưỡng môi trường và theo lịch, đồng thời phát hiện sự cố cả ở môi trường lẫn ở chính thiết bị. Toàn bộ dữ liệu được lưu lại phục vụ phân tích, và kiến trúc được thiết kế mở để dễ dàng bổ sung cảm biến hoặc khu vực mới khi cần.

5. KẾT QUẢ ĐẠT ĐƯỢC

- ☑ Báo cáo đồ án tốt nghiệp.
- ☑ Mô hình / Thiết bị / Phần mềm hoàn chỉnh.
- ☐ Mã nguồn chương trình.
- ☐ Kết quả mô phỏng và thực nghiệm.
- ☐ Bài báo khoa học (nếu có).
- ☐ Sản phẩm khác: .....

6. KIỂM TRA ĐẠO VĂN

- Phần mềm kiểm tra: Turnitin
- Ngày kiểm tra: 25/6/2026
- Tỷ lệ tương đồng (Similarity Index): 39%

- Kết luận:

  - ☑ Đạt yêu cầu theo quy định của Nhà trường.
  - ☐ Không đạt yêu cầu theo quy định của Nhà trường.

7. CAM KẾT CỦA SINH VIÊN

Tôi cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Tôi hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện	Sinh viên thực hiện
	
Phạm Minh Tiến	Trần Trung Hiếu

Trường Đại học CNKT TP.HCM
Khoa Điện – Điện tử
BM KT ĐT-TT

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên sinh viên/MSSV: Phạm Minh Tiến - 21119140

Họ và tên sinh viên/MSSV: Trần Trung Hiếu - 21119067

Ngành: Công nghệ Kỹ thuật Máy tính

Tên đề tài: Hệ thống quản lý nông trại nhà kính

Giảng viên hướng dẫn: PGS. TS. Phan Văn Ca

1. Nhận xét chung về tinh thần, thái độ làm việc, nội dung và kết quả thực hiện đề tài

Sinh viên có tinh thần và thái độ tốt, nội dung và kết quả thực hiện phù hợp với mục tiêu ban đầu

2. Đánh giá chung: Mức độ hoàn thành các mục tiêu đề ra:

☐ Xuất sắc ☐ Tốt ☑ Khá ☐ Đạt yêu cầu ☐ Chưa đạt yêu cầu

3. Kết luận

☑ **Đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.**

☐ **Không đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.**

Lý do (nếu không đồng ý):

.....

TP. Hồ Chí Minh, ngày 25 tháng 6 năm 2026

GIẢNG VIÊN HƯỚNG DẪN

Phan Văn Ca

ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự Do – Hạnh phúc

Tp. HCM , ngày 10 tháng 7 năm 2026

BẢN GIẢI TRÌNH CHỈNH SỬA ĐỒ ÁN TỐT NGHIỆP

SAU PHẢN BIỆN

NGÀNH: CNKT MÁY TÍNH (HỆ ĐT CLC)

1. Tên đề tài: HỆ THỐNG QUẢN LÝ NÔNG TRẠI NHÀ KÍNH
2. Tên sinh viên: Phạm Minh Tiến MSSV 21119140
   Tên sinh viên: Trần Trung Hiếu MSSV 21119067
3. GVHD:
4. Hội đồng bảo vệ HĐ 1, phòng A104 , ngày 10 tháng 7 năm 2026
5. Giải trình chỉnh sửa báo cáo đồ án tốt nghiệp theo ý kiến của thầy/cô Phản biện:

TT	Nội dung góp ý của thầy/cô Phản biện	Kết quả chỉnh sửa, bổ sung	Ghi chú
1	Một số hình vẽ, chữ viết còn nhỏ và mờ, bị mất một phần.	Đã vẽ lại 7 lưu đồ thuật toán (Hình 3.35-3.41) theo chuẩn IEEE, bổ sung 5 ảnh minh họa cơ sở dữ liệu MongoDB (Hình 3.30, 4.20-4.23).	
2	Một số lỗi typos, chính tả cần sửa.	Đã rà soát và sửa toàn bộ lỗi chính tả, thuật ngữ không thống nhất trong toàn bài (ZigBee, LoRa, quản lí/quản lý...).	
3	Tài liệu tham khảo: lựa chọn các bài tạp chí trong nước và nước ngoài, sách, không sử dụng các nguồn web, các trang mạng.	Đã cập nhật và chỉnh sửa toàn bộ nguồn web bằng các tài liệu tham khảo là bài báo khoa học và sách trong, ngoài nước.	
4	Tài liệu tham khảo phải được trích dẫn trong luận	Đã chèn trích dẫn [1]-[13] tại các vị trí liên quan trong luận	

	văn, trích dẫn theo thứ tự tăng dần.	văn, đánh số tăng dần theo đúng thứ tự xuất hiện.	
5	Các công thức phải được đánh lại, không sử dụng kiểu copy các công thức dưới dạng hình để dán vào luận văn; đánh số theo chương.	Đã rà soát toàn bộ luận văn, xác nhận không có công thức toán học nào được trình bày dưới dạng hình ảnh.	
6	Rà soát cơ sở lý thuyết, những lý thuyết quá đơn giản, chung chung thì bỏ.	Đã xóa triệt để các đoạn lý thuyết suông (định nghĩa OSI, vai trò cơ sở dữ liệu, ứng dụng thực tiễn chuẩn truyền thông...), chỉ giữ lại bảng so sánh, số liệu và nội dung liên hệ trực tiếp đến hệ thống nhà kính.	
7	Chỉnh sửa lại các lưu đồ cho đúng chuẩn.	Đã vẽ lại toàn bộ lưu đồ theo chuẩn IEEE, đường nối vuông góc, không chồng chéo.	
8	Caption cho bảng và hình đúng chuẩn.	Đã kiểm tra và xác nhận caption Hình/Bảng trong toàn luận văn đúng định dạng, đánh số nhất quán theo chương.	

Xác nhận của Trưởng ngành

PGS.TS. Trương Ngọc Sơn

Nhóm thực hiện báo cáo

(Ký họ và tên)

Phạm Minh Tiến

Hiếu

Trần Trung Hiếu

LỜI CẢM ƠN

Trong suốt quá trình học tập tại trường và thực hiện khóa luận tốt nghiệp, nhóm chúng tôi đã nhận được rất nhiều sự quan tâm, giúp đỡ và hướng dẫn từ quý Thầy Cô, gia đình và bạn bè. Những kiến thức chuyên môn cùng những kinh nghiệm quý báu được tích lũy trong thời gian học tập đã tạo nền tảng quan trọng để chúng tôi hoàn thành đề tài này.

Nhóm xin gửi lời cảm ơn chân thành đến quý Thầy Cô ngành Công nghệ Kỹ thuật Máy tính, khoa Điện – Điện tử, trường Đại học Sư phạm Kỹ thuật Thành phố Hồ Chí Minh đã tận tình giảng dạy, truyền đạt kiến thức và tạo điều kiện thuận lợi cho chúng tôi trong suốt quá trình học tập và nghiên cứu.

Đặc biệt, nhóm xin bày tỏ lòng biết ơn sâu sắc đến Thầy Phan Văn Ca, người đã trực tiếp hướng dẫn và đồng hành cùng nhóm trong suốt quá trình thực hiện đề tài. Những định hướng chuyên môn, sự tận tâm hỗ trợ và những góp ý quý báu của Thầy đã giúp nhóm từng bước khắc phục khó khăn, hoàn thiện nội dung nghiên cứu và hoàn thành khóa luận này.

Nhóm cũng xin chân thành cảm ơn gia đình, bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện thuận lợi để chúng tôi có thể tập trung học tập và thực hiện đề tài.

Mặc dù đã nỗ lực hết sức, khóa luận khó tránh khỏi những thiếu sót nhất định. Nhóm rất mong nhận được những ý kiến đóng góp từ quý Thầy Cô để tiếp tục hoàn thiện kiến thức và kỹ năng của mình trong tương lai.

Nhóm thực hiện xin chân thành cảm ơn!

TÓM TẮT

Hệ thống quản lý nông trại nhà kính được xây dựng nhằm hỗ trợ giám sát các thông số môi trường và điều khiển thiết bị điện từ xa trong nhà kính. Hệ thống cho phép thu thập, truyền nhận, lưu trữ và hiển thị dữ liệu theo thời gian thực, đồng thời cung cấp các chức năng điều khiển phục vụ quá trình quản lý và vận hành.

Dữ liệu môi trường như nhiệt độ, độ ẩm và cường độ ánh sáng được thu thập thông qua các Node cảm biến bố trí tại nhiều khu vực khác nhau trong nhà kính. Mỗi Node sử dụng kiến trúc gồm hai vi điều khiển, trong đó ESP32 đảm nhiệm việc đọc dữ liệu cảm biến và giao tiếp truyền thông, còn STM32 thực hiện điều khiển các thiết bị điện. Dữ liệu từ Node được truyền về Gateway thông qua giao thức LoRa, sau đó Gateway sử dụng giao thức MQTT để trao đổi dữ liệu với máy chủ thông qua kết nối Internet.

Hệ thống sử dụng MongoDB để lưu trữ dữ liệu cảm biến, thông tin người dùng, Gateway, Node và các thiết bị điện. Phần mềm máy chủ được phát triển trên nền tảng Node.js, cung cấp các API phục vụ việc quản lý dữ liệu, giám sát hệ thống và xử lý các yêu cầu điều khiển từ người dùng.

Người dùng tương tác với hệ thống thông qua giao diện Website hỗ trợ trên nhiều loại thiết bị khác nhau. Giao diện cho phép theo dõi dữ liệu cảm biến dưới dạng biểu đồ thời gian thực, kiểm tra trạng thái hoạt động của thiết bị và thực hiện điều khiển từ xa. Bên cạnh phương thức điều khiển thủ công, hệ thống còn hỗ trợ điều khiển theo lịch trình hoặc theo các điều kiện môi trường được thiết lập trước.

Với sự kết hợp giữa mạng cảm biến, Gateway, máy chủ và giao diện Web, hệ thống hình thành một giải pháp giám sát và điều khiển tập trung cho mô hình nhà kính. Kiến trúc được thiết kế theo hướng mở, cho phép bổ sung thêm Node, cảm biến hoặc các chức năng mới khi có nhu cầu mở rộng trong tương lai.

MỤC LỤC

CHƯƠNG 1 GIỚI THIỆU 1

1.1 GIỚI THIỆU 1

1.2 MỤC TIÊU ĐỀ TÀI 2

1.3 PHƯƠNG PHÁP NGHIÊN CỨU 3

1.4 GIỚI HẠN NGHIÊN CỨU 4

1.5 BỐ CỤC QUYỂN BÁO CÁO 5

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT 6

2.1 TỔNG QUAN VỀ CHUẨN TRUYỀN THÔNG KHÔNG DÂY 6

2.1.1 Chuẩn truyền thông là gì? 6

2.1.2 So sánh ưu, nhược điểm của truyền thông không dây và có dây 7

2.1.3 Các công nghệ truyền thông không dây phổ biến 8

2.1.4 Ứng dụng thực tiễn của các chuẩn truyền thông không dây . 17

2.2 CƠ SỞ DỮ LIỆU 17

2.2.1 Khái niệm 17

2.2.2 Các vai trò của Cơ sở dữ liệu 18

2.2.3 Phân loại 18

CHƯƠNG 3 THIẾT KẾ HỆ THỐNG 22

3.1 YÊU CẦU CHUNG CỦA HỆ THỐNG 22

3.2 CHỨC NĂNG KỸ THUẬT CỦA HỆ THỐNG 22

3.3 SƠ ĐỒ KHỐI HỆ THỐNG 24

3.3.1 Hệ thống phần cứng 26

3.3.2 Cơ sở dữ liệu – Server Backend 50

3.3.3 Giao diện người dùng 64
3.4 THUẬT TOÁN VÀ PHÂN LUỒNG DỮ LIỆU 70
3.4.1 Quản lý dữ liệu người dùng 70
3.4.2 Quản lý dữ liệu dự án 73
CHƯƠNG 4 KẾT QUẢ 88
4.1 TỔNG QUAN KẾT QUẢ ĐẠT ĐƯỢC 88
4.2 KẾT QUẢ PHẦN CỨNG 89
4.2.1 Phần cứng Node 89
4.2.2 Phần cứng Gateway 90
4.3 KẾT QUẢ PHẦN MỀM 91
4.3.1 Giao diện Website 91
4.3.2 Cơ sở dữ liệu 98
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 104
5.1 KẾT LUẬN 104
5.1.1 Các vấn đề đã giải quyết được 104
5.1.2 Ưu điểm của hệ thống 105
5.1.3 Hạn chế của hệ thống 105
5.2 HƯỚNG PHÁT TRIỂN 106
TÀI LIỆU THAM KHẢO 108

DANH MỤC HÌNH

HÌNH 2.1: SƠ ĐỒ KẾT NỐI THIẾT BỊ TRONG MẠNG WIFI..... 9

HÌNH 2.2: SƠ ĐỒ KẾT NỐI CÁC THIẾT BỊ TRONG BLUETOOTH MESH 10

HÌNH 2.3: SƠ ĐỒ KẾT NỐI CÁC THIẾT BỊ TRONG MẠNG ZIGBEE THEO DẠNG LƯỚI 10

HÌNH 2.4: SƠ ĐỒ KẾT NỐI CÁC THIẾT BỊ TRONG MẠNG LORAWAN..... 11

HÌNH 2.5: BIỂU ĐỒ SO SÁNH BĂNG THÔNG VÀ KHOẢNG CÁCH GIỮA CÁC CHUẨN TRUYỀN THÔNG 13

HÌNH 2.6: MÔ HÌNH GIAO TIẾP HTTP 13

HÌNH 2.7: MÔ HÌNH GIAO TIẾP MQTT..... 14

HÌNH 3.1: SƠ ĐỒ KHỐI HỆ THỐNG QUẢN LÝ NÔNG TRẠI NHÀ KÍNH 24

HÌNH 3.2: SƠ ĐỒ KHỐI PHẦN CỨNG..... 26

HÌNH 3.3: HÌNH ẢNH THỰC TẾ MODULE RELAY KÍCH 4 CỔNG . 29

HÌNH 3.4: HÌNH ẢNH THỰC TẾ MODULE ESP32 (Ở NODE) 32

HÌNH 3.5: HÌNH ẢNH THỰC TẾ VI ĐIỀU KHIỂN STM32F103C8T6 BLUEPILL..... 32

HÌNH 3.6: HÌNH ẢNH THỰC TẾ MODULE LORA E32 433T30D 34

HÌNH 3.7: HÌNH ẢNH THỰC TẾ MODULE ESP32 (Ở GATEWAY) .. 38

HÌNH 3.8: LUỒNG DỮ LIỆU PHÁT TẠI GATEWAY 41

HÌNH 3.9: LUỒNG DỮ LIỆU THU TẠI NODE..... 42
HÌNH 3.10: SƠ ĐỒ NGUYÊN LÝ TỔNG QUAN CỦA NODE 43
HÌNH 3.11: SƠ ĐỒ NGUYÊN LÝ BỘ XỬ LÝ TRUNG TÂM NODE... 44
HÌNH 3.12: SƠ ĐỒ NGUYÊN LÝ KHỐI CẢM BIẾN 45
HÌNH 3.13: SƠ ĐỒ NGUYÊN LÝ KHỐI NGUỒN 46
HÌNH 3.14: SƠ ĐỒ NGUYÊN LÝ CÁC HEADER NGUỒN..... 46
HÌNH 3.15: SƠ ĐỒ NGUYÊN LÝ KẾT NỐI MODULE LORA..... 47
HÌNH 3.16: SƠ ĐỒ NGUYÊN LÝ KẾT NỐI CỦA BUZZER..... 48
HÌNH 3.17: SƠ ĐỒ NGUYÊN LÝ TỔNG QUAN CỦA GATEWAY 48
HÌNH 3.18: SƠ ĐỒ NGUYÊN LÝ CỦA ESP32 49
HÌNH 3.19: SƠ ĐỒ TỔNG QUAN CỦA SERVER BACKEND - CƠ SỞ DỮ LIỆU. 51
HÌNH 3.20: KIẾN TRÚC CÂY THƯ MỤC BACKEND. 53
HÌNH 3.21: LƯU ĐỒ HOẠT ĐỘNG TỔNG QUAN CỦA BACKEND. 56
HÌNH 3.22: LƯU ĐỒ LUỒNG HOẠT ĐỘNG THỰC HIỆN API CỦA BACKEND 57
HÌNH 3.23: LƯU ĐỒ LUỒNG HOẠT ĐỘNG THỰC HIỆN MQTT CỦA BACKEND. 58
HÌNH 3.24: KHAI BÁO CÁC ĐỐI TƯỢNG CÓ TRONG USER. 59
HÌNH 3.25: KHAI BÁO CÁC ĐỐI TƯỢNG CÓ TRONG GATEWAYS. 60

HÌNH 3.26: KHAI BÁO CÁC ĐỐI TƯỢNG CÓ TRONG NODES. 60
HÌNH 3.27: KHAI BÁO CÁC ĐỐI TƯỢNG CÓ TRONG DEVICES. ... 61
HÌNH 3.28: KHAI BÁO CÁC ĐỐI TƯỢNG CÓ TRONG SENSORS.... 61
HÌNH 3.29: DANH SÁCH CÁC API CUNG CẤP CHO WEBSITE..... 62
HÌNH 3.30: MÃ KEY ĐỂ BẢO MẬT API. 62
HÌNH 3.31: KHI KHÔNG NHẬP KEY HOẶC KEY SAI SẼ KHÔNG PHẢN HỒI. 63
HÌNH 3.32: NHẬP API KEY ĐÚNG. 64
HÌNH 3.33: CẤU TRÚC WEBSITE CƠ BẢN. 67
HÌNH 3.34: MÔ HÌNH GIAO TIẾP GIỮA CLIENT VÀ SERVER THÔNG QUA REST API. 68
HÌNH 3.35: LƯU ĐỒ THUẬT TOÁN ĐĂNG KÝ NGƯỜI DÙNG..... 71
HÌNH 3.36: LƯU ĐỒ THUẬT TOÁN ĐĂNG NHẬP. 72
HÌNH 3.37: LƯU ĐỒ THUẬT TOÁN THÊM DỰ ÁN MỚI..... 74
HÌNH 3.38: LƯU ĐỒ THUẬT TOÁN CÀI ĐẶT PHẦN CỨNG MỚI ... 75
HÌNH 3.39: LƯU ĐỒ THUẬT TOÁN CHỈNH SỬA DỰ ÁN 76
HÌNH 3.40: LƯU ĐỒ THUẬT TOÁN XÓA TÊN GATEWAY (PHẦN CỨNG)..... 77
HÌNH 3.41: LƯU ĐỒ THUẬT TOÁN XÓA DỰ ÁN 78
HÌNH 3.42: LƯU ĐỒ THUẬT TOÁN TẠO MỚI NODE CHO GATEWAY 79

HÌNH 3.43: LƯU ĐỒ THUẬT TOÁN ĐỌC DỮ LIỆU CẢM BIẾN..... 81
HÌNH 3.44: LƯU ĐỒ THUẬT TOÁN ĐIỀU KHIỂN THIẾT BỊ 82
HÌNH 3.45: LƯU ĐỒ THUẬT TOÁN SỬA CHỮA KẾT NỐI MQTT... 84
HÌNH 3.46: LƯU ĐỒ THUẬT TOÁN SỬA CHỮA KẾT NỐI WIFI 85
HÌNH 3.47: LƯU ĐỒ THUẬT TOÁN XỬ LÝ GÓI TIN LORA..... 86
HÌNH 3.48: LƯU ĐỒ THUẬT TOÁN GỬI VÀ CHỜ PHẢN HỒI LORA 87
HÌNH 4.1: HÌNH ẢNH THỰC TẾ BOARD PHẦN CỨNG NODE (NHÀ KÍNH). 89
HÌNH 4.2: HÌNH ẢNH THỰC TẾ BOARD PHẦN CỨNG GATEWAY. 90
HÌNH 4.3: GIAO DIỆN ĐĂNG KÝ TÀI KHOẢN NGƯỜI DÙNG. 91
HÌNH 4.4: GIAO DIỆN ĐĂNG NHẬP NGƯỜI DÙNG. 92
HÌNH 4.5: GIAO DIỆN TRANG CHỦ HỆ THỐNG..... 92
HÌNH 4.6: GIAO DIỆN TRANG THÔNG TIN NGƯỜI DÙNG. 93
HÌNH 4.7: FORM CẬP NHẬT THÔNG TIN CÁ NHÂN..... 93
HÌNH 4.8: FORM THAY ĐỔI MẬT KHẨU NGƯỜI DÙNG. 94
HÌNH 4.9: FORM THAY ĐỔI AVATAR NGƯỜI DÙNG..... 94
HÌNH 4.10: GIAO DIỆN QUẢN LÝ DỰ ÁN..... 95
HÌNH 4.11: FORM THÊM DỰ ÁN MỚI..... 95
HÌNH 4.12: FORM CHỈNH SỬA THÔNG TIN DỰ ÁN ĐÃ CÓ..... 95

HÌNH 4.13: FORM THÊM NHÀ KÍNH MỚI VÀO DỰ ÁN. 96
HÌNH 4.14: FORM CHỈNH SỬA NHÀ KÍNH ĐÃ CÓ..... 96
HÌNH 4.15: FORM CẤU HÌNH GATEWAY MỚI. 96
HÌNH 4.16: GIAO DIỆN QUẢN LÝ NHÀ KÍNH (NODE). 97
HÌNH 4.17: GIAO DIỆN HIỂN THỊ THÔNG SỐ CẢM BIẾN. 97
HÌNH 4.18: GIAO DIỆN HIỂN THỊ BIỂU ĐỒ CẢM BIẾN VÀ ĐIỀU KHIỂN THIẾT BỊ. 98
HÌNH 4.19: DỮ LIỆU CỦA USER ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 98
HÌNH 4.20: DỮ LIỆU CỦA GATEWAYS ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 99
HÌNH 4.21: DỮ LIỆU CỦA NODE ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 99
HÌNH 4.22: DỮ LIỆU CẢM BIẾN NHIỆT ĐỘ THUỘC SENSORS ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 100
HÌNH 4.23: DỮ LIỆU CẢM BIẾN ĐỘ ẨM THUỘC SENSORS ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 101
HÌNH 4.24: DỮ LIỆU CẢM BIẾN ÁNH SÁNG THUỘC SENSORS ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 101
HÌNH 4.25: DỮ LIỆU THIẾT BỊ QUẠT GIÓ THUỘC DEVICES ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 102

HÌNH 4.26: DỮ LIỆU THIẾT BỊ ĐÈN CHIẾU THUỘC DEVICES ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 102
HÌNH 4.27: DỮ LIỆU THIẾT BỊ MÁY BƠM THUỘC DEVICES ĐƯỢC LƯU TRÊN CƠ SỞ DỮ LIỆU. 103

DANH MỤC BẢNG

BẢNG 2.1: SO SÁNH TRUYỀN THÔNG CÓ DÂY VÀ KHÔNG DÂY 7

BẢNG 2.2: SO SÁNH CÁC CÔNG NGHỆ TRUYỀN THÔNG KHÔNG DÂY TRONG IOT 11

BẢNG 2.3: SO SÁNH CÁC GIAO THỨC HTTP, MQTT VÀ WEBSOCKET TRONG IOT 15

BẢNG 2.4: SO SÁNH HAI KIỂU CƠ SỞ DỮ LIỆU QUAN HỆ VÀ KHÔNG QUAN HỆ 19

BẢNG 3.1: SO SÁNH MỘT SỐ VI ĐIỀU KHIỂN PHỔ BIẾN 31

BẢNG 3.2: SO SÁNH CÁC PHƯƠNG PHÁP BIỂU DIỄN DỮ LIỆU CHO TRUYỀN THÔNG LORA 39

CÁC TỪ VIẾT TẮT

Từ viết tắt	Ý nghĩa
ADC	Analog-to-Digital Converter
API	Application Programming Interface
ARM	Advanced RISC Machine
Base64	Base64 Encoding
BLE	Bluetooth Low Energy
CPU	Central Processing Unit
CSS	Chirp Spread Spectrum
ESP32	Espressif 32-bit Microcontroller
ESP8266	Espressif 8266 WiFi MCU
FEC	Forward Error Correction
Flash	Flash Memory
GND	Ground
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
I2C	Inter-Integrated Circuit
IEEE	Institute of Electrical and Electronics Engineers
IoT	Internet of Things (Internet vạn vật)
JSON	JavaScript Object Notation
KB	Kilobyte
LAN	Local Area Network
LoRa	Long Range Radio
LoRaWAN	Long Range Wide Area Network
LTE	Long Term Evolution
MB	Megabyte
MCU	Microcontroller Unit

MQTT	Message Queuing Telemetry Transport
OSI	Open Systems Interconnection
PCB	Printed Circuit Board
PWM	Pulse Width Modulation
QoS	Quality of Service
RAM	Random Access Memory
REST API	Representational State Transfer API
RF	Radio Frequency
RTC	Real-Time Clock
RXD	Receive Data
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SRAM	Static Random Access Memory
STM32	STMicroelectronics 32-bit MCU Family
TCP/IP	Transmission Control Protocol / Internet Protocol
TLV	Type – Length – Value
TXD	Transmit Data
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
WiFi	Wireless Fidelity
XML	eXtensible Markup Language

CHƯƠNG 1

GIỚI THIỆU

1.1 GIỚI THIỆU

Trong bối cảnh cuộc Cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ, Việt Nam đang đẩy nhanh quá trình chuyển đổi số nhằm xây dựng nền kinh tế và xã hội số. Sự phát triển của các công nghệ như trí tuệ nhân tạo, điện toán đám mây, dữ liệu lớn và đặc biệt là Internet of Things (IoT) đang tạo ra nhiều thay đổi trong hoạt động sản xuất, quản lý và đời sống. Nhờ khả năng kết nối và trao đổi dữ liệu giữa các thiết bị, IoT ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau.

Để đáp ứng nhu cầu phát triển các hệ thống công nghệ số, Việt Nam đã và đang đầu tư mạnh vào hạ tầng viễn thông, mạng 5G cùng các nền tảng dữ liệu quy mô lớn. Đây là nền tảng quan trọng cho việc triển khai các ứng dụng IoT trong quản lý đô thị, giao thông, giáo dục và y tế. Một ví dụ tiêu biểu là ứng dụng VNeID do Bộ Công an phát triển, cho phép tích hợp các thông tin cá nhân trên môi trường số, từng bước thay thế giấy tờ truyền thống và nâng cao hiệu quả quản lý dữ liệu dân cư.

Trong lĩnh vực y tế, IoT góp phần hỗ trợ công tác chăm sóc sức khỏe thông qua các thiết bị theo dõi từ xa như máy đo huyết áp, cảm biến nhịp tim hoặc thiết bị giám sát đường huyết. Các dữ liệu sức khỏe được thu thập liên tục giúp người dùng chủ động theo dõi tình trạng cơ thể, đồng thời hỗ trợ bác sĩ đánh giá và theo dõi bệnh nhân từ xa. Theo định hướng chuyển đổi số ngành y tế, đến năm 2025 phần lớn các cơ sở khám chữa bệnh sẽ triển khai hồ sơ sức khỏe điện tử, tạo điều kiện thuận lợi cho việc lưu trữ, chia sẻ và khai thác dữ liệu y tế một cách hiệu quả hơn.

Không chỉ trong y tế, IoT còn được ứng dụng mạnh mẽ trong nông nghiệp công nghệ cao. Các cảm biến môi trường có khả năng giám sát nhiệt độ, độ ẩm, cường độ ánh sáng và chất lượng không khí theo thời gian thực, giúp người nông dân điều chỉnh điều kiện sản xuất phù hợp. Bên cạnh đó, các hệ thống tưới tự động dựa trên dữ liệu cảm biến

và thông tin thời tiết giúp tối ưu lượng nước sử dụng, giảm chi phí vận hành và nâng cao năng suất cây trồng. Trong chăn nuôi, việc gắn cảm biến theo dõi sức khỏe vật nuôi giúp phát hiện sớm các dấu hiệu bất thường, từ đó hạn chế rủi ro dịch bệnh và nâng cao hiệu quả quản lý trang trại [1].

Mặc dù mang lại nhiều lợi ích trong việc tự động hóa và quản lý dữ liệu, việc triển khai các hệ thống IoT trên quy mô lớn vẫn còn nhiều thách thức. Khi số lượng thiết bị kết nối tăng lên, lượng dữ liệu được truyền tải liên tục có thể gây quá tải hệ thống mạng, làm phát sinh độ trễ hoặc mất gói tin. Ngoài ra, việc lưu trữ và xử lý khối lượng dữ liệu lớn cũng đòi hỏi hạ tầng máy chủ có khả năng mở rộng linh hoạt và đáp ứng tốc độ truy xuất cao.

Xuất phát từ những yêu cầu thực tế đó, nhóm thực hiện đề tài "Hệ thống quản lý nông trại nhà kính" nhằm xây dựng một giải pháp giám sát và điều khiển từ xa dựa trên nền tảng IoT. Hệ thống sử dụng công nghệ truyền thông không dây LoRa để kết nối các khu vực nhà kính với Gateway trung tâm. Dữ liệu sau khi được thu thập sẽ được chuyển tiếp lên máy chủ thông qua giao thức MQTT và hiển thị trên giao diện Web Dashboard để người dùng theo dõi, quản lý và điều khiển từ xa. Giải pháp này hướng đến việc nâng cao hiệu quả truyền dữ liệu, giảm chi phí triển khai, tăng khả năng mở rộng hệ thống và đảm bảo hoạt động ổn định trong các mô hình nhà kính có quy mô lớn.

Việc nghiên cứu và phát triển các hệ thống IoT ứng dụng trong nông nghiệp không chỉ góp phần nâng cao hiệu quả sản xuất mà còn hỗ trợ quá trình chuyển đổi số của ngành nông nghiệp Việt Nam. Đây được xem là một trong những hướng tiếp cận phù hợp nhằm tối ưu hóa nguồn lực, tiết kiệm chi phí và phát triển nền nông nghiệp theo hướng hiện đại, bền vững.

1.2 MỤC TIÊU ĐỀ TÀI

Đề tài này hướng đến việc phát triển một hệ thống giám sát và điều khiển từ xa cho nông trại nhà kính dựa trên nền tảng IoT, với các mục tiêu cụ thể như sau:

- **Thu thập dữ liệu môi trường** từ các cảm biến (nhiệt độ, độ ẩm không khí, cường độ ánh sáng) được triển khai tại nhiều khu vực nhà kính khác nhau trong cùng một nông trại.
- **Truyền dữ liệu về trung tâm xử lý** thông qua giao thức không dây tầm xa LoRa, đảm bảo kết nối ổn định, tiết kiệm chi phí và phù hợp với các nông trại có địa hình rộng, phân tán nhiều khu vực.
- **Tổng hợp, phân tích và hiển thị dữ liệu** trên nền tảng Website một cách trực quan, giúp người dùng theo dõi các thông số theo thời gian thực, xem biểu đồ thống kê, phân tích xu hướng và nhận cảnh báo khi phát hiện bất thường.
- **Điều khiển từ xa các thiết bị điện** trong nhà kính (đèn chiếu sáng, máy bơm tưới tiêu, quạt thông gió) theo ba chế độ: thủ công, theo lịch trình định sẵn hoặc tự động theo điều kiện môi trường (ví dụ: tự động bật quạt khi nhiệt độ vượt ngưỡng, tự động bật đèn khi chỉ số ánh sáng thấp).
- **Đảm bảo khả năng mở rộng** của hệ thống, cho phép dễ dàng bổ sung thêm các Node cảm biến và khu vực nhà kính mới mà không cần thay đổi đáng kể kiến trúc tổng thể.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Phương pháp nghiên cứu trong đề tài này tập trung vào việc phân tích, thiết kế, triển khai và kiểm thử một hệ thống IoT gồm các thành phần Node (đặt tại từng nhà kính), Gateway (trung tâm thu nhận dữ liệu) và Server (máy chủ xử lý, lưu trữ và hiển thị). Cụ thể:

- **Thu thập và phân tích yêu cầu:** Tìm hiểu các yêu cầu chức năng và phi chức năng của hệ thống quản lý nông trại nhà kính dựa trên tài liệu tham khảo và yêu cầu thực tế. Phân tích các giao thức truyền thông liên quan (MQTT, UART, LoRa) và phần cứng phù hợp (vi điều khiển, cảm biến môi trường, module truyền thông). Đánh giá các công nghệ và thiết bị có thể áp dụng để giải quyết bài toán đặt ra.
- **Thiết kế hệ thống:** Xác định kiến trúc tổng thể bao gồm các thiết bị cảm biến, mạng truyền thông tầm xa, cơ sở dữ liệu và giao diện người dùng. Xác định các

thuật toán và phương pháp xử lý dữ liệu phù hợp với bài toán giám sát và điều khiển môi trường nhà kính.

- **Thi công hệ thống:** Dựa trên thiết kế, lựa chọn các thiết bị cảm biến và module IoT phù hợp; xây dựng phần cứng Node có khả năng đọc cảm biến, điều khiển thiết bị điện và truyền dữ liệu qua LoRa; xây dựng Gateway chuyển tiếp dữ liệu lên Server qua MQTT; phát triển cơ sở dữ liệu lưu trữ và xây dựng giao diện Web dashboard để người dùng theo dõi, điều khiển.
- **Kiểm thử và đánh giá:** Triển khai hệ thống vào môi trường thực tế và tiến hành thử nghiệm hoạt động. Đánh giá hiệu quả của hệ thống trong việc lưu trữ dữ liệu theo thời gian thực, đo lường khoảng cách truyền dẫn LoRa giữa các Node và Gateway, cũng như độ tin cậy của cơ chế điều khiển từ xa.
- **Đưa ra hướng tối ưu, cải tiến:** Dựa trên kết quả đánh giá, đề xuất các hướng tối ưu và cải tiến hệ thống trong tương lai nhằm nâng cao hiệu quả, độ tin cậy và khả năng mở rộng.

1.4 GIỚI HẠN NGHIÊN CỨU

Trong quá trình thực hiện đề tài, nhóm nghiên cứu đã đặt ra một số giới hạn nhằm tập trung vào các mục tiêu chính và đảm bảo tiến độ hoàn thành. Các giới hạn chính bao gồm:

- **Giới hạn về dữ liệu**: Hệ thống chỉ xử lý và lưu trữ các loại dữ liệu cảm biến cơ bản phục vụ canh tác nhà kính như nhiệt độ, độ ẩm, ánh sáng, chưa mở rộng sang các loại cảm biến chuyên biệt hoặc dữ liệu hình ảnh, video. Dữ liệu thu thập được giới hạn ở mức phù hợp với khả năng lưu trữ và xử lý của server hiện tại, chưa áp dụng các giải pháp big data hoặc xử lý dữ liệu lớn.
- **Giới hạn mô hình:** Mô hình tập trung vào các cảm biến môi trường cơ bản của nhà kính, chưa bao gồm các thiết bị đầu cuối đa dạng hoặc các cảm biến chuyên biệt cho từng loại cây trồng đặc thù. Dữ liệu được đơn giản hóa, chưa hỗ trợ các quy trình phức tạp hoặc tính năng nâng cao như phân tích dữ liệu lớn, học máy (machine learning) để dự đoán sinh trưởng cây trồng.

- **Giới hạn về thực tế:** Hệ thống được thiết kế và thử nghiệm trong phạm vi nhỏ với số lượng Node (nhà kính) và Gateway giới hạn, chưa chứng minh được khả năng mở rộng cho quy mô nông trại lớn hoặc môi trường công nghiệp phức tạp. Khoảng cách truyền dẫn LoRa được giới hạn trong phạm vi lý thuyết (khoảng 8 km), chưa kiểm tra trong môi trường có nhiều vật cản hoặc tầm xa hơn.

1.5 BỐ CỤC QUYỂN BÁO CÁO

Báo cáo khóa luận tốt nghiệp được tổ chức thành 5 chương:

- **Chương 1:** Giới thiệu
- **Chương 2:** Cơ sở lý thuyết
- **Chương 3:** Thiết kế hệ thống
- **Chương 4:** Kết quả
- **Chương 5:** Kết luận và hướng phát triển

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ CHUẨN TRUYỀN THÔNG KHÔNG DÂY

2.1.1 Chuẩn truyền thông là gì?

Chuẩn truyền thông (Communication Protocol) là tập hợp các quy tắc và quy ước được xây dựng nhằm đảm bảo quá trình trao đổi dữ liệu giữa các thiết bị diễn ra một cách chính xác và nhất quán, giúp các thiết bị có phần cứng hoặc nền tảng phần mềm khác nhau vẫn trao đổi thông tin hiệu quả.

Dựa trên môi trường truyền dẫn, các chuẩn truyền thông hiện nay được chia thành hai nhóm chính: truyền thông có dây (qua cáp, đạt độ tin cậy và tốc độ cao nhưng hạn chế về tính linh hoạt) và truyền thông không dây (qua sóng điện từ, linh hoạt và dễ mở rộng nhưng dễ chịu nhiễu). Bảng 2.1 so sánh chi tiết các tiêu chí kỹ thuật giữa hai loại hình này.

Đối với các mô hình nông trại nhà kính có quy mô lớn và phân bố trên nhiều khu vực khác nhau, việc lựa chọn chuẩn truyền thông phù hợp đóng vai trò rất quan trọng. Một giao thức truyền thông hiệu quả không chỉ đảm bảo dữ liệu từ các cảm biến được thu thập và truyền tải liên tục về trung tâm quản lý mà còn giúp giảm chi phí triển khai, tiết kiệm năng lượng và tạo điều kiện thuận lợi cho việc mở rộng hệ thống trong tương lai. Vì vậy, việc nghiên cứu và lựa chọn công nghệ truyền thông phù hợp là một trong những yếu tố quyết định đến hiệu quả vận hành của hệ thống quản lý nhà kính thông minh.

2.1.2 So sánh ưu, nhược điểm của truyền thông không dây và có dây

Hai loại hình truyền thông không dây và có dây đều sở hữu những đặc điểm kỹ thuật cũng như phạm vi ứng dụng khác nhau, nên việc so sánh để làm rõ thế mạnh và hạn chế riêng giữa chúng là điều cần thiết. Bảng 2.1 trình bày một cách khách quan các tiêu chí kỹ thuật cơ bản nhằm làm nổi bật sự khác biệt giữa hai loại hình. Tùy thuộc vào đặc điểm môi trường triển khai, yêu cầu hệ thống và mục tiêu vận hành, người thiết kế sẽ lựa chọn giải pháp truyền thông phù hợp với từng ngữ cảnh sử dụng.

Bảng 2.1: So sánh truyền thông có dây và không dây

Tiêu chí	Truyền thông có dây	Truyền thông không dây
Tính tin cậy	Rất cao, ít chịu ảnh hưởng từ nhiễu môi trường	Phụ thuộc môi trường; dễ bị nhiễu, suy hao, mất gói
Băng thông	Rộng, ổn định; phù hợp truyền tốc độ cao	Hạn chế hơn; dễ bị ảnh hưởng bởi nhiễu tần số và giới hạn phổ
Độ trễ	Thấp, truyền dẫn nhanh và ổn định	Có thể cao hơn do xử lý đồng bộ và nhiễu sóng
Chi phí triển khai	Cao do yêu cầu vật tư (cáp, thiết bị) và công lắp đặt	Thấp hơn, dễ triển khai trong môi trường phức tạp
Tính linh hoạt	Hạn chế do phụ thuộc cấu trúc vật lý mạng	Rất cao, dễ di chuyển, lắp đặt và mở rộng
Khả năng mở rộng	Hạn chế, phụ thuộc hạ tầng vật lý	Dễ dàng mở rộng bằng cách bổ sung thiết bị đầu cuối
Bảo trì – sửa chữa	Phức tạp nếu sự cố phần cứng hoặc đứt cáp	Đơn giản hơn, không yêu cầu xử lý vật lý nhiều
Bảo mật	Tốt hơn do khó truy cập trái phép	Dễ bị nghe lén, giả mạo nếu không mã hóa phù hợp

Chịu nhiễu môi trường	Rất tốt, tín hiệu ổn định	Kém hơn, tín hiệu dễ suy hao bởi vật cản và nhiễu điện từ
Tiêu thụ năng lượng	Thường thấp hơn	Cao hơn do thiết bị phải phát sóng liên tục

Trong quá trình xây dựng hệ thống quản lý nông trại nhà kính, một trong những yêu cầu quan trọng là đảm bảo khả năng thu thập và truyền dữ liệu từ nhiều cảm biến đặt tại các vị trí khác nhau một cách liên tục, ổn định và theo thời gian thực. Việc kết nối các thiết bị này đòi hỏi một phương thức truyền thông phù hợp với tính chất phân tán về mặt địa lý, đồng thời đảm bảo tính linh hoạt và khả năng mở rộng trong tương lai. Từ những phân tích trên, có thể thấy so với truyền thông có dây, truyền thông không dây thể hiện nhiều ưu điểm vượt trội trong bối cảnh này, đặc biệt là về chi phí triển khai, khả năng thích ứng với các điều kiện môi trường khác nhau, cũng như khả năng mở rộng hệ thống một cách dễ dàng mà không cần thay đổi hạ tầng vật lý hiện có. Do đó, truyền thông không dây là lựa chọn phù hợp và hiệu quả cho hệ thống IoT nhà kính trải dài trên nhiều khu vực.

2.1.3 Các công nghệ truyền thông không dây phổ biến

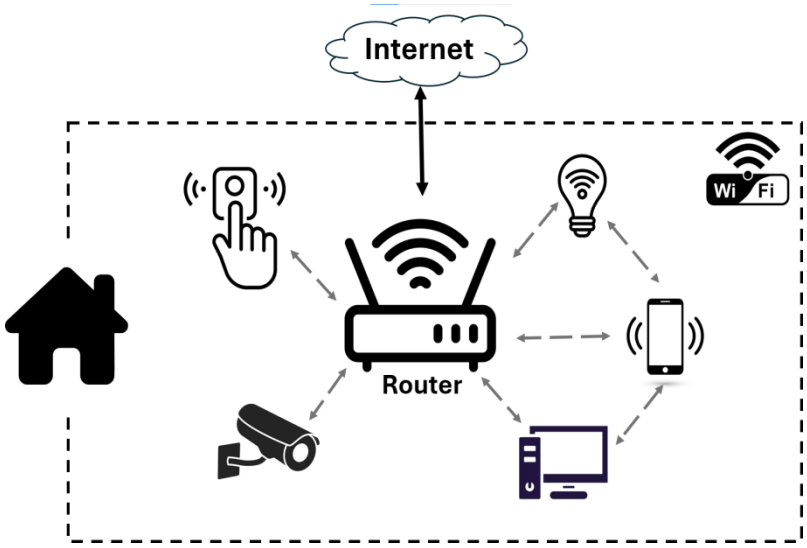
Hiện nay, với sự phát triển không ngừng của công nghệ, chúng ta đang chứng kiến sự xuất hiện của nhiều chuẩn truyền thông không dây khác nhau, mỗi công nghệ có đặc điểm riêng biệt và phù hợp với những yêu cầu sử dụng khác nhau. Việc lựa chọn một công nghệ truyền thông không dây không chỉ đơn thuần là việc tìm kiếm một kết nối ổn định, mà còn là việc cân nhắc kỹ lưỡng các yếu tố như phạm vi phủ sóng, băng thông, khả năng tiết kiệm năng lượng và yêu cầu về độ bảo mật. Tùy vào từng ứng dụng cụ thể và môi trường hoạt động, các công nghệ này sẽ đem lại những lợi ích khác nhau, và không có một chuẩn truyền thông nào là "tốt nhất" cho mọi tình huống.

Các chuẩn truyền thông không dây phổ biến hiện nay gồm Wi-Fi, Bluetooth, Zigbee, LoRa và các giao thức mạng như HTTP, MQTT, WebSocket, mỗi công nghệ có đặc điểm riêng phù hợp với từng yêu cầu ứng dụng của hệ thống quản lý nông trại nhà kính.

2.1.3.1 Các chuẩn giao tiếp trong mạng cục bộ (Device to Device)

- **WiFi (Wireless Fidelity)**

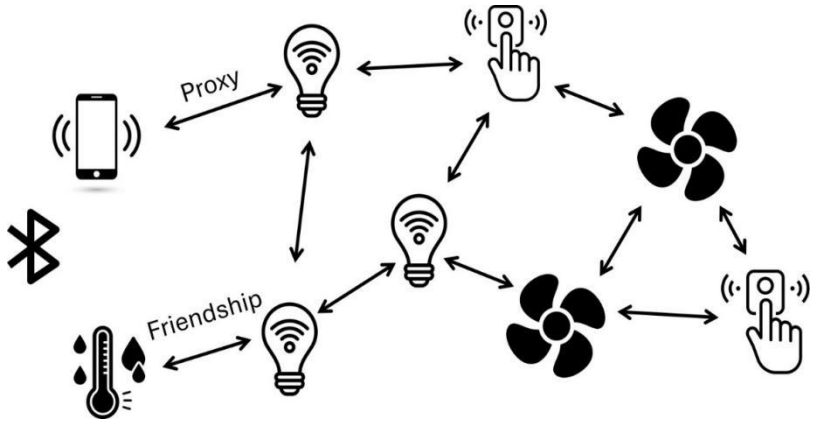
WiFi là công nghệ truyền thông không dây phổ biến được xây dựng dựa trên họ tiêu chuẩn IEEE 802.11 [5], hoạt động trên các băng tần 2.4/5/6 GHz với tốc độ truyền dữ liệu cao, phù hợp cho các thiết bị IoT cần băng thông lớn như camera giám sát. Tuy nhiên, WiFi có phạm vi hoạt động hạn chế (50-100m) và tiêu thụ năng lượng cao, chưa phù hợp với các nút cảm biến chạy pin trong thời gian dài.



Hình 2.1: Sơ đồ kết nối thiết bị trong mạng WiFi

- **Bluetooth**

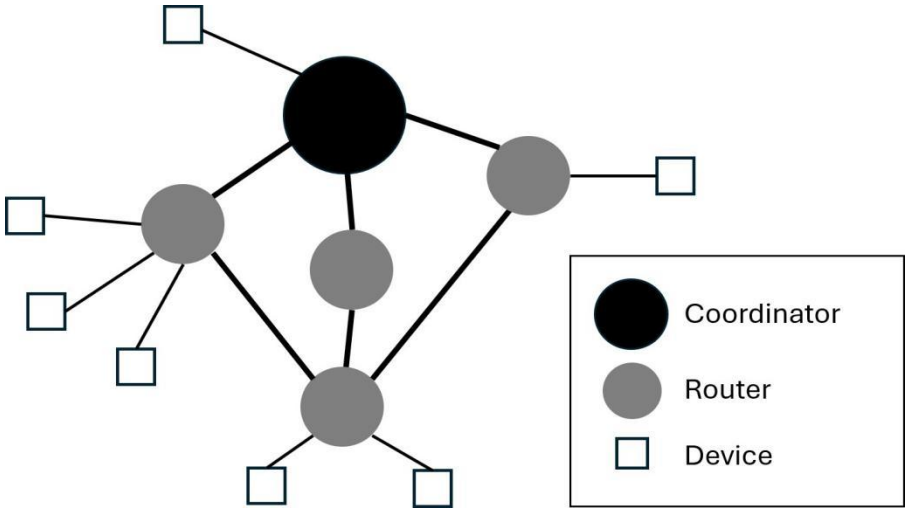
Bluetooth là công nghệ truyền thông không dây tầm ngắn, chuẩn hóa theo IEEE 802.15.1, hoạt động trên băng tần 2.4 GHz với hai nhóm chính là Bluetooth Classic và Bluetooth Low Energy (BLE). BLE tiêu thụ năng lượng rất thấp và phổ biến trong thiết bị đeo, cảm biến sức khỏe, nhưng khoảng cách truyền ngắn nên khó đáp ứng các hệ thống giám sát trải rộng trên diện tích lớn.



Hình 2.2: Sơ đồ kết nối các thiết bị trong Bluetooth Mesh

- **Zigbee**

Zigbee là công nghệ truyền thông không dây tiêu thụ năng lượng thấp, xây dựng trên tiêu chuẩn IEEE 802.15.4, hoạt động ở băng tần 2.4 GHz. Nhờ khả năng thiết lập mạng lưới (mesh) tự tổ chức và tự phục hồi (Hình 2.3), Zigbee phù hợp với hệ thống nhà thông minh có số lượng thiết bị lớn, nhưng khoảng cách truyền giữa các nút ngắn (vài chục mét) và tốc độ thấp.

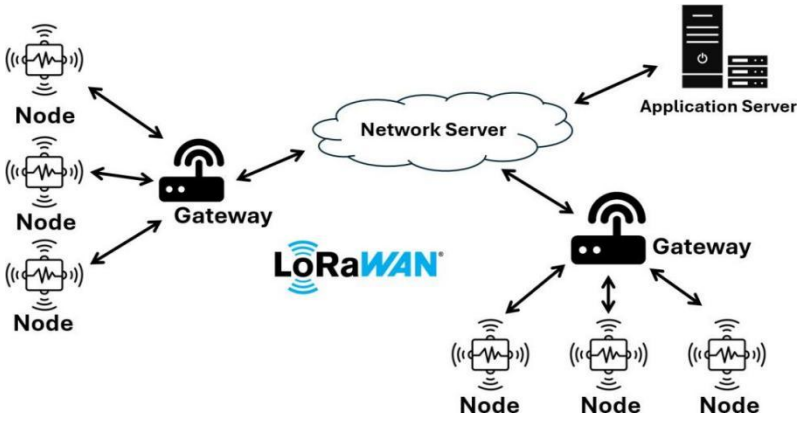


Hình 2.3: Sơ đồ kết nối các thiết bị trong mạng Zigbee theo dạng lưới

- **LoRa (Long Range Radio)**

LoRa (Long Range Radio) là công nghệ truyền thông không dây tầm xa, hiệu quả năng lượng cao, hoạt động trên băng tần sub-GHz (433/868/915 MHz tùy khu vực), phù hợp cho giao tiếp Device-to-Device trong mạng cục bộ hoặc mạng diện rộng.

LoRa sử dụng kỹ thuật điều chế Chirp Spread Spectrum (CSS) giúp truyền dữ liệu ổn định trong môi trường nhiễu, với tốc độ thấp (0.3-50 kbps) phù hợp cho dữ liệu cảm biến nhỏ. Trong giao tiếp Device-to-Device, LoRa thường được sử dụng thông qua mạng LoRaWAN[2] (Hình 2.4).



Hình 2.4: Sơ đồ kết nối các thiết bị trong mạng LoRaWAN

Cấu trúc mạng LoRaWAN được thiết kế theo mô hình ngôi sao (star topology), gồm bốn thành phần chính: Node (thu thập dữ liệu), Gateway (trung gian kết nối Internet), máy chủ mạng và máy chủ ứng dụng.

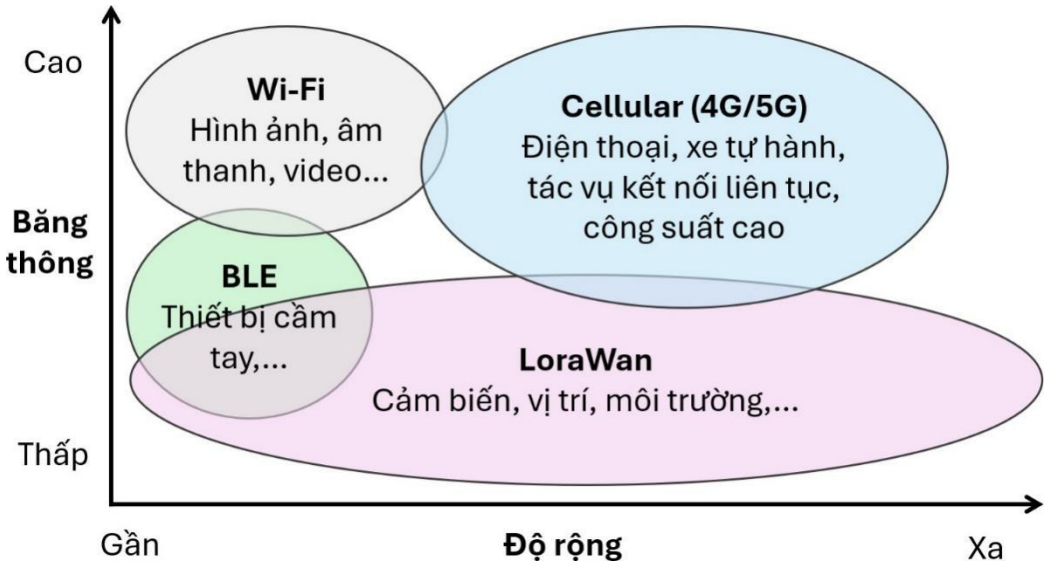
Để so sánh các đặc điểm của các công nghệ truyền thông không dây phổ biến trong giao tiếp Device-to-Device giữa các thiết bị IoT, bảng 2.2 trình bày chi tiết cung cấp cái nhìn toàn diện về ưu - nhược điểm và các mặt nổi bật của các chuẩn truyền thông, từ đó cung cấp nền tảng để lựa chọn công nghệ phù hợp với các yêu cầu cụ thể của hệ thống IoT.

Bảng 2.2: So sánh các công nghệ truyền thông không dây trong IoT

Chỉ tiêu	WiFi	Bluetooth	Zigbee	LoRa
Khoảng cách	50-100m, suy hao mạnh bởi vật cản	10-100m, suy hao mạnh	10-100m, mở rộng qua mạng lưới, dễ suy hao bởi vật cản.	Vượt trội 2-5 km (đô thị) đến 8-10km (nông thôn), lý tưởng cho IoT phân tán.
Tốc độ	Cao , phù hợp video, hình ảnh.	Trung bình (1-3 Mbps), đủ	Thấp (250 kbps), phù hợp dữ liệu nhỏ.	Rất thấp (0.3-50 kbps), chỉ phù hợp dữ liệu cảm biến.

		cho dữ liệu cảm biến.		
Điện năng	Tiêu thụ cao	Tiêu thụ thấp (BLE),	Tiêu thụ rất thấp	Tiêu thụ rất thấp
Khả năng mở rộng	Hỗ trợ hàng trăm thiết bị, cần router mạnh.	Giới hạn số thiết bị.	Hỗ trợ hàng nghìn thiết bị qua mạng lưới	Hỗ trợ hàng nghìn thiết bị qua LoRaWAN, cần cổng Gateway.
Chi phí triển khai	Cao do cần router, điểm truy cập mạnh.	Thấp, tích hợp dễ trên thiết bị tiêu dùng.	Thấp, cần hub điều khiển đơn giản.	Trung bình, yêu cầu cổng gateway.
Ứng dụng tiêu biểu	Camera giám sát, loa thông minh, nhà thông minh.	Thiết bị đeo, cảm biến y tế, nhà thông minh.	Đèn thông minh, cảm biến môi trường, công nghiệp.	Nông nghiệp thông minh, giám sát môi trường, đô thị thông minh.

Với mục tiêu xây dựng "Hệ thống quản lý nông trại nhà kính" có quy mô không gian rộng và nhiều khu vực phân tán, LoRa nổi bật là lựa chọn tối ưu so với WiFi, Bluetooth hay Zigbee nhờ khả năng truyền dữ liệu tầm xa và hiệu quả năng lượng vượt trội, cho phép các cảm biến và thiết bị điện phân bố rải rác giữa các nhà kính vẫn truyền dữ liệu ổn định dù có nhiễu sóng hoặc vật cản. Mặc dù tốc độ truyền thấp (0.3–50 kbps), điều này lại khiến LoRa lý tưởng cho dữ liệu cảm biến nhỏ — đúng đặc thù dữ liệu môi trường nhà kính. Thêm vào đó, việc hỗ trợ hàng nghìn thiết bị thông qua mạng LoRaWAN mang lại tính linh hoạt và khả năng mở rộng cao khi nông trại phát triển thêm khu vực mới.



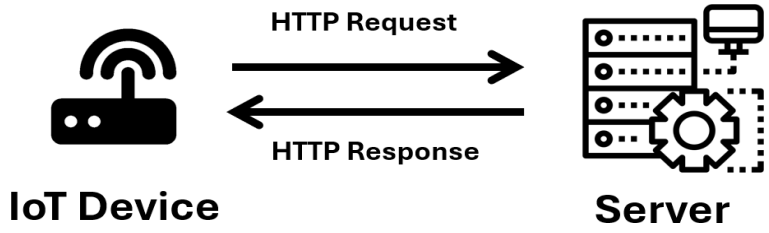
Hình 2.5: Biểu đồ so sánh băng thông và khoảng cách giữa các chuẩn truyền thông

Với hệ thống quản lý nông trại nhà kính quy mô rộng, LoRa là lựa chọn tối ưu nhờ truyền xa, hiệu quả năng lượng vượt trội, khả năng xuyên vật cản tốt và mở rộng qua LoRaWAN — cho phép các cảm biến đặt rải rác giữa các nhà kính truyền dữ liệu ổn định dù có nhiễu hoặc vật cản.

2.1.3.2 Giao tiếp giữa thiết bị IoT và Internet (Device to Network)

- **Giao thức HTTP (HyperText Transfer Protocol)**

HTTP (HyperText Transfer Protocol), giao thức nền tảng của World Wide Web, hoạt động theo mô hình client-server trên TCP/IP, thực thi dựa vào phương thức request/response (Hình 2.6). HTTP trở thành một lựa chọn phổ biến trong lĩnh vực IoT nhờ tính đơn giản và khả năng tích hợp linh hoạt. Với khả năng tương thích rộng rãi với hầu hết cơ sở hạ tầng mạng hiện có, HTTP cho phép các thiết bị dễ dàng kết nối với các dịch vụ đám mây, mang lại sự thuận tiện trong triển khai.

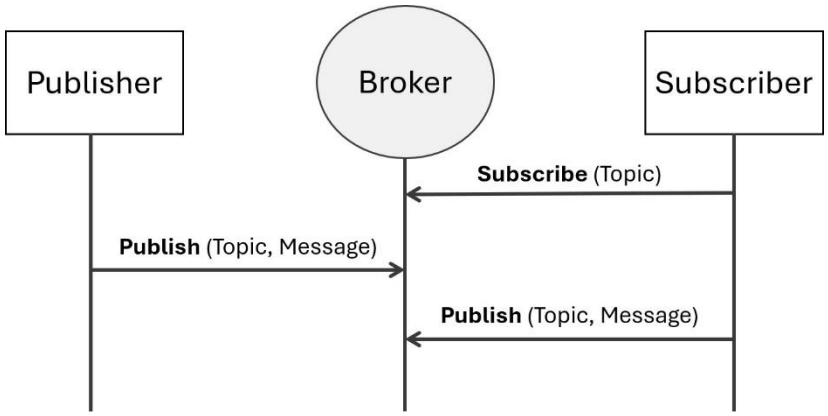


Hình 2.6: Mô hình giao tiếp HTTP

Mặc dù còn những hạn chế nhưng nhờ vào tính phổ biến và linh hoạt, HTTP vẫn hữu ích trong các tình huống như cập nhật firmware, gửi báo cáo định kỳ, tải hình ảnh/video không liên tục, hoặc tích hợp với hệ thống web thông qua API. Dù không tối ưu cho thiết bị IoT tiết kiệm năng lượng, HTTP vẫn có vai trò quan trọng trong các kịch bản phù hợp như giao tiếp giữa Website với Server,…

- **Giao thức MQTT (Message Queuing Telemetry Transport)**

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông nhẹ, được thiết kế đặc biệt cho các môi trường hạn chế về băng thông, độ trễ cao hoặc không ổn định, khiến nó trở thành lựa chọn lý tưởng cho việc truyền dữ liệu cảm biến trong IoT. Hoạt động theo mô hình xuất bản/đăng ký (publish/subscribe)[3] trên TCP/IP, MQTT cho phép thiết bị gửi và nhận dữ liệu một cách hiệu quả thông qua một trung gian gọi là broker (Hình 2.7). Trong đó, các Publisher sẽ xuất bản gói tin lên một Topic trên Broker và các Subscriber đã đăng ký Topic đó sẽ nhận được gói tin và xử lý.



Hình 2.7: Mô hình giao tiếp MQTT

Giao thức này hoạt động theo mô hình đẩy (push-based), cho phép server gửi thông báo tức thời đến thiết bị mà không cần thiết bị phải chủ động yêu cầu, hỗ trợ các ứng dụng thời gian thực như giám sát cảm biến môi trường hoặc điều khiển thiết bị từ xa.

Nhờ những ưu điểm này, MQTT được ứng dụng rộng rãi trong các hệ thống cần truyền dữ liệu cảm biến liên tục và hiệu quả, chẳng hạn như giám sát nhiệt độ, độ ẩm, hoặc chất lượng không khí trong nông nghiệp thông minh, y tế từ xa, hoặc nhà thông minh. Với sự tập trung

vào truyền dữ liệu cảm biến, MQTT không chỉ đáp ứng yêu cầu tiết kiệm năng lượng và băng thông mà còn đảm bảo tính nhanh nhạy và đáng tin cậy, khiến nó trở thành lựa chọn lý tưởng cho hệ thống IoT của bạn.

- **Giao thức WebSocket**

WebSocket là một giao thức truyền thông hai chiều trên nền TCP/IP, được thiết kế để duy trì kết nối liên tục giữa client và server. Nhờ đó, nó đặc biệt phù hợp với các ứng dụng IoT yêu cầu tương tác thời gian thực, như giám sát công nghiệp hoặc điều khiển thiết bị thông minh. Thay vì phải thiết lập lại kết nối cho mỗi lần gửi dữ liệu như HTTP, WebSocket cho phép trao đổi thông tin tức thì, giúp giảm độ trễ và tăng hiệu quả truyền tải.

Tuy nhiên, WebSocket cũng tồn tại những hạn chế đáng chú ý trong bối cảnh thiết bị IoT năng lượng thấp. Mặc dù có tiêu đề nhỏ hơn HTTP, WebSocket vẫn chưa được tối ưu cho việc truyền các gói dữ liệu siêu nhỏ chỉ vài byte. Điều này dẫn đến lãng phí băng thông trong các ứng dụng truyền tải ít và không liên tục. Quan trọng hơn, kết nối liên tục buộc thiết bị luôn ở trạng thái hoạt động, khiến không thể tận dụng chế độ ngủ tiết kiệm năng lượng – một yếu tố then chốt trong các hệ thống IoT chạy pin.

Chúng ta có thể tóm gọn các ưu - nhược điểm của các chuẩn truyền thông mạng trong bảng sau (Bảng 2.3):

Bảng 2.3: So sánh các giao thức HTTP, MQTT và WebSocket trong IoT

Chỉ tiêu	**HTTP**	**MQTT**	**WebSocket**
Phạm vi ứng dụng	Truyền dữ liệu lớn, không thường xuyên (cập nhật firmware, API web).	Truyền dữ liệu cảm biến nhỏ, tần suất cao, thời gian thực.	Truyền dữ liệu cảm biến thời gian thực, tương tác hai chiều.
Kích thước gói tin	Lớn, tối thiểu 200 bytes và không giới hạn	Cực kỳ nhỏ gọn, tối ưu cho dữ liệu cảm biến, tối thiểu 2 bytes, tối đa 256MB.	Có kích thước tối thiểu 2 bytes sau khi đã kết nối.

Tiêu thụ năng lượng	Cao do thiết lập/đóng kết nối mỗi yêu cầu, không phù hợp cho thiết bị pin.	Rất thấp, dùng cơ chế giữ kết nối, lý tưởng cho thiết bị tiết kiệm pin.	Cao hơn MQTT do duy trì kết nối liên tục, kém hiệu quả cho thiết bị pin.
Thời gian thực	Không hỗ trợ thông báo tức thời.	Hỗ trợ thông báo thời gian thực.	Tốt, hỗ trợ truyền hai chiều thời gian thực.
Băng thông	Tiêu tốn lớn do tiêu đề lớn, không hiệu quả trong mạng yếu.	Tiết kiệm, phù hợp với mạng băng thông thấp hoặc không ổn định.	Tiết kiệm hơn HTTP nhưng kém hơn MQTT trong mạng yếu.
Khả năng mở rộng	Giới hạn, yêu cầu tài nguyên lớn cho nhiều thiết bị đồng thời.	Cao, hỗ trợ hàng nghìn thiết bị qua broker, quản lý dễ dàng.	Tốt, nhưng phức tạp hơn khi quản lý nhiều kết nối liên tục.
Chi phí triển khai	Thấp, cơ sở hạ tầng web phổ biến, nhưng không tối ưu cho IoT.	Trung bình, cần broker, nhưng đơn giản cho IoT.	Trung bình, cần server hỗ trợ WebSocket, phức tạp hơn HTTP.
Ứng dụng tiêu biểu	Cập nhật firmware, tải video từ camera giám sát, tích hợp API web.	Giám sát cảm biến (nhiệt độ, độ ẩm), điều khiển thiết bị thời gian thực.	Điều khiển thiết bị thông minh, hiển thị dữ liệu cảm biến trên web.

Dựa vào các thông tin trên, chúng tôi quyết định chọn giao thức mạng MQTT để kết nối giữa Gateway và Internet trong hệ thống vì các tính năng vượt trội và phù hợp. Trước hết, MQTT có kích thước tiêu đề rất nhỏ và cơ chế hoạt động nhẹ, phù hợp với các thiết bị nhúng có tài nguyên hạn chế như ESP32, đồng thời dữ liệu gói tin của MQTT rất nhỏ, phù hợp với việc truyền dữ liệu cảm biến và các lệnh điều khiển thiết bị trong hệ thống. Ngoài ra, MQTT hoạt động theo mô hình publish/subscribe, cho phép truyền dữ liệu một cách linh hoạt, hiệu quả và tiết kiệm băng thông — đặc biệt hữu ích trong môi trường mạng không ổn định hoặc băng thông thấp. Bên cạnh đó, MQTT hỗ trợ cơ

chế QoS (Quality of Service) với nhiều mức độ tin cậy khác nhau [4], giúp đảm bảo dữ liệu được truyền tải an toàn và phù hợp với yêu cầu của hệ thống giám sát thời gian thực.

Chính nhờ những ưu điểm này, MQTT trở thành lựa chọn tối ưu cho các ứng dụng IoT hiện đại và "Hệ thống quản lý nhà kính" là một trong số đó.

2.1.4 Ứng dụng thực tiễn của các chuẩn truyền thông không dây

Trên thực tế, phần lớn các hệ thống IoT hiện nay đều kết hợp đồng thời hai hình thức giao tiếp nói trên để tận dụng ưu điểm của từng công nghệ. Các cảm biến hoặc bộ điều khiển ở tầng thiết bị thường sử dụng các kết nối tiết kiệm năng lượng như Zigbee hoặc LoRa để truyền dữ liệu về Gateway. Sau đó, Gateway thực hiện nhiệm vụ chuyển tiếp dữ liệu lên nền tảng đám mây thông qua các giao thức như MQTT hoặc CoAP trên nền mạng WiFi hoặc LTE. Trong hệ thống quản lý nhà kính thông minh, các Node cảm biến được lắp đặt tại từng khu vực sẽ sử dụng LoRa để gửi các thông số môi trường như nhiệt độ, độ ẩm, cường độ ánh sáng và chất lượng không khí về Gateway trung tâm. Tiếp đó, Gateway truyền dữ liệu đến Server thông qua MQTT, cho phép người dùng theo dõi và quản lý trên giao diện Web.

2.2 CƠ SỞ DỮ LIỆU

2.2.1 Khái niệm

Cơ sở dữ liệu (Database) là tập hợp các dữ liệu được tổ chức và lưu trữ một cách có hệ thống nhằm phục vụ cho việc quản lý, truy xuất và xử lý thông tin. Trong các hệ thống công nghệ thông tin hiện đại, cơ sở dữ liệu đóng vai trò là nơi lưu giữ toàn bộ dữ liệu cần thiết cho quá trình vận hành và phát triển ứng dụng. Nhờ khả năng quản lý dữ liệu hiệu quả, cơ sở dữ liệu được ứng dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, giáo dục, quản lý nhân sự, tài chính, ngân hàng và y tế.

Trong lĩnh vực Internet vạn vật (IoT), cơ sở dữ liệu giữ vai trò đặc biệt quan trọng do phải tiếp nhận và lưu trữ lượng lớn dữ liệu được tạo ra liên tục từ các cảm biến và thiết bị đầu cuối. Đối với đề tài xây dựng hệ thống quản lý nông trại nhà kính, cơ sở dữ

liệu được sử dụng để lưu trữ thông tin môi trường, trạng thái hoạt động của thiết bị và các dữ liệu phục vụ quá trình giám sát, điều khiển từ xa.

Hệ thống cần có khả năng tiếp nhận dữ liệu đồng thời từ nhiều cảm biến được lắp đặt tại các nhà kính khác nhau. Các thông số như nhiệt độ, độ ẩm, cường độ ánh sáng hoặc chất lượng không khí được gửi về máy chủ theo chu kỳ định sẵn. Vì vậy, cơ sở dữ liệu phải đáp ứng được các yêu cầu về hiệu năng xử lý, tính ổn định và khả năng mở rộng nhằm đảm bảo dữ liệu luôn được lưu trữ đầy đủ và chính xác.

Ví dụ, cảm biến nhiệt độ trong nhà kính có thể gửi dữ liệu định kỳ vài phút một lần. Mỗi bản ghi được lưu trữ bao gồm thời gian đo, giá trị cảm biến, vị trí lắp đặt, khu vực nhà kính và trạng thái hoạt động của thiết bị. Từ những dữ liệu này, hệ thống có thể xây dựng biểu đồ theo dõi, phân tích xu hướng biến đổi môi trường và phát hiện các bất thường để đưa ra cảnh báo kịp thời.

2.2.2 Các vai trò của Cơ sở dữ liệu

Cơ sở dữ liệu là thành phần cốt lõi của hệ thống, đảm nhiệm các vai trò: truy xuất và thống kê dữ liệu nhanh chóng, lưu trữ có cấu trúc kèm cơ chế sao lưu/phục hồi, hỗ trợ thao tác thêm/sửa/xóa và phân quyền truy cập, bảo mật thông tin, hạn chế dữ liệu dư thừa, và khả năng mở rộng khi khối lượng dữ liệu tăng.

2.2.3 Phân loại

Hiện nay, cơ sở dữ liệu được chia thành hai nhóm chính gồm cơ sở dữ liệu quan hệ (Relational Database) và cơ sở dữ liệu phi quan hệ (NoSQL Database).

- **Cơ sở dữ liệu quan hệ (RDBMS - Relational Database Management System)**

Cơ sở dữ liệu quan hệ là mô hình lưu trữ dữ liệu dựa trên các bảng có mối liên kết với nhau. Trong mỗi bảng, dữ liệu được tổ chức thành các hàng và cột, trong đó mỗi hàng biểu diễn một bản ghi riêng biệt và mỗi cột mô tả một thuộc tính của dữ liệu. Để phân biệt các bản ghi, hệ thống sử dụng khóa chính (Primary Key), đồng thời thiết lập mối quan hệ giữa các bảng thông qua khóa ngoại (Foreign Key).

Ngôn ngữ SQL (Structured Query Language) được sử dụng để thực hiện các thao tác quản lý dữ liệu như truy vấn, thêm mới, chỉnh sửa và xóa dữ liệu. Nhờ khả năng đảm

bảo tính nhất quán và toàn vẹn dữ liệu, cơ sở dữ liệu quan hệ được sử dụng phổ biến trong các hệ thống quản lý doanh nghiệp, thương mại điện tử, tài chính và ngân hàng.

Một số hệ quản trị cơ sở dữ liệu quan hệ được sử dụng rộng rãi hiện nay gồm MySQL, PostgreSQL và Microsoft SQL Server.

- **Cơ sở dữ liệu phi quan hệ (NoSQL)**

NoSQL là nhóm cơ sở dữ liệu không sử dụng mô hình bảng quan hệ truyền thống. Dữ liệu có thể được lưu trữ dưới nhiều dạng khác nhau như tài liệu (Document), cặp khóa – giá trị (Key-Value), cột rộng (Wide Column) hoặc đồ thị (Graph). Đặc điểm nổi bật của NoSQL là không yêu cầu cấu trúc dữ liệu cố định và có khả năng mở rộng linh hoạt theo nhu cầu thực tế.

Loại cơ sở dữ liệu này đặc biệt phù hợp với các hệ thống có lượng dữ liệu lớn, tốc độ tăng trưởng cao hoặc yêu cầu xử lý dữ liệu theo thời gian thực. Các ứng dụng phổ biến của NoSQL bao gồm mạng xã hội, hệ thống Big Data, nền tảng phân tích dữ liệu và các dịch vụ trực tuyến có số lượng người dùng lớn.

Một số hệ quản trị NoSQL được sử dụng rộng rãi hiện nay gồm MongoDB, Cassandra và Redis. Nhờ khả năng mở rộng tốt và hiệu năng cao, NoSQL đang trở thành lựa chọn phù hợp cho nhiều hệ thống hiện đại có quy mô lớn[6].

Trong quá trình thiết kế hệ thống lưu trữ dữ liệu cho các ứng dụng hiện đại, đặc biệt là trong lĩnh vực IoT, việc lựa chọn loại cơ sở dữ liệu phù hợp đóng vai trò quan trọng. Hai kiểu cơ sở dữ liệu phổ biến hiện nay là cơ sở dữ liệu quan hệ (Relational Database) và cơ sở dữ liệu không quan hệ (NoSQL – Non-relational Database). Mỗi loại đều có những ưu điểm và hạn chế riêng, tùy thuộc vào yêu cầu cụ thể của hệ thống như cấu trúc dữ liệu, khả năng mở rộng, hiệu năng truy vấn và tính linh hoạt. Bảng 2.4 dưới đây trình bày sự so sánh giữa hai kiểu cơ sở dữ liệu này trên một số khía cạnh quan trọng, giúp làm rõ cơ sở lý luận trong việc lựa chọn giải pháp lưu trữ dữ liệu cho dự án [7].

Bảng 2.4: So sánh hai kiểu cơ sở dữ liệu quan hệ và không quan hệ

	Cơ sở dữ liệu quan hệ	**Cơ sở dữ liệu không quan hệ**

Cấu trúc dữ liệu	Cơ sở dữ liệu quan hệ lưu trữ dữ liệu trong các bảng có liên quan bằng khóa chính và khóa ngoài.	Cơ sở dữ liệu NoSQL có thể lưu trữ dữ liệu ở các định dạng khác nhau, chẳng hạn như tài liệu, biểu đồ hoặc cặp khóa-giá trị.
Tính linh hoạt	Biểu thị dữ liệu dưới dạng bảng, hàng và cột.	Biểu thị dữ liệu dưới dạng biểu đồ, các cặp khóa - giá trị và nhiều hơn thế.
Khả năng mở rộng	Cơ sở dữ liệu quan hệ yêu cầu một lược đồ cố định và nghiêm ngặt phải được tuân thủ nghiêm ngặt.	Cơ sở dữ liệu NoSQL rất linh hoạt và có thể dễ dàng thích ứng với những thay đổi trong yêu cầu của dự án.
Ngôn ngữ truy vấn	Cơ sở dữ liệu quan hệ lý tưởng cho các truy vấn phức tạp liên quan đến nhiều bộ dữ liệu	Cơ sở dữ liệu NoSQL hiệu quả hơn đối với các truy vấn đơn giản và truy cập nhanh vào dữ liệu cụ thể.
Phần mềm	MySQL, Oracle, MS-SQL.	MongoDB, Cassandra, HBase, CouchDB.

Trong các dự án IoT, dữ liệu được thu thập từ nhiều loại cảm biến và thiết bị khác nhau. Mỗi thiết bị có thể tạo ra các tập dữ liệu với cấu trúc, số lượng trường thông tin và tần suất gửi dữ liệu không giống nhau. Điều này đặt ra yêu cầu về một giải pháp lưu trữ có khả năng thích nghi tốt với sự thay đổi của dữ liệu trong quá trình vận hành hệ thống.

So với cơ sở dữ liệu quan hệ truyền thống, vốn yêu cầu cấu trúc bảng và lược đồ dữ liệu được xác định trước, các hệ quản trị cơ sở dữ liệu NoSQL mang lại tính linh hoạt cao hơn trong việc lưu trữ dữ liệu không đồng nhất. Vì lý do đó, đề tài lựa chọn MongoDB làm hệ quản trị cơ sở dữ liệu cho hệ thống quản lý nông trại nhà kính.

MongoDB là cơ sở dữ liệu NoSQL thuộc nhóm document-oriented, lưu trữ dữ liệu dưới dạng BSON (Binary JSON). Mỗi tài liệu (document) có thể chứa các trường dữ liệu khác nhau mà không cần thay đổi cấu trúc của toàn bộ cơ sở dữ liệu. Đặc điểm này đặc biệt phù hợp với các hệ thống IoT, nơi số lượng cảm biến và loại dữ liệu có thể thay đổi theo từng giai đoạn triển khai.

Bên cạnh tính linh hoạt về cấu trúc dữ liệu, MongoDB còn cung cấp khả năng ghi và truy xuất dữ liệu với hiệu suất cao. Trong hệ thống nhà kính thông minh, các Node cảm biến liên tục gửi các thông số môi trường như nhiệt độ, độ ẩm không khí, độ ẩm đất, cường độ ánh sáng và chất lượng không khí về máy chủ. MongoDB cho phép lưu trữ và xử lý các dữ liệu này một cách hiệu quả, đáp ứng yêu cầu giám sát gần thời gian thực của hệ thống.

Ngoài ra, MongoDB hỗ trợ các cơ chế quản lý dữ liệu tự động như tạo chỉ mục (Indexing), phân mảnh dữ liệu (Sharding) và thiết lập thời gian tồn tại của bản ghi (TTL Index). Nhờ đó, hệ thống có thể tự động loại bỏ những dữ liệu không còn cần thiết sau một khoảng thời gian xác định, giúp tối ưu dung lượng lưu trữ và đơn giản hóa quá trình quản trị cơ sở dữ liệu.

Một ưu điểm khác của MongoDB là khả năng mở rộng theo chiều ngang, cho phép hệ thống duy trì hiệu năng khi số lượng nhà kính, cảm biến và thiết bị tăng lên trong tương lai. Đồng thời, MongoDB có thể tích hợp dễ dàng với các công nghệ được sử dụng trong đề tài như Node.js, MQTT và các thiết bị IoT sử dụng ESP32, giúp rút ngắn thời gian phát triển và triển khai hệ thống.

Với các ưu điểm về tính linh hoạt, hiệu suất xử lý, khả năng mở rộng và sự phù hợp với đặc thù dữ liệu IoT, MongoDB được lựa chọn làm giải pháp lưu trữ dữ liệu cho hệ thống quản lý nông trại nhà kính trong đề tài này.

CHƯƠNG 3
THIẾT KẾ HỆ THỐNG

3.1 YÊU CẦU CHUNG CỦA HỆ THỐNG

Hệ thống được thiết kế nhằm giám sát và điều khiển các thiết bị điện trên phạm vi rộng theo thời gian thực, đồng thời cung cấp khả năng phân tích dữ liệu trực quan và quản lý hiệu quả. Dữ liệu được thu thập từ các cảm biến môi trường như nhiệt độ, độ ẩm, ánh sáng tại nhiều khu vực cách xa nhau.

Đồng thời, dữ liệu cảm biến được truyền về một nền tảng web với giao diện trực quan, hỗ trợ điều khiển từ xa thiết bị điện (bật/tắt) và phản hồi trạng thái tức thời. Hệ thống hỗ trợ lập lịch tự động (ví dụ: bật đèn lúc 18:00) cũng như điều khiển theo điều kiện môi trường (ví dụ: bật quạt khi nhiệt độ > 30°C).

Về mặt kỹ thuật, hệ thống đảm bảo kết nối ổn định trong môi trường có nhiễu hoặc vật cản, đồng thời hỗ trợ mở rộng lên hàng nghìn node mà không ảnh hưởng đáng kể đến hiệu suất. Các node được thiết kế tiết kiệm năng lượng và hoạt động độc lập, tránh gây xung đột lẫn nhau.

Thêm vào đó, Website cần phải có khả năng hỗ trợ đa nền tảng (Laptop, điện thoại...), hiển thị dữ liệu thời gian thực dưới dạng biểu đồ và lưu trữ trong cơ sở dữ liệu tối thiểu 30 ngày.

3.2 CHỨC NĂNG KỸ THUẬT CỦA HỆ THỐNG

Từ yêu cầu của người dùng, hệ thống được phân tích và mô tả qua cấu trúc gồm bốn thành phần tích hợp với nhau, đáp ứng giám sát, điều khiển IoT thời gian thực trên phạm vi rộng 2-8 km, truyền dữ liệu cảm biến nhỏ, phân tích trực quan, và quản lý hiệu quả cho nông nghiệp thông minh, giám sát môi trường và đại đa số hệ thống dữ liệu phân tán.

- Node: Đặt tại nhiều khu vực cách xa nhau để thu thập dữ liệu cảm biến (nhiệt độ, độ ẩm, ánh sáng) với các thông số kỹ thuật:

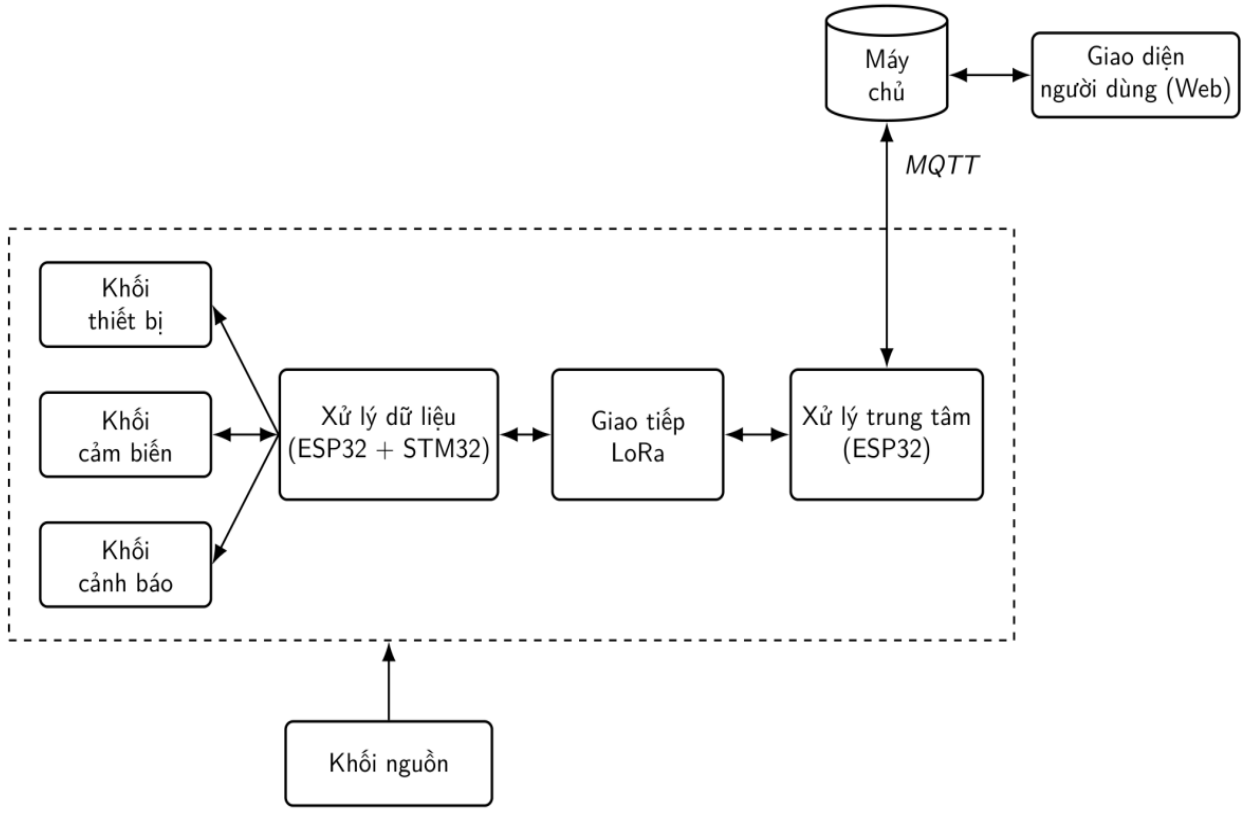
- Vi điều khiển: CPU tốc độ 240 MHz, SRAM 520 KB, độ trễ xử lý <100 ms cho gói tin LoRa (tổng độ trễ <1 giây). Tốc độ xử lý 50–100 MHz, RAM 10 KB–4 MB, tối ưu năng lượng cho xử lý dữ liệu và điều khiển, có khả năng giao tiếp nhiều chuẩn khác nhau như UART, SPI, I2C và hỗ trợ đọc dữ liệu ADC.
- Cảm biến: Nhiệt độ (±0.5°C, -40°C đến 85°C), độ ẩm (±2%, 0–100% RH), ánh sáng (0–65535 lux).
- Thiết bị điện: Rơ-le, tải tối đa 10A, 220V, hỗ trợ điều khiển đèn, quạt từ xa.
- Nguồn điện: Linh hoạt 3.3–12V, dòng điện 2A.
- Giao thức không dây: Băng tần sub-GHz, công suất 20 dBm, phạm vi 2–8 km, gói tin 6–20 byte.
- Bộ định danh: Phân biệt các Node trong cùng mạng LoRaWAN với nhau.

- Gateway: Đặt tại trung tâm, đảm bảo truyền dữ liệu ổn định trong môi trường nhiễu, với các thông số kỹ thuật:
  - Bộ xử lý: CPU tốc độ 240 MHz, SRAM 520 KB, độ trễ xử lý <100 ms cho gói tin LoRa (tổng độ trễ <1 giây).
  - Giao thức không dây: Có khả năng kết nối được nhiều Node và với khoảng cách 2-8km.
  - Kết nối mạng: Có khả năng kết nối Internet thông qua WiFi, LTE hoặc Sim khi được tích hợp Module.
  - Giao thức: MQTT 3.1.1, hỗ trợ QoS 0–2, truyền dữ liệu cảm biến đến server qua Wi-Fi hoặc 4G.
  - Nguồn điện: 3.3V - 12V DC, tiêu thụ 500mA–2A.
  - Chức năng bổ sung: Tự động khôi phục kết nối Internet, MQTT Server, có thông báo khi phát sinh lỗi và gửi lại gói tin khi không thể truyền đi.
- Server: Lưu trữ và xử lý dữ liệu, cung cấp API cho website, với các thông số kỹ thuật:

- Cơ sở dữ liệu: lưu trữ 30 ngày, truy vấn <1s cho 1000 bản ghi.
- Giao tiếp: API RESTful, xử lý JSON qua MQTT và HTTP.
- Website: Giao diện đa nền tảng, hiển thị dữ liệu trực quan và điều khiển từ xa, với các tính năng:
- Giao diện: Framework JavaScript, tương thích 320x480px đến 3840x2160px.
- Hiển thị: Biểu đồ cảm biến.
- Điều khiển: Cấu hình lịch trình, điều kiện bật tắt cho thiết bị.

3.3 SƠ ĐỒ KHỐI HỆ THỐNG

Dựa vào yêu cầu chung và chức năng kỹ thuật mong muốn của hệ thống, nhóm nghiên cứu đã chia hệ thống "Quản lý nông trại nhà kính" thành các khối riêng biệt (Hình 3.1) với các chức năng, vai trò riêng và kết nối với nhau để tạo thành một hệ thống hoàn chỉnh nhằm đáp ứng nhu cầu giám sát và điều khiển IoT thời gian thực trong phạm vi từ 2 đến 8 km.



Hình 3.1: Sơ đồ khối hệ thống Quản lý nông trại nhà kính

Hệ thống được xây dựng gồm ba thành phần chính: khối phần cứng, khối cơ sở dữ liệu và khối giao diện người dùng. Các thành phần này phối hợp với nhau nhằm thực hiện chức năng thu thập dữ liệu, giám sát và điều khiển thiết bị từ xa trong môi trường nhà kính.

Khối phần cứng bao gồm một Gateway trung tâm và nhiều Node được triển khai tại các khu vực nhà kính. Mỗi Node được trang bị các cảm biến môi trường như cảm biến nhiệt độ, độ ẩm và cường độ ánh sáng để thu thập dữ liệu thực tế. Ngoài ra, Node còn có khả năng kết nối với các thiết bị chấp hành như relay, quạt thông gió, hệ thống chiếu sáng hoặc các thiết bị điện khác nhằm phục vụ nhu cầu điều khiển tự động. Dữ liệu đo được từ cảm biến sẽ được truyền không dây về Gateway. Gateway đóng vai trò là trung tâm trung gian, thực hiện việc tiếp nhận dữ liệu từ các Node, gửi dữ liệu lên máy chủ thông qua kết nối Internet, đồng thời nhận các lệnh điều khiển từ hệ thống quản lý và chuyển tiếp đến đúng Node tương ứng. Nhờ đó, quá trình giám sát và điều khiển được thực hiện liên tục với độ trễ thấp.

Khối cơ sở dữ liệu chịu trách nhiệm lưu trữ và quản lý toàn bộ thông tin của hệ thống. Các dữ liệu được lưu trữ bao gồm thông tin người dùng, Gateway, Node cảm biến, thiết bị điều khiển cũng như dữ liệu môi trường được thu thập theo thời gian thực. Khối này không chỉ đảm nhiệm chức năng lưu trữ mà còn hỗ trợ xử lý dữ liệu, cung cấp thông tin cho giao diện hiển thị và duy trì sự liên kết giữa các thành phần trong hệ thống. Bên cạnh đó, cơ sở dữ liệu còn là nơi tiếp nhận các yêu cầu điều khiển từ người dùng, sau đó chuyển đến Gateway để thực hiện điều khiển thiết bị tại các Node tương ứng, đảm bảo tính đồng bộ và chính xác trong quá trình vận hành.

Khối giao diện người dùng được xây dựng dưới dạng ứng dụng Web, cho phép truy cập trên nhiều nền tảng khác nhau như máy tính, máy tính bảng và điện thoại thông minh. Thông qua giao diện này, người dùng có thể theo dõi các thông số môi trường và trạng thái hoạt động của thiết bị theo thời gian thực dưới dạng bảng dữ liệu hoặc biểu đồ trực quan. Hệ thống cũng hỗ trợ lưu trữ lịch sử hoạt động, giúp người dùng tra cứu dữ liệu cũ, đánh giá xu hướng biến động của môi trường và phát hiện các bất thường trong quá trình vận hành. Ngoài chức năng giám sát, giao diện còn cung cấp khả năng điều khiển

thiết bị từ xa thông qua các thao tác trực tiếp hoặc thiết lập các kịch bản tự động. Ví dụ, người dùng có thể cài đặt lịch bật/tắt thiết bị theo thời gian hoặc xây dựng các điều kiện hoạt động dựa trên dữ liệu cảm biến, chẳng hạn tự động kích hoạt quạt thông gió khi nhiệt độ vượt quá ngưỡng cho phép. Nhờ đó, hệ thống góp phần nâng cao mức độ tự động hóa, giảm sự can thiệp thủ công và tối ưu hiệu quả quản lý nhà kính.

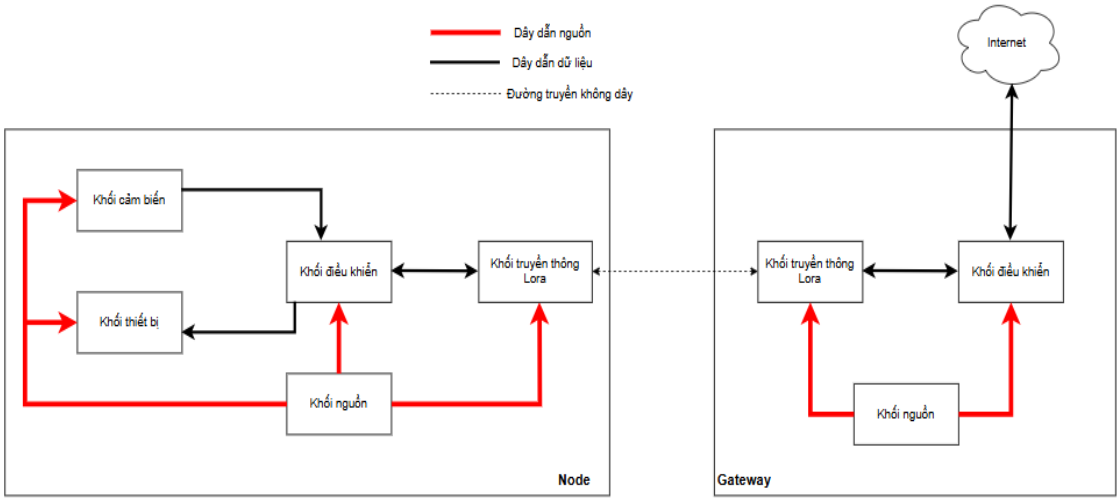
3.3.1 Hệ thống phần cứng

3.3.1.1 Yêu cầu kỹ thuật

Theo yêu cầu chung của hệ thống thu thập, truyền dữ liệu về nơi xử lý trung tâm thì phần cứng cần phải có các chức năng như sau:

- Đọc dữ liệu từ các cảm biến.
- Bật tắt thủ công thiết bị điện hoặc hẹn lịch bật tắt.
- Có thể truyền dữ liệu đi xa.
- Thiết bị có kết nối WiFi, LTE.
- Gửi dữ liệu, trạng thái lên Server Backend: 1 phút gửi dữ liệu cảm biến một lần.

3.3.1.2 Sơ đồ khối phần cứng



Hình 3.2: Sơ đồ khối phần cứng

Hệ thống phần cứng của đề tài được thiết kế gồm hai thành phần chính là Node và Gateway (Hình 3.2). Mỗi thành phần được cấu thành từ nhiều khối chức năng khác nhau, phối hợp với nhau để thực hiện nhiệm vụ thu thập dữ liệu, truyền thông và điều khiển thiết bị trong hệ thống quản lý nhà kính.

A. Khối Node

Node là thiết bị được lắp đặt tại từng khu vực nhà kính, có chức năng thu thập dữ liệu môi trường và thực hiện điều khiển các thiết bị điện tại chỗ. Cấu trúc của Node bao gồm năm khối chức năng chính như sau:

- Khối cảm biến: Khối này có nhiệm vụ thu thập các thông số môi trường như nhiệt độ, độ ẩm, cường độ ánh sáng và các đại lượng khác tùy theo yêu cầu của hệ thống. Các cảm biến có thể sử dụng nhiều chuẩn giao tiếp khác nhau như Analog, Digital, UART hoặc I2C. Dữ liệu sau khi được đo sẽ được chuyển đến vi điều khiển để xử lý.
- Khối thiết bị: Đây là khối thực hiện các tác động điều khiển lên môi trường thông qua các thiết bị điện như relay, quạt thông gió, đèn chiếu sáng hoặc máy bơm nước. Tín hiệu điều khiển được tạo ra từ các chân xuất tín hiệu số của vi điều khiển.
- Khối nguồn: Cung cấp năng lượng cho toàn bộ mạch Node. Tùy theo thiết kế phần cứng, nguồn nuôi có thể được lấy từ bộ chuyển đổi AC-DC, pin sạc hoặc hệ thống năng lượng mặt trời. Ngoài ra, khối này còn tích hợp các mạch ổn áp nhằm đảm bảo điện áp đầu ra phù hợp với yêu cầu hoạt động của vi điều khiển và cảm biến.
- Khối truyền thông LoRa: Khối này chịu trách nhiệm truyền nhận dữ liệu giữa Node và Gateway. Tùy thuộc vào yêu cầu triển khai, hệ thống có thể sử dụng các chuẩn giao tiếp như LoRa, RF hoặc UART. Sau khi được xử lý tại Node, dữ liệu sẽ được đóng gói thành các gói tin và gửi đến Gateway thông qua khối truyền thông này.
- Khối vi điều khiển (MCU): Đây là thành phần trung tâm của Node, thực hiện các chức năng đọc dữ liệu cảm biến, xử lý thuật toán điều khiển, quản lý thiết bị chấp

hành và đóng gói dữ liệu truyền thông. Đồng thời, MCU cũng tiếp nhận các lệnh điều khiển từ Gateway để thực hiện các thao tác tương ứng trên thiết bị điện.

B. Khối Gateway

Gateway đóng vai trò là thiết bị trung gian giữa mạng Node và máy chủ quản lý. Thiết bị này tiếp nhận dữ liệu từ các Node, xử lý và chuyển tiếp dữ liệu lên Server thông qua kết nối Internet. Đồng thời, Gateway cũng tiếp nhận các lệnh điều khiển từ Server và gửi đến các Node tương ứng. Cấu trúc Gateway gồm các khối chức năng sau:

- Khối truyền thông LoRa: Khối này thực hiện giao tiếp hai chiều với các Node trong hệ thống. Dữ liệu thu thập từ các Node được tiếp nhận và chuyển đến vi điều khiển để xử lý. Ngược lại, các lệnh điều khiển từ Server cũng được truyền đến Node thông qua khối này.
- Khối nguồn: Có chức năng cung cấp điện năng cho toàn bộ mạch Gateway. Nguồn nuôi có thể được cấp từ bộ chuyển đổi điện áp ngoài, cổng USB hoặc các nguồn DC phù hợp với yêu cầu thiết kế.
- Khối vi điều khiển (MCU): Đây là bộ phận xử lý trung tâm của Gateway. MCU tiếp nhận và giải mã các gói dữ liệu từ Node, sau đó truyền dữ liệu lên Server thông qua các giao thức mạng sử dụng kết nối WiFi, Ethernet hoặc LTE. Bên cạnh đó, MCU còn tiếp nhận các lệnh điều khiển từ Server, đóng gói lại theo định dạng truyền thông nội bộ và gửi đến Node tương ứng để thực hiện điều khiển các thiết bị điện trong hệ thống.

3.3.1.3 Đặc tả kỹ thuật các khối

A. Khối cảm biến - Sensors:

Khối cảm biến có vai trò thu thập các thông số môi trường để phục vụ việc giám sát và điều khiển trong hệ thống. Hệ thống tạo ra với mục đích nền tảng, vì thế thay vì thiết kế cảm biến đặc trưng cho từng lĩnh vực cụ thể, hệ thống lựa chọn các loại cảm biến phổ biến, dễ tích hợp và phù hợp với nhiều ứng dụng thực tiễn. Một số cảm biến được lựa chọn như:

- Cảm biến nhiệt độ và độ ẩm (DHT11): Được sử dụng để giám sát môi trường trong nhà, nhà kính, hoặc khu vực cần kiểm soát nhiệt – ẩm.
- Cảm biến ánh sáng (LDR): Phục vụ cho các ứng dụng điều khiển chiếu sáng thông minh, tự động bật/tắt đèn theo cường độ sáng môi trường.

B. Khối thiết bị – Devices:

Khối Devices là một thành phần quan trọng trong hệ thống phần cứng, chịu trách nhiệm điều khiển các thiết bị điện gia dụng như đèn chiếu sáng, quạt, máy bơm, thiết bị tưới tiêu, ổ cắm điện, v.v. Việc điều khiển được thực hiện thông qua các mạch relay đóng/ngắt dòng điện cấp cho thiết bị (Hình 3.3).



Hình 3.3: Hình ảnh thực tế module Relay kích 4 cổng

Các relay này được điều khiển bởi vi điều khiển (MCU2) thông qua các chân Digital Output. Tín hiệu từ MCU2 có thể được kích hoạt theo nhiều phương thức điều khiển khác nhau, đảm bảo tính linh hoạt và đáp ứng các nhu cầu sử dụng thực tế. Cụ thể:

- Điều khiển thủ công: Người dùng có thể bật/tắt thiết bị trực tiếp thông qua giao diện người dùng trên trình duyệt web hoặc ứng dụng di động. Lệnh điều khiển được gửi đến Gateway, sau đó được truyền xuống Node và thực thi bởi MCU để kích hoạt relay tương ứng.
- Điều khiển tự động theo cảm biến: Hệ thống có thể cấu hình tự động bật/tắt thiết bị dựa vào giá trị cảm biến thu thập được. Ví dụ: nếu nhiệt độ môi trường

vượt ngưỡng định sẵn, hệ thống tự động bật quạt hoặc điều hòa; nếu độ ẩm đất thấp, hệ thống kích hoạt máy bơm tưới cây.

- Điều khiển theo lịch trình: Thiết bị cũng có thể hoạt động theo lịch trình đã thiết lập từ trước (ví dụ: bật đèn lúc 18h và tắt lúc 22h), phù hợp với nhu cầu tự động hóa trong gia đình hoặc trong các ứng dụng nông nghiệp thông minh.

Khối Devices có thể mở rộng để điều khiển nhiều thiết bị cùng lúc, mỗi thiết bị được quản lý bởi một kênh relay riêng biệt. Ngoài ra, để đảm bảo an toàn và tính ổn định khi vận hành với tải lớn (ví dụ: động cơ, máy bơm công suất cao), các relay cần có dòng chịu tải phù hợp và có thể kết hợp với opto cách ly nhằm hạn chế nhiễu điện.

Việc thiết kế khối này hướng đến sự đơn giản, dễ mở rộng, và khả năng tùy biến cao – từ điều khiển một vài thiết bị cơ bản đến quản lý hệ thống phức tạp với hàng chục thiết bị hoạt động song song.

C. Khối Vi điều khiển tại Node:

Khối MCU là trung tâm xử lý của mỗi Node trong hệ thống. Tại đây, vi điều khiển thực hiện các chức năng chính như:

- Đọc dữ liệu được thu thập từ các cảm biến.
- Xử lý tín hiệu và thực hiện các logic điều khiển thiết bị.
- Đóng gói dữ liệu và truyền về Gateway thông qua khối giao truyền thông LoRa.
- Nhận dữ liệu từ khối truyền thông LoRa và thực thi yêu cầu.

Để thực hiện được các tác vụ trên một cách hiệu quả và ổn định, việc lựa chọn vi điều khiển phù hợp là rất quan trọng. Dưới đây, bảng 3.1 sẽ so sánh một số vi điều khiển phổ biến có thể dùng cho hệ thống:

Bảng 3.1: So sánh một số vi điều khiển phổ biến

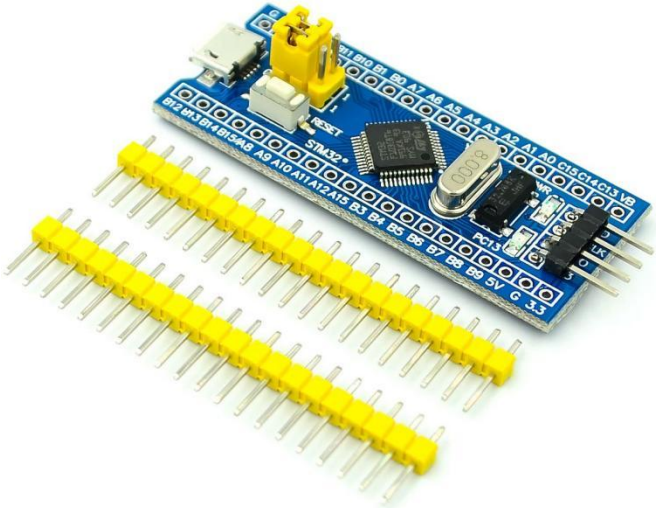
Vi điều khiển	Tốc độ xử lý	Số chân I/O	Giao tiếp hỗ trợ	Giá thành
ATmega328P	16 MHz	23	UART, I2C, SPI	Trung bình
ESP8266	80–160 MHz	17	UART, SPI, WiFi	Rẻ
ESP32	240 MHz	>30	UART, I2C, SPI, WiFi, BLE	Trung bình
STM32F103C8T6	72 MHz	>37	UART, I2C, SPI, PWM, ADC	Rẻ

Trong quá trình lựa chọn vi điều khiển cho Node, nhóm đã xem xét bốn dòng vi điều khiển phổ biến: ATmega328P, ESP8266, ESP32 và STM32F103C8T6. ATmega328P ổn định, dễ lập trình nhưng tốc độ và khả năng giao tiếp hạn chế. ESP8266 có WiFi và chi phí thấp nhưng ít chân I/O và bộ nhớ nhỏ. ESP32 mạnh với 240 MHz lõi kép, tích hợp WiFi/BLE và nhiều giao thức (UART, SPI, I2C, ADC), phù hợp cho đọc cảm biến và truyền thông không dây, nhưng số chân GPIO hạn chế và tính thời gian thực không tuyệt đối khi điều khiển nhiều tải công suất. STM32F103C8T6 (ARM Cortex-M3, 72 MHz) hỗ trợ đầy đủ UART, SPI, I2C, ADC, PWM với >37 chân I/O, lý tưởng để điều khiển nhiều thiết bị thời gian thực, nhưng không có truyền thông không dây tích hợp.

Vì không một MCU đơn lẻ nào đáp ứng trọn vẹn cả hai yêu cầu (đọc cảm biến + giao tiếp LoRa với Gateway, và điều khiển ổn định nhiều thiết bị điện), nhóm chọn kiến trúc lai ESP32 (Hình 3.4) + STM32F103C8T6 (Hình 3.5) kết nối qua UART. ESP32 đảm nhận lớp cảm biến và truyền thông — đọc các cảm biến môi trường, điều khiển module LoRa, đồng thời dùng WiFi tích hợp làm phương án dự phòng. STM32F103C8T6 đảm nhận lớp điều khiển thiết bị — nhận lệnh từ ESP32 và đóng/ngắt các thiết bị điện (đèn, máy bơm, quạt) qua GPIO, đảm bảo thời gian thực và độ tin cậy cho thao tác chuyển mạch tải.



Hình 3.4: Hình ảnh thực tế module ESP32 (ở Node)



Hình 3.5: Hình ảnh thực tế vi điều khiển STM32F103C8T6 Bluepill

Thông số ESP32:

- Kiến trúc: Lõi kép Xtensa LX6 32-bit, xung nhịp tối đa 240 MHz, hỗ trợ Deep Sleep.
- Bộ nhớ: Flash 4 MB, SRAM 520 KB.
- Ngoại vi: 3 UART, 2 SPI, 2 I2C, 16 kênh PWM, RTC.
- Kết nối không dây tích hợp: WiFi 802.11 b/g/n, Bluetooth 4.2 + BLE.

- I/O: Tối đa 34 chân GPIO đa chức năng.
- ADC**:** 2 ADC 12-bit, 18 kênh analog.
- Nguồn: 3.3V, logic I/O 3.3V.
- Nhiệt độ hoạt động: -40°C đến +85°C.

Thông số STM32F103C8T6 [8]:

- Kiến trúc: Lõi ARM Cortex-M3 32-bit, xung nhịp tối đa 72 MHz, hỗ trợ Sleep/Stop/Standby.
- Bộ nhớ: Flash 64/128 KB, SRAM 20 KB.
- Ngoại vi: 3 bộ TIM 16-bit, 1 TIM nâng cao (PWM), 2 SPI, 2 USART, 1 I2C, 1 USB Device, JTAG/SWD.
- I/O: Tối đa 37 chân GPIO đa chức năng.
- ADC: 2 ADC 12-bit, 10 kênh analog.
- Nguồn: 2.0–3.6V, logic I/O 3.3V.
- Nhiệt độ hoạt động: -40°C đến +85°C.

D. Khối nguồn – Power:

Khối nguồn đóng vai trò cung cấp năng lượng ổn định cho toàn bộ hệ thống Node, đảm bảo hoạt động liên tục và tin cậy trong các điều kiện môi trường khác nhau. Trong dự án này, do hệ thống Node có thể sử dụng nhiều loại vi điều khiển và cảm biến khác nhau với mức điện áp hoạt động không đồng nhất (thường là 5V hoặc 3.3V), việc thiết kế khối nguồn phải đảm bảo khả năng linh hoạt trong cung cấp điện áp phù hợp cho từng thành phần.

Để đáp ứng yêu cầu này, nhóm lựa chọn sử dụng một adapter 12V làm nguồn cấp đầu vào chính cho hệ thống. Điện áp 12V này sau đó sẽ được hạ xuống thành 5V và 3.3V thông qua các mạch ổn áp (voltage regulator) như LM2596 để cung cấp cho vi điều khiển và cảm biến tương ứng. Việc lựa chọn nguồn đầu vào là 12V giúp hệ thống dễ dàng tích hợp thêm các thành phần mở rộng như relay, mô-đun truyền thông hoặc động cơ nhỏ mà không cần nguồn cấp riêng biệt.

E. Khối truyền thông LoRa:

Khối truyền thông LoRa đóng vai trò cầu nối giữa các Node và Gateway trong việc truyền dữ liệu cảm biến và các tín hiệu điều khiển thiết bị. Trong dự án này, nhóm quyết định sử dụng giao thức truyền thông không dây LoRa để đảm nhiệm vai trò này.

Việc lựa chọn LoRa làm phương thức truyền thông cục bộ được đưa ra dựa trên các phân tích lý thuyết đã trình bày trước đó. So với các phương án truyền thông khác như WiFi, Zigbee hoặc Bluetooth, LoRa có ưu điểm vượt trội về khoảng cách truyền (lên đến vài km trong điều kiện lý tưởng), mức tiêu thụ năng lượng thấp và khả năng xuyên vật cản tốt. Đây là các yếu tố đặc biệt quan trọng đối với các hệ thống IoT triển khai ở môi trường rộng lớn như nông nghiệp, nhà xưởng hoặc các khu vực không có sẵn hạ tầng mạng WiFi.

Ngoài ra, giao tiếp LoRa còn hỗ trợ kiến trúc mạng nhiều Node (multi-node topology), rất phù hợp với các hệ thống cần mở rộng quy mô trong tương lai mà không đòi hỏi phải thay đổi phần cứng cốt lõi. Trong hệ thống hiện tại, module E32-433T30D (Hình 3.6) được sử dụng để hiện thực hóa giao tiếp LoRa, dựa trên chip Semtech SX1278 – một chip truyền thông LoRa hoạt động ở băng tần 433 MHz, hỗ trợ tốc độ dữ liệu từ 0.018 kbps đến 37.5 kbps, với khoảng cách truyền lên tới vài kilomet trong điều kiện lý tưởng.



Hình 3.6: Hình ảnh thực tế module LoRa E32 433T30D

Với những lợi thế trên, khối truyền thông sử dụng LoRa không chỉ giúp hệ thống hoạt động hiệu quả trong điều kiện không có kết nối Internet tại các Node, mà còn tạo

nền tảng để mở rộng số lượng Node trong tương lai mà không ảnh hưởng đến hiệu suất tổng thể.

Một số thông số quan trọng có thể cấu hình bao gồm:

- **Địa chỉ thiết bị (ADDH, ADDL):** Cho phép định danh từng node trong mạng, từ đó hỗ trợ hai phương thức truyền:
  - Fixed Transmission: Gửi dữ liệu đích danh đến một thiết bị cụ thể.
  - Broadcast: Gửi đến tất cả các thiết bị trong phạm vi, không cần địa chỉ đích.
- **Kênh truyền (CHAN):** Mỗi giá trị kênh tương ứng với một tần số cụ thể trong dải 433 MHz. Việc thay đổi kênh giúp tránh nhiễu và cho phép nhiều cặp thiết bị hoạt động song song mà không ảnh hưởng lẫn nhau.
- **Tốc độ dữ liệu không khí (Air Data Rate):** Đây là tốc độ truyền tín hiệu qua sóng RF. Tốc độ thấp cho phép truyền xa hơn nhưng dữ liệu ít hơn, trong khi tốc độ cao phù hợp cho môi trường gần, tốc độ nhanh.
- **Tốc độ UART (UART Baud Rate):** Đây là tốc độ giao tiếp giữa mô-đun và vi điều khiển (MCU). Việc cấu hình phù hợp đảm bảo tín hiệu UART được truyền và nhận chính xác, không bị sai lệch dữ liệu.
- **Chế độ truyền dữ liệu:** LoRa có khả năng linh hoạt để cấu hình với hai kiểu truyền khác nhau như:
  - Transparent Transmission (Trong suốt): Mọi thiết bị cùng kênh và địa chỉ đều nhận được dữ liệu. Đơn giản, dễ triển khai, nhưng không kiểm soát được đối tượng nhận cụ thể.
  - Fixed Transmission (Cố định): Dữ liệu mang theo thông tin địa chỉ đích, chỉ thiết bị phù hợp mới xử lý, từ đó tăng tính chọn lọc và bảo mật ở mức cơ bản.
- **Sửa lỗi FEC (Forward Error Correction):** Tính năng này thêm thông tin dư thừa vào gói tin, cho phép phát hiện và tự sửa các lỗi nhỏ xảy ra trong quá trình truyền — một điểm cộng lớn khi hoạt động trong môi trường nhiễu hoặc khoảng cách xa.

- **Thời gian đánh thức (Wireless Wake-up Time):** Đây là khoảng thời gian mô-đun "thức dậy" để kiểm tra dữ liệu đến. Cấu hình phù hợp giúp cân bằng giữa *tiết kiệm năng lượng* và *tốc độ phản hồi.*

Khi hai mô-đun LoRa truyền dữ liệu với nhau, quá trình truyền nhận trải qua nhiều bước quan trọng nhằm đảm bảo dữ liệu được gửi chính xác, đúng địa chỉ và với độ tin cậy cao. Dưới đây là trình tự các bước cơ bản trong quá trình truyền thông giữa hai mô-đun:

+ Khởi tạo và cấu hình:

- Cả hai mô-đun được cấu hình trước khi truyền, bao gồm các thông số: địa chỉ (ADDH, ADDL), kênh tần số (CHAN), tốc độ UART, tốc độ không khí, chế độ truyền (trong suốt hoặc cố định), FEC, công suất phát, thời gian đánh thức, v.v.
- Vi điều khiển giao tiếp với mô-đun qua UART để gửi các lệnh cấu hình hoặc dữ liệu.

+ Chuyển sang chế độ hoạt động:

- Sau khi cấu hình, mô-đun được đưa về chế độ MODE 0 (Normal) để bắt đầu hoạt động truyền và nhận dữ liệu.

+ Tạo và gửi dữ liệu:

- Dữ liệu cần truyền (ví dụ: chuỗi ký tự, cảm biến, thông báo) được tạo ở phía gửi.
- Dữ liệu có thể được mã hóa để đảm bảo an toàn khi truyền các ký tự đặc biệt hoặc nhị phân. Tuy nhiên, tại phần cứng chưa tích hợp chức năng bảo mật dữ liệu mà yêu cầu phải sử dụng phần mềm.
- Vi điều khiển gửi dữ liệu qua UART đến mô-đun LoRa. Mô-đun sẽ chuyển dữ liệu thành sóng vô tuyến và phát đi qua ăng-ten.

+ Nhận dữ liệu ở phía đích:

- Mô-đun phía nhận luôn ở trạng thái lắng nghe (nếu đã thức), sẵn sàng tiếp nhận gói tin phù hợp với kênh và địa chỉ đã cấu hình.

- Nếu chế độ Fixed Transmission được sử dụng, mô-đun sẽ lọc theo địa chỉ đích để tránh nhận nhầm.

+ Xử lý dữ liệu nhận:

- Sau khi nhận dữ liệu, mô-đun gửi dữ liệu về vi điều khiển qua UART.
- Nếu dữ liệu đã được mã hóa trước đó, vi điều khiển sẽ giải mã để lấy lại nội dung gốc.
- • Dữ liệu sau đó được xử lý (lưu trữ, điều khiển thiết bị, v.v.) tùy theo yêu cầu của hệ thống.

+ Kiểm tra lỗi và phản hồi (nếu có):

- Với chế độ có FEC, một số lỗi nhẹ trong quá trình truyền có thể được tự động sửa.
- Hệ thống có thể lập trình để gửi lại gói tin nếu không nhận được phản hồi hoặc nhận lỗi.

Tuy nhiên trong truyền thông LoRa, các mô-đun không sử dụng giao thức xác nhận như TCP/IP. Do đó, việc đảm bảo độ tin cậy (như gửi lại gói tin, xác nhận đã nhận thành công) cần được xử lý ở tầng ứng dụng do người lập trình xây dựng.

F. Khối vi điều khiển tại Gateway – MCU:

Tại Gateway, vi điều khiển đóng vai trò trung tâm xử lý, chịu trách nhiệm tiếp nhận dữ liệu từ các Node (thông qua mô-đun LoRa), xử lý và truyền lên Server, đồng thời nhận lệnh từ Server và gửi lại cho các Node để điều khiển thiết bị.

Trong dự án này, nhóm lựa chọn sử dụng vi điều khiển ESP32 (Hình 3.7) – một dòng chip tích hợp WiFi, được thiết kế tối ưu cho các ứng dụng IoT. Quyết định này được đưa ra dựa trên các yếu tố sau:

- Tích hợp WiFi mạnh mẽ: ESP32 có khả năng kết nối mạng trực tiếp mà không cần module bổ sung, giúp đơn giản hóa phần cứng và tiết kiệm chi phí. Dữ liệu từ Node sau khi được xử lý có thể gửi trực tiếp lên Server qua HTTP hoặc MQTT thông qua mạng WiFi.

- Hiệu năng xử lý cao: ESP32 sử dụng lõi kép Xtensa 32-bit, xung nhịp lên đến 240 MHz, đủ mạnh để thực hiện các tác vụ xử lý, đóng gói dữ liệu và giao tiếp mạng đồng thời.
- Hỗ trợ nhiều giao thức: Ngoài UART để giao tiếp với mô-đun LoRa, ESP32 còn hỗ trợ các giao thức như SPI, I2C, PWM,... tạo điều kiện cho việc mở rộng hệ thống hoặc tích hợp thêm chức năng.
- Chi phí hợp lý: Mặc dù tích hợp nhiều chức năng, ESP32 vẫn có giá thành phải chăng, phù hợp với các dự án IoT quy mô vừa và lớn.



Hình 3.7: Hình ảnh thực tế module ESP32 (Ở Gateway)

Với ESP32, Gateway không chỉ làm nhiệm vụ trung gian truyền dữ liệu mà còn có thể thực hiện xử lý sơ bộ, lưu tạm dữ liệu khi mất kết nối, giám sát trạng thái các Node, đồng thời đảm bảo giao tiếp thời gian thực với hệ thống điều khiển trung tâm (Server). Đây là thành phần then chốt đảm bảo tính linh hoạt, ổn định và mở rộng cho toàn bộ hệ thống.

Khác với việc truyền văn bản thuần, dữ liệu trao đổi giữa Node và Gateway qua LoRa cần được tuần tự hóa (serialize) thành chuỗi byte trước khi phát đi và được giải tuần tự (deserialize) ở phía nhận. Do LoRa có tốc độ thấp và hoạt động bán song công với kích thước gói tin hạn chế, việc lựa chọn định dạng biểu diễn dữ liệu sao cho nhỏ gọn, ít tốn thời gian phát (airtime) và nhẹ về tài nguyên là yếu tố quyết định hiệu năng của hệ thống. Một số phương án phổ biến để biểu diễn dữ liệu trên đường truyền gồm:

chuỗi văn bản JSON, mã hóa biểu diễn Base64, và đóng gói nhị phân theo kiểu TLV (Type–Length–Value) của Protobuf. Bảng 3.2 trình bày sự so sánh giữa các phương án này.

Bảng 3.2: So sánh các phương pháp biểu diễn dữ liệu cho truyền thông LoRa

Tiêu chí	JSON (văn bản)	Base64	Nhị phân TLV (kiểu Protobuf)
Kích thước gói tin	Lớn, kèm theo tên trường, dấu ngoặc và dấu phân cách	Phình thêm khoảng 33% so với dữ liệu nhị phân gốc	Nhỏ gọn nhất, mỗi trường chỉ gồm mã trường, kiểu và giá trị
Chi phí tính toán, RAM/Flash	Cao, cần bộ phân tích chuỗi (parser)	Thấp nhưng chỉ là tầng biểu diễn, không giảm dữ liệu	Rất thấp, chỉ sao chép theo offset đã định nghĩa sẵn
Khả năng mở rộng trường	Cao	Không liên quan (chỉ bao bọc dữ liệu)	Cao, trường tùy chọn, phía nhận tự bỏ qua trường không nhận diện
Mức phù hợp với LoRa	Kém, tốn airtime và dễ vượt giới hạn gói	Kém, làm dữ liệu phình to, ngược với mục tiêu	Tốt, tối ưu airtime, phù hợp với gói tin chỉ vài chục byte

Do dữ liệu của hệ thống chủ yếu là số (giá trị cảm biến, mã thiết bị, trạng thái) và yêu cầu cốt lõi là tiết kiệm băng thông LoRa, nhóm quyết định sử dụng phương pháp đóng gói nhị phân TLV theo kiểu Protobuf thay vì JSON hay Base64. Cần lưu ý rằng Base64 thực chất là kỹ thuật mã hóa biểu diễn làm tăng kích thước dữ liệu khoảng 33%, nên không phù hợp cho đường truyền tốc độ thấp; vì vậy phương pháp này không được sử dụng trên chặng LoRa.

Cơ chế mã hóa TLV được hiện thực trong lớp ProtoCodec cùng các định nghĩa bản tin đi kèm. Mỗi bản tin là một cấu trúc gồm các cặp cờ has_field và giá trị tương ứng; khi mã hóa, chỉ những trường có mặt mới được ghi vào bộ đệm theo định dạng: mã trường (field_id, 1 byte) – kiểu dữ liệu (type, 1 byte) – giá trị (value, kích thước theo

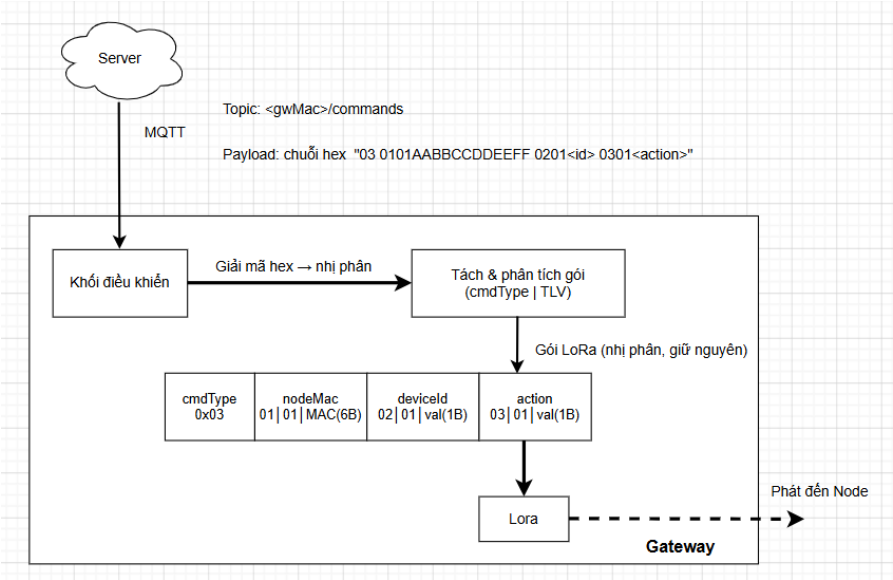
lược đồ). Trong đó, kiểu dữ liệu nhận các giá trị: 0 là BOOL, 1 là UINT8, 2 là UINT16, 3 là UINT32 và 4 là FLOAT. Một số quy ước đặc biệt: địa chỉ MAC được biểu diễn bằng kiểu UINT8 nhưng có độ dài 6 byte; số thực FLOAT chiếm 4 byte theo thứ tự little-endian của vi điều khiển. Nhờ cơ chế chỉ ghi những trường có mặt, cùng một định nghĩa bản tin vẫn linh hoạt cho nhiều ngữ cảnh khác nhau trong khi gói tin luôn được giữ ở mức tối thiểu.

Mỗi gói tin LoRa được bọc thêm một byte mã lệnh (CommandType) ở đầu để phía nhận biết cách giải mã phần thân, có dạng tổng quát: mã lệnh (cmdType, 1 byte) – phần thân TLV của bản tin tương ứng. Các mã lệnh chính của hệ thống gồm: gửi dữ liệu cảm biến định kỳ, điều khiển thiết bị, truy vấn trạng thái thiết bị, quét Node, đổi Gateway chủ, đồng bộ thời gian, xóa Gateway, cùng các mã phản hồi tương ứng cho từng lệnh.

Hệ thống sử dụng hai dạng biểu diễn ở hai chặng truyền khác nhau, mỗi dạng phù hợp với đặc thù của chặng đó. Trên chặng LoRa giữa Node và Gateway, dữ liệu được truyền dưới dạng nhị phân thuần gồm mã lệnh và phần thân TLV nhằm tối ưu airtime; module LoRa E32 truyền và nhận theo khung có độ dài nên xử lý tốt dữ liệu nhị phân mà không cần bọc văn bản. Trên chặng MQTT giữa Gateway và Server, ở chiều lên, Gateway giải mã gói LoRa rồi chuyển sang định dạng JSON trước khi gửi lên broker, giúp Server xử lý đơn giản; ở chiều xuống, gói lệnh nhị phân được biểu diễn dưới dạng chuỗi hex để truyền an toàn qua MQTT, sau đó Gateway tự động nhận diện chuỗi hex và giải mã ngược về nhị phân trước khi chuyển tiếp qua LoRa.

Dưới đây là luồng dữ liệu thực tế được thiết kế cho việc giao tiếp LoRa của hệ thống.

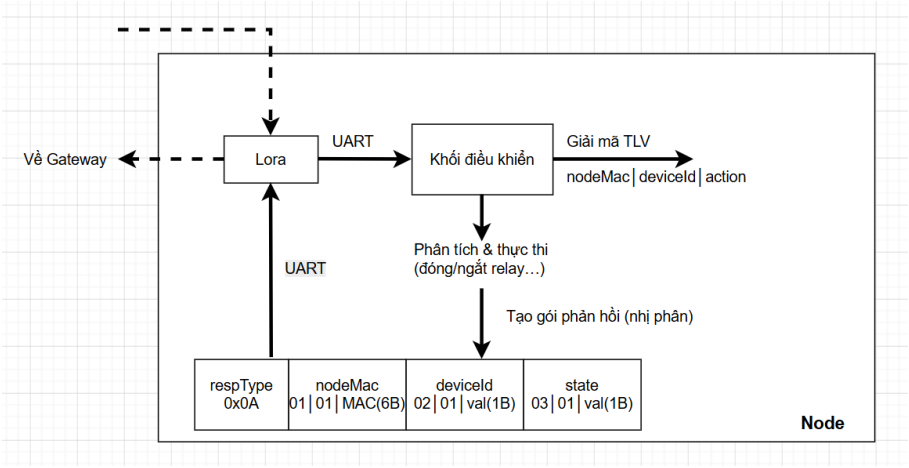
Tại Gateway – Quá trình truyền dữ liệu đến Node (Hình 3.8):



Hình 3.8: Luồng dữ liệu phát tại Gateway

- Nhận lệnh từ Server: Gateway đăng ký topic điều khiển và nhận gói lệnh từ Server dưới dạng chuỗi hex.
- Giải mã và phân tích: Khối điều khiển ESP32 tự nhận diện và giải mã hex về nhị phân, tách byte đầu làm mã lệnh và phần còn lại là bản tin TLV.
- Đóng gói LoRa: Gateway giữ nguyên gói nhị phân gồm mã lệnh và phần thân TLV, không bọc thêm Base64.
- Truyền qua UART đến LoRa: dữ liệu được gửi tới module ESP32 qua giao tiếp UART và phát không dây đến Node đích.
- Đăng ký chờ phản hồi: đồng thời, Gateway lưu lệnh vào hàng chờ để tự động gửi lại nếu không nhận được phản hồi tương ứng trong khoảng thời gian quy định.

Tại Node – Quá trình nhận dữ liệu từ Gateway và tạo phản hồi (Hình 3.9):



Hình 3.9: Luồng dữ liệu thu tại Node

- Nhận dữ liệu qua LoRa: Node nhận gói tin gồm mã lệnh và phần thân TLV từ Gateway thông qua module LoRa.
- Giải mã TLV: khối điều khiển giải mã phần thân theo đúng lược đồ của mã lệnh, khôi phục các trường ban đầu như địa chỉ Node, mã thiết bị và hành động.
- Kiểm tra và thực thi: Node chỉ xử lý khi địa chỉ trong gói tin trùng với danh tính của mình, sau đó thực hiện hành động tương ứng như đóng/ngắt relay.
- Tạo phản hồi: kết quả xử lý được đóng gói thành gói phản hồi gồm mã lệnh phản hồi và phần thân TLV chứa địa chỉ Node, mã thiết bị và trạng thái mới.
- Mã hóa và gửi lại: gói phản hồi được phát ngược về Gateway qua LoRa. Gateway giải mã, chuyển thành JSON và gửi lên Server để cập nhật trạng thái.

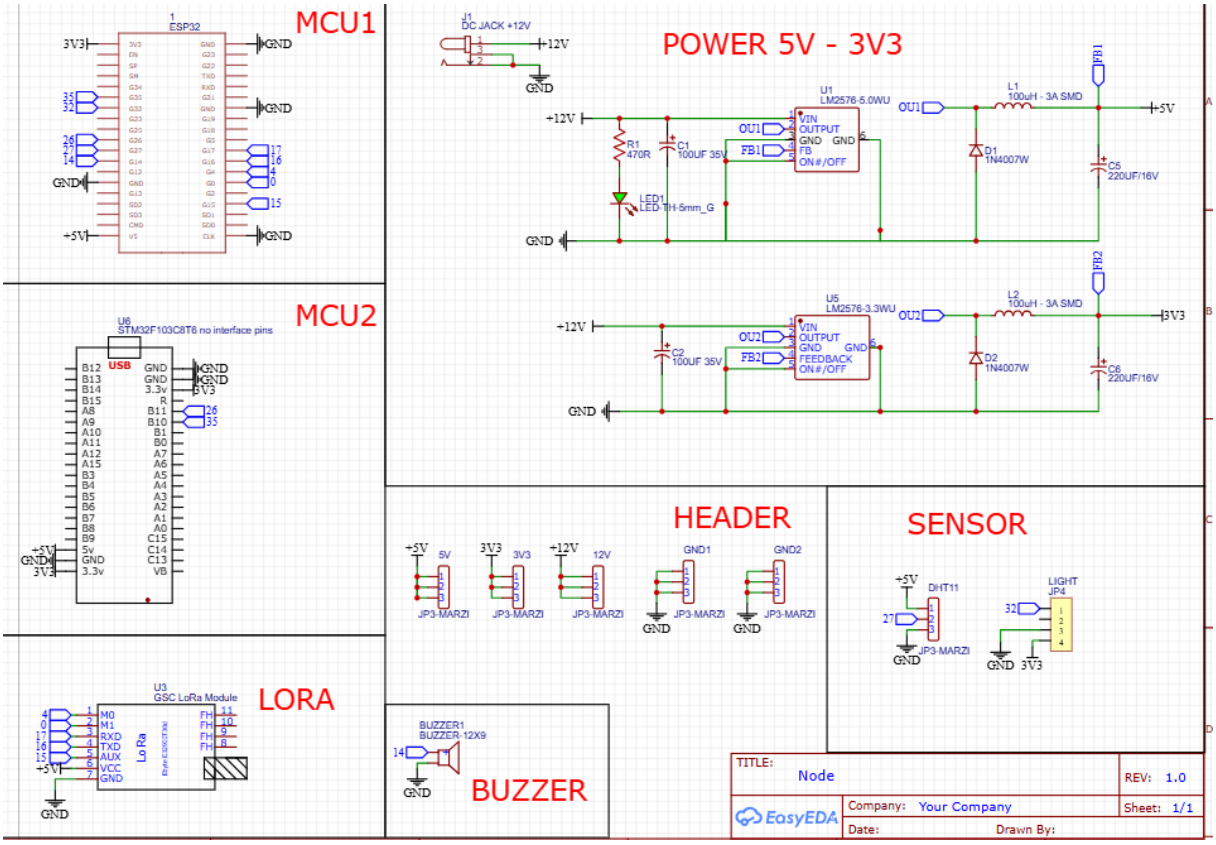
3.3.1.4 Sơ đồ nguyên lý

3.3.1.4.1 Sơ đồ nguyên lý mạch NODE

Sơ đồ nguyên lý được thiết kế trên phần mềm EasyEDA, giúp bố trí mạch một cách trực quan, dễ kiểm tra và thuận tiện cho việc thiết kế PCB về sau. Sơ đồ bao gồm đầy đủ các khối chức năng như: cảm biến, vi điều khiển, thiết bị điều khiển, giao tiếp không dây và nguồn cấp. Việc thiết kế sơ đồ nguyên lý đóng vai trò quan trọng trong việc kiểm tra tính đầy đủ của linh kiện, đảm bảo khả năng tương thích giữa các khối chức năng và

giúp phát hiện sớm các lỗi về kết nối logic trước khi bước vào giai đoạn vẽ mạch in (PCB).

Sau khi hoàn thành bước phân tích hệ thống và xác định chức năng của từng thành phần, nhóm tiến hành thiết kế sơ đồ nguyên lý cho mạch Node (Hình 3.10) – đây là phần tử đầu trong hệ thống, có nhiệm vụ thu thập dữ liệu môi trường và điều khiển thiết bị điện theo yêu cầu từ phía Gateway.



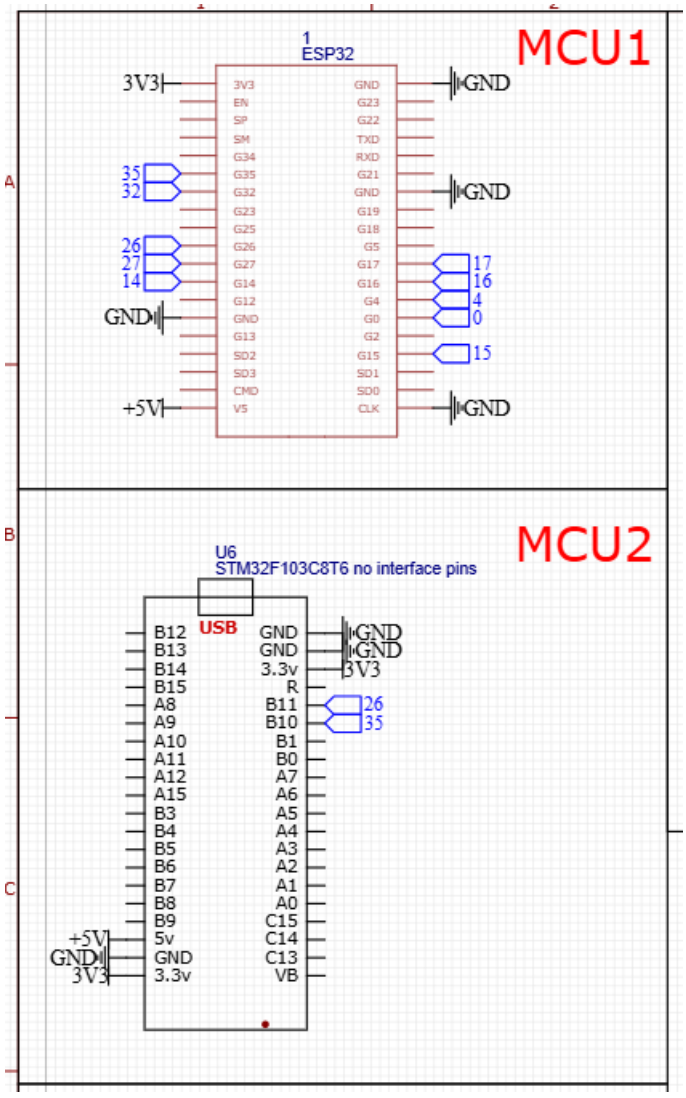
Hình 3.10: Sơ đồ nguyên lý tổng quan của Node

Trong thiết kế hệ thống hiện tại, sơ đồ nguyên lý tổng quan của một Node sử dụng kiến trúc hai vi điều khiển ESP32 (MCU1) và STM32 (MCU2) kết hợp với các thành phần như cảm biến môi trường và module LoRa. Các thành phần này phối hợp chặt chẽ để thu thập dữ liệu, điều khiển thiết bị và truyền thông với Gateway. Hai MCU kết nối với nhau qua UART để chia sẻ dữ liệu và phân chia nhiệm vụ, giúp tối ưu hiệu suất, đảm bảo tính ổn định và khả năng mở rộng hệ thống.

Trong cấu trúc khối điều khiển, vi điều khiển thứ nhất chịu trách nhiệm giao tiếp với module LoRa, thu thập và xử lý dữ liệu từ các cảm biến môi trường như nhiệt độ,

độ ẩm, ánh sáng. Dữ liệu sau khi được xử lý sẽ được đóng gói theo định dạng chuẩn và truyền đến Gateway.

Song song đó, vi điều khiển còn lại tập trung điều khiển đến 16 thiết bị điện độc lập. Hai vi điều khiển trao đổi dữ liệu với nhau qua chuẩn UART, cho phép tổng hợp dữ liệu cảm biến và trạng thái thiết bị thành một gói tin duy nhất trước khi gửi lên Gateway. Sự phân tách nhiệm vụ như vậy không chỉ tối ưu hiệu suất xử lý mà còn nâng cao tính ổn định và khả năng mở rộng của toàn hệ thống.(Hình 3.11).



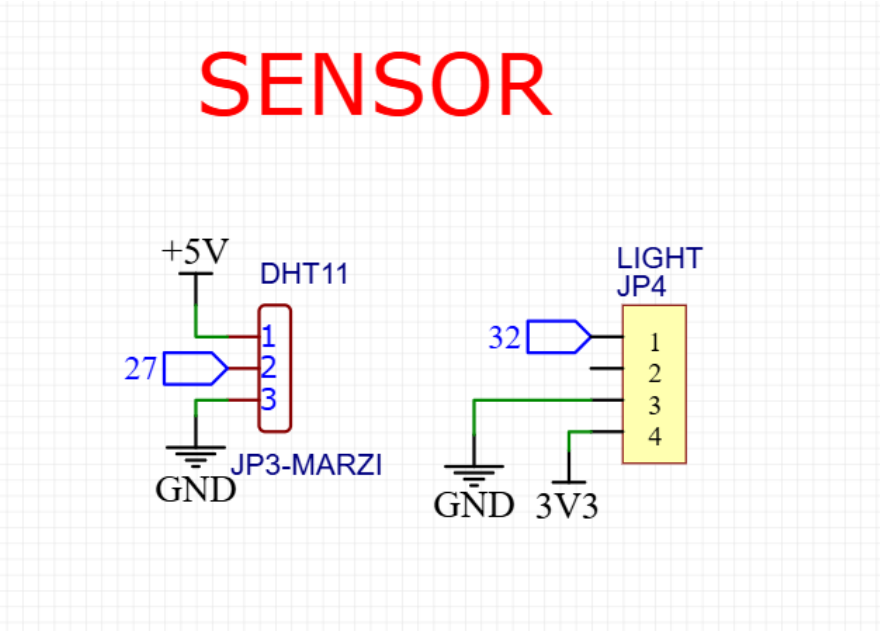
Hình 3.11: Sơ đồ nguyên lý bộ xử lý trung tâm Node

Dưới đây là một số chức năng các chân I/O của hai vi điều khiển:

- MCU1_32: Chân đọc cảm biến ánh sáng.

- MCU1_27: Chân đọc cảm biến DHT11.
- MCU1_17, MCU1_16, MCU1_4, MCU1_0, MCU1_15: Kết nối với module LoRa.
- MCU1_14: Chân điều khiển Buzzer.
- MCU1_26, MCU1_35: Kết nối với 2 chân MCU2_B11 và MCU2_B10 để gửi tín hiệu điều khiển thiết bị.

Tiếp theo là về sơ đồ nguyên lý của các sensors (Hình 3.12) . Các module được sử dụng là DHT11, LDR. Điện áp cần cấp cho các module này là 5V, kết nối chung GND với vi điều khiển và kết nối theo các chuẩn giao tiếp UART, digital và analog tương ứng với các cấu hình của cảm biến.

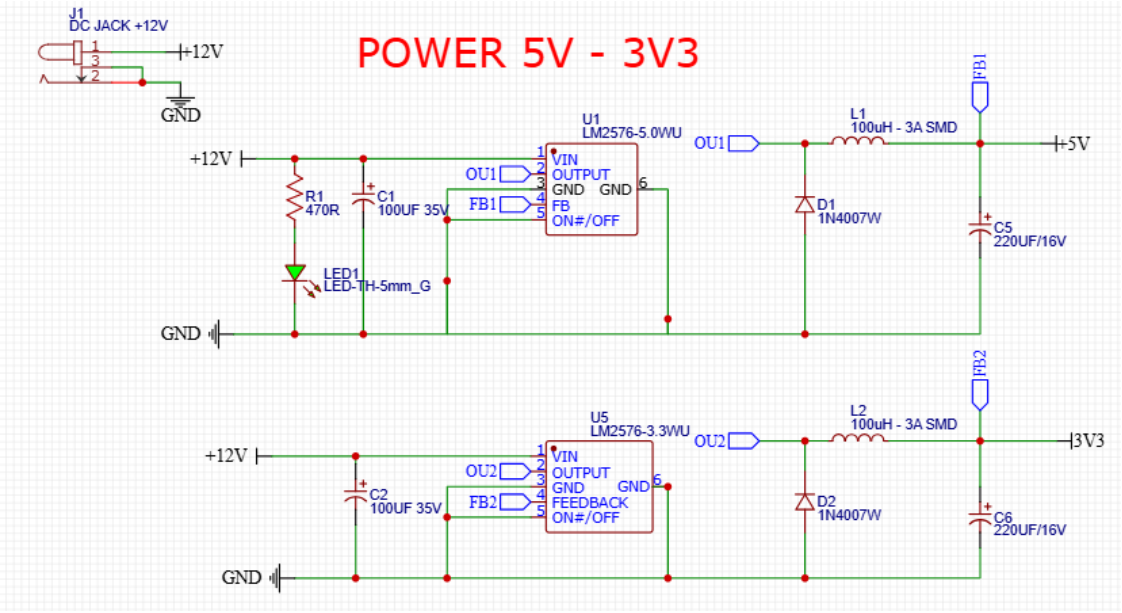


Hình 3.12: Sơ đồ nguyên lý khối cảm biến

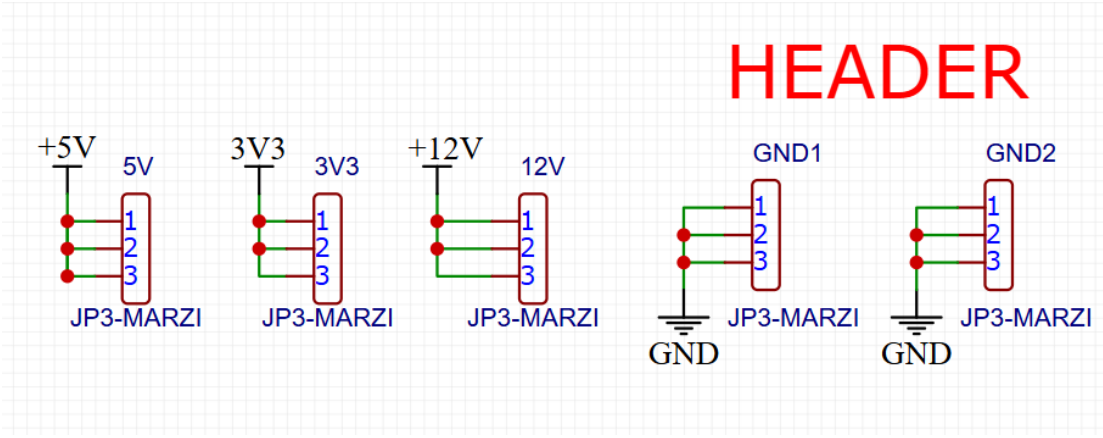
- Cảm biến DHT11 kết nối với chân 27 của MCU1.
- Cảm biến ánh sáng kết nối với chân 32 của MCU1.

Tiếp đến khối nguồn, Node và Gateway dùng nguồn giống nhau nên thiết kế cùng (Hình 3.13). Nguồn nuôi MCU là 5V nên thiết kế mạch hạ áp dùng IC LM2576S-5.0 để hạ áp xuống 5V, cùng lúc đó có thể nuôi các module khác các cảm biến và module LoRa. Ngoài ra còn có các header ra ngoài để sử dụng các ngoài khác như 12V, 5V, 3V3, GND hỗ trợ việc mở rộng hệ thống (Hình 3.14). Có Jack DC để sử dụng adapter 12V và LED

báo nguồn. Khi LM2576 đóng (ON), dòng điện đi qua cuộn cảm (L1, L2) và tích trữ năng lượng ở dạng từ trường. Khi LM2576 ngắt (OFF), cuộn cảm tiếp tục duy trì dòng điện thông qua diode Schottky và tải, nhờ vào năng lượng đã tích trữ. Diode Schottky (D1, D2) có điện áp rơi thấp, đáp ứng nhanh giúp giảm tổn hao và nhiễu điện áp. Cuộn cảm 100uH làm trơn dòng điện đầu ra và duy trì dòng liên tục. Tụ điện trước (C21, C23) và sau (C22, C24) IC giúp ổn định điện áp và giảm dao động khi tải thay đổi.



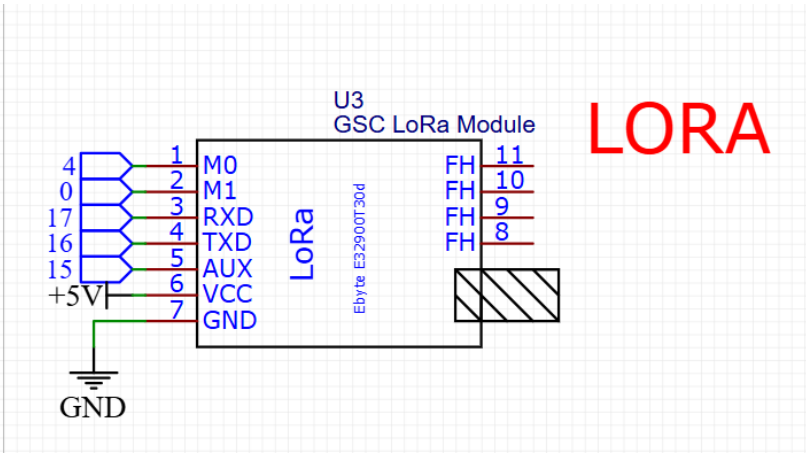
Hình 3.13: Sơ đồ nguyên lý khối nguồn



Hình 3.14: Sơ đồ nguyên lý các header nguồn

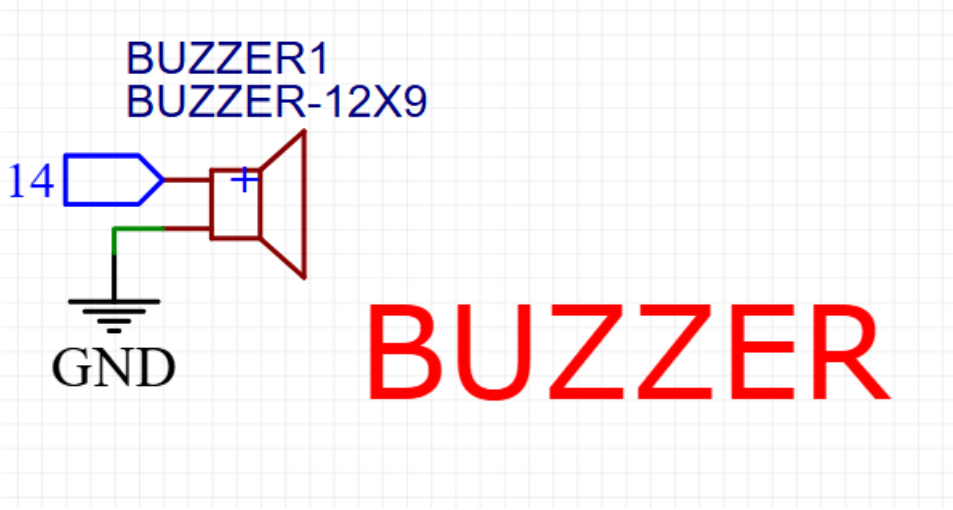
Tiếp theo là sơ đồ nguyên lý kết nối giữa vi điều khiển và module LoRa E32 433T30D (Hình 3.15). Module này có tổng cộng 7 chân kết nối, trong đó hai chân M0

và M1 dùng để cấu hình chế độ hoạt động, cho phép lựa chọn giữa các chế độ như truyền – nhận bình thường, truyền tức thời, hoặc cấu hình thông số. Chân TXD và RXD là cặp giao tiếp UART nối trực tiếp với các chân UART tương ứng của vi điều khiển, dùng để truyền và nhận dữ liệu. Chân AUX đóng vai trò là tín hiệu báo trạng thái hoạt động, thường được dùng để xác định khi nào module đang bận hoặc đã sẵn sàng gửi/nhận dữ liệu, giúp việc đồng bộ dữ liệu giữa MCU và LoRa hiệu quả hơn. Ngoài ra, module sử dụng nguồn cấp 5V và GND phải được nối chung với khối điều khiển.



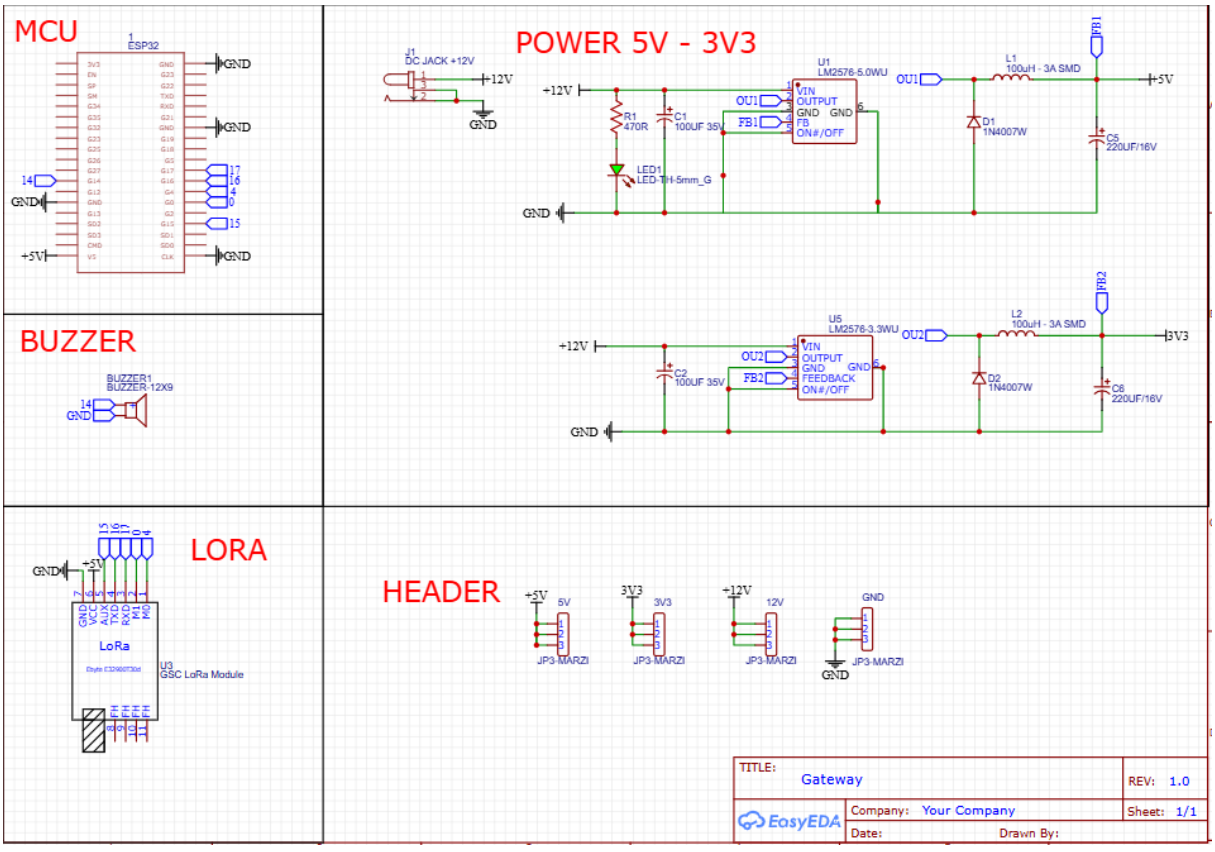
Hình 3.15: Sơ đồ nguyên lý kết nối module LoRa

MCU1 còn được kết nối với một Buzzer qua chân 14, chân còn lại nối trực tiếp xuống GND (Hình 3.16). Thông qua chân này, vi điều khiển có thể phát cảnh báo âm thanh trong các tình huống như vượt ngưỡng cảm biến, mất kết nối với Gateway hoặc thiết bị gặp sự cố. Ngoài ra, buzzer cũng có thể được sử dụng để phản hồi trạng thái hoạt động của Node (như báo hiệu khởi động thành công hoặc truyền dữ liệu hoàn tất), giúp người vận hành dễ dàng theo dõi và kiểm tra hệ thống tại hiện trường.



Hình 3.16: Sơ đồ nguyên lý kết nối của Buzzer

3.3.1.4.2 Sơ đồ nguyên lý mạch GATEWAY

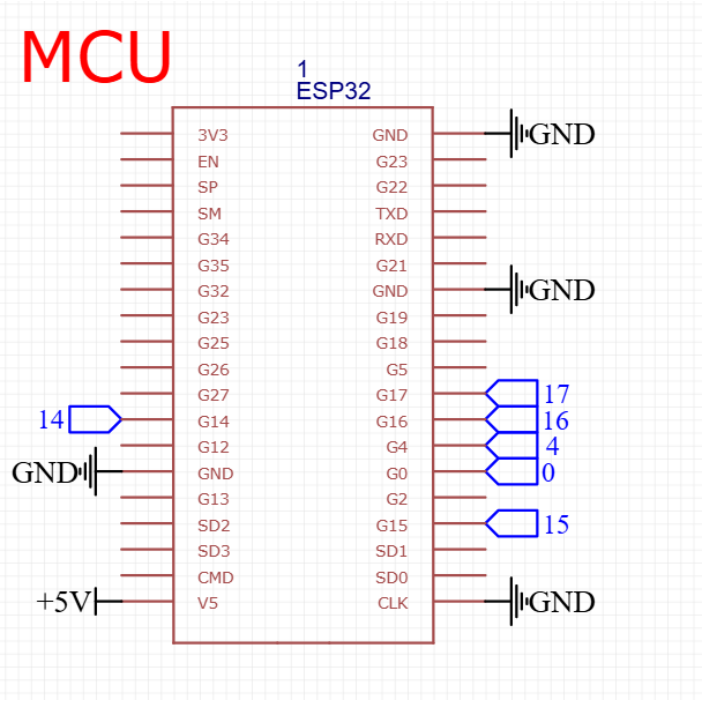


Hình 3.17: Sơ đồ nguyên lý tổng quan của Gateway

Gateway trong hệ thống được thiết kế với cấu trúc đơn giản, chỉ bao gồm ba thành phần chính: vi điều khiển trung tâm ESP32, module LoRa để giao tiếp với các Node,

khối nguồn cung cấp điện ổn định và Buzzer để thực hiện chức năng cảnh báo giống với Node.

Như đã đề cập ở trên, vi điều khiển ESP32 sẽ chịu trách nhiệm kết nối và điều phối hoạt động của toàn bộ mạch, bao gồm kết nối trực tiếp với module LoRa qua UART để nhận dữ liệu từ các Node, kết nối với nguồn nuôi 5V và GND để đảm bảo hoạt động ổn định, đồng thời các chân giao tiếp dự phòng cũng được đưa ra đầu cắm để hỗ trợ kiểm tra hoặc mở rộng chức năng trong quá trình triển khai hệ thống.



Hình 3.18: Sơ đồ nguyên lý của ESP32

- Buzzer: Nối với chân 14 của ESP32.
- Chân 17,16,4,0,15: Dùng để kết nối với module LoRa.

Bên cạnh đó, các khối và linh kiện như module LoRa, buzzer cảnh báo và mạch nguồn cũng được thiết kế đồng bộ với cấu trúc phần cứng của Node, đảm bảo tính nhất quán trong hệ thống và thuận tiện cho việc bảo trì, thay thế hoặc mở rộng khi cần. Các thành phần này đã được trình bày chi tiết trong phần mô tả Node ở trên.

3.3.2 Cơ sở dữ liệu – Server Backend

3.3.2.1 Yêu cầu chức năng của cơ sở dữ liệu

Với Server - Backend, hệ thống yêu cầu các chức năng sau:

- Nhận dữ liệu và yêu cầu từ hệ thống phần cứng, tiến hành thực hiện một số chức năng:
  - Lưu thông tin các dữ liệu cảm biến và thiết bị điều khiển vào cơ sở dữ liệu theo thời gian thực.
  - Tính toán dữ liệu cảm biến theo giờ, phút, giây.
  - Bật/ tắt thiết bị điều khiển theo lịch đặt trước.
  - Lưu tất cả các gateway chưa được cấu hình vào cơ sở dữ liệu.
- Nhận dữ liệu và yêu cầu từ phía người dùng thông qua Website, tiến hành thực hiện một số chức năng:
  - Lưu thông tin người dùng vào cơ sở dữ liệu.
  - Lưu thông tin Gateway do người dùng đó quản lý vào cơ sở dữ liệu.
  - Lưu thông tin nhà kính được tạo từ Gateway vào cơ sở dữ liệu.
  - Nhận yêu cầu đặt lịch bật/ tắt thiết bị điều khiển.
  - Nhận yêu cầu đặt điều kiện bật/tắt thiết bị điều khiển theo giá trị của cảm biến.

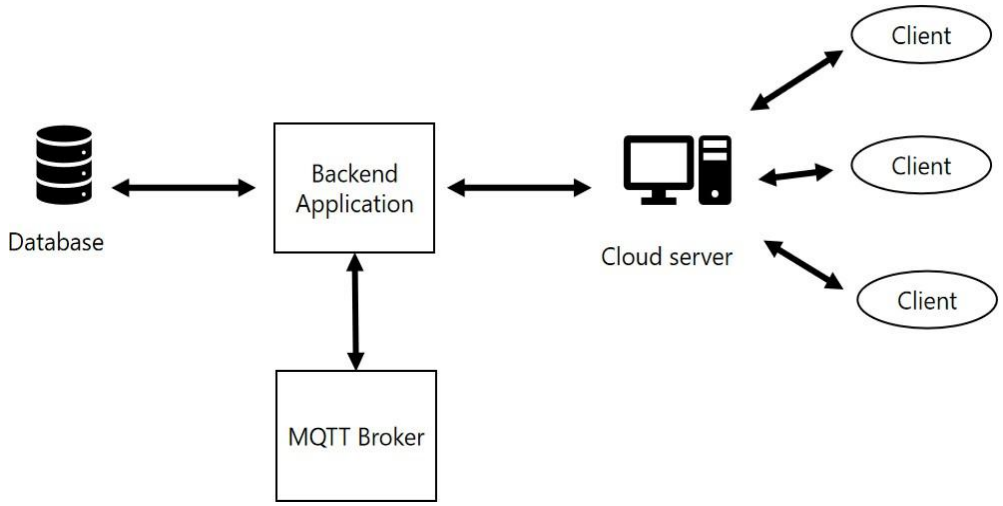
3.3.2.2 Sơ đồ tổng quan của Cơ sở dữ liệu - Server Backend

Dựa trên các yêu cầu chức năng đối với Cơ sở dữ liệu - Server Backend, nhóm nghiên cứu đã tiến hành phân tích và chia hệ thống backend thành bốn khối chức năng riêng biệt (Hình 3.19). Mỗi khối đảm nhận một vai trò cụ thể, khi kết hợp lại sẽ tạo thành một hệ thống backend hoàn chỉnh, đáp ứng đầy đủ các yêu cầu:

- Database**:** Nơi lưu trữ dữ liệu người dùng, giá trị cảm biến, trạng thái thiết bị điều khiển.
- MQTT Broker: Phân luồng dữ liệu từ phần cứng đến Backend Application.
- Backend Application: là thành phần đóng vai trò quan trọng nhất trong hệ thống chịu trách nhiệm xử lý hầu hết tác vụ:

- Giao tiếp với MQTT Broker để nhận dữ liệu cảm biến.
- Phân tích, xử lý, lưu trữ dữ liệu vào database.
- Xử lý các điều kiện bật/ tắt thiết bị.
- Theo dõi dữ liệu cảm biến theo thời gian thực, gửi lệnh điều khiển thiết bị.

Cloud Server: Backend Application được triển khai trên môi trường phát triển và hoạt động trên Cloud (Render).



Hình 3.19: Sơ đồ tổng quan của Server Backend - Cơ sở dữ liệu.

Xuất phát từ yêu cầu thực tế của hệ thống là cần lưu trữ và xử lý khối lượng lớn dữ liệu cảm biến có cấu trúc đa dạng, nhóm đã nhận định rằng mô hình cơ sở dữ liệu phi quan hệ (NoSQL) là giải pháp tối ưu nhất. Do đó, nhóm quyết định lựa chọn MongoDB làm hệ quản trị cơ sở dữ liệu chính cho dự án. MongoDB hỗ trợ lưu trữ dữ liệu dưới dạng tài liệu (Document) có cấu trúc tương tự JSON một cách linh hoạt, rất thích hợp cho các gói tin cấu trúc thay đổi tùy theo từng loại thiết bị IoT. Bên cạnh đó, việc MongoDB dễ dàng tích hợp với backend Node.js và sở hữu khả năng mở rộng (scalability) cao sẽ đảm bảo cho hệ thống hoạt động ổn định trong tương lai. Dưới đây là các thông số giới hạn của phiên bản Cloud MongoDB Atlas (gói miễn phí) được áp dụng trong hệ thống [9]:

- Dung lượng tối đa là 512MB.
- Có thể kết nối 500 kết nối đồng thời.

- Tối đa 100 database, 500 collection trên toàn bộ cluster (ví dụ có 10 database, mỗi database trung bình chỉ có 50 collection).
- Kích thước tối đa của một Document là 16MB.

Về khối MQTT Broker, nhóm nghiên cứu sử dụng dịch vụ cung cấp MQTT Broker của HiveMQ – đây là nền tảng cung cấp MQTT Broker nổi tiếng và miễn phí. Tuy nhiên, dùng các MQTT Broker bên thứ ba cũng có các ưu và nhược điểm như:

- Ưu điểm: Sử dụng miễn phí (phù hợp với giai đoạn đầu của hệ thống), tốc độ khá ổn, hỗ trợ các giao thức cần thiết với các bảo mật như cần phải có "Host", "Username" và "Password".
- Nhược điểm: Đây là một Broker công khai nên dễ dàng lộ thông tin, hay bị mất kết nối với Broker (tuy nhiên không đáng kể).

Về khối Backend Application, đây là khối chứa các ứng dụng chạy phía Server-Backend. Trong giai đoạn phát triển và thử nghiệm, khối này được triển khai trên môi trường Web App (Render), vận hành trên nền tảng Node.js (một môi trường thực thi JavaScript ngoài trình duyệt). Backend Application đảm nhận các chức năng cốt lõi của hệ thống bao gồm: cung cấp các API RESTful để trao đổi dữ liệu nội bộ, kết nối với cơ sở dữ liệu và tiếp nhận/xử lý các gói tin từ giao thức MQTT. Dưới đây là phương thức kết nối của Backend với các khối khác trong môi trường thử nghiệm:

- Kết nối với Database: Sử dụng thư viện "*mongoose*" để kết nối với MongoDB.
- Kết nối với Cloud Server: Sử dụng Render để chạy dự án NodeJS sau mỗi lần cập nhật mã nguồn mới lên Github, sau khi chạy dự án xong thì phía Client có thể dùng đường dẫn để thực hiện các API RESTful từ phía Backend.
- Kết nối với Môi trường Phát triển: Sử dụng máy chủ trên Render để khởi chạy và vận hành trực tiếp dự án Node.js sau mỗi lần cập nhật mã nguồn mới. Sau khi dự án được kích hoạt thành công, phía Client (ứng dụng Web hoặc Mobile) có thể sử dụng đường dẫn cục bộ hoặc địa chỉ IP mạng nội bộ (ví dụ: https://greenhouse-datn.onrender.com/pages/index.html) để thực hiện và gọi các API RESTful từ phía Backend.

- Kết nối với MQTT Broker: Sử dụng thư viện "*mqtt*" cung cấp cho dự án NodeJS.

3.3.2.3 Thiết kế Cơ sở dữ liệu - Server Backend

3.3.2.3.1 Xây dựng kiến trúc cho Backend

Để có một hệ thống dự án NodeJS cho Backend hoàn chỉnh cần phải có kiến trúc thư mục rõ ràng, dễ quản lý giúp tiết kiệm thời gian và dễ dàng chỉnh sửa mã nguồn, chia các chức năng vào từng một folder riêng (Hình 3.20).

```
controllers
├ conditionsController.js
├ controlController.js
├ crudFactory.js
├ devicesController.js
├ gatewaysController.js
├ newGatewayController.js
├ nodeController.js
├ notifiesController.js
├ schedulesController.js
├ sensorsController.js
└ usersController.js
models
├ baseModel.js
├ Condition.js
├ Device.js
├ Gateway.js
├ NewGateway.js
├ Node.js
├ Notify.js
├ Schedule.js
├ Sensor.js
└ User.js
routes
├ conditions.js
├ devices.js
├ gateways.js
├ index.js
├ newGateways.js
├ nodes.js
├ notifies.js
├ schedules.js
├ sensors.js
└ users.js
lib
├ encodeConfig.js
└ encodeControl.js
scripts
├ seed.js
└ seedMongo.js
server.js
mqttHandler.js
package.json
db.json
.env
.gitignore
```

Hình 3.20: Kiến trúc cây thư mục Backend.

Kiến trúc của các tập tin trong Backend được tổ chức theo mô hình MVC (Model – View – Controller) [10]. Trong đó, Model đảm nhận phần dữ liệu — nơi định nghĩa các schema và thực hiện mọi tương tác với cơ sở dữ liệu MongoDB; Controller chịu trách

nhiệm xử lý logic nghiệp vụ và điều phối luồng dữ liệu giữa Model với phía client; còn Routes đóng vai trò tiếp nhận và chuyển tiếp các yêu cầu từ client đến controller tương ứng. Cách phân tách này tạo ra sự rạch ròi về trách nhiệm giữa các thành phần, nhờ đó hệ thống dễ bảo trì, dễ mở rộng và thuận lợi cho việc kiểm thử.

Về thư mục controllers, đây là nơi điều hành các logic quan trọng của ứng dụng; bên trong chứa nhiều tập tin chức năng, mỗi tập tin phụ trách một nhóm nghiệp vụ riêng. Ngoài ra, các controller này dùng chung một tập tin crudFactory.js để sinh tự động các thao tác thêm/đọc/sửa/xóa (CRUD) cơ bản, giúp giảm trùng lặp mã nguồn giữa các tài nguyên.

Về thư mục models, đây là khu vực định nghĩa schema (cấu trúc dữ liệu) cho cơ sở dữ liệu, chẳng hạn như các trường nameUser, birthday, phone, email,… Mỗi schema đóng vai trò là cấu trúc đại diện cho một bảng (collection) trong MongoDB, được dùng để xác định dữ liệu đầu vào hợp lệ, kiểu dữ liệu và các ràng buộc, đồng thời hỗ trợ việc truy xuất, cập nhật hay xóa dữ liệu một cách nhất quán. Các schema cùng kế thừa cấu hình chung từ baseModel.js để thống nhất quy ước định danh giữa các collection.

Về thư mục routes, nơi khai báo các API như POST, GET, PATCH, DELETE để phía client sử dụng [11], ví dụ:

- routes/sensors.js: định nghĩa các đường dẫn như GET /sensors (lấy dữ liệu cảm biến), POST /sensors (thêm cảm biến mới).
- routes/devices.js: định nghĩa DELETE /devices/:id (xóa thông tin một thiết bị), PATCH /devices/:id (thay đổi thông tin thiết bị theo id).

Toàn bộ các route được gom lại trong routes/index.js và đặt dưới tiền tố chung /v1; mỗi yêu cầu đến nhóm /v1 đều phải kèm khóa bảo mật x-api-key hợp lệ mới được xử lý.

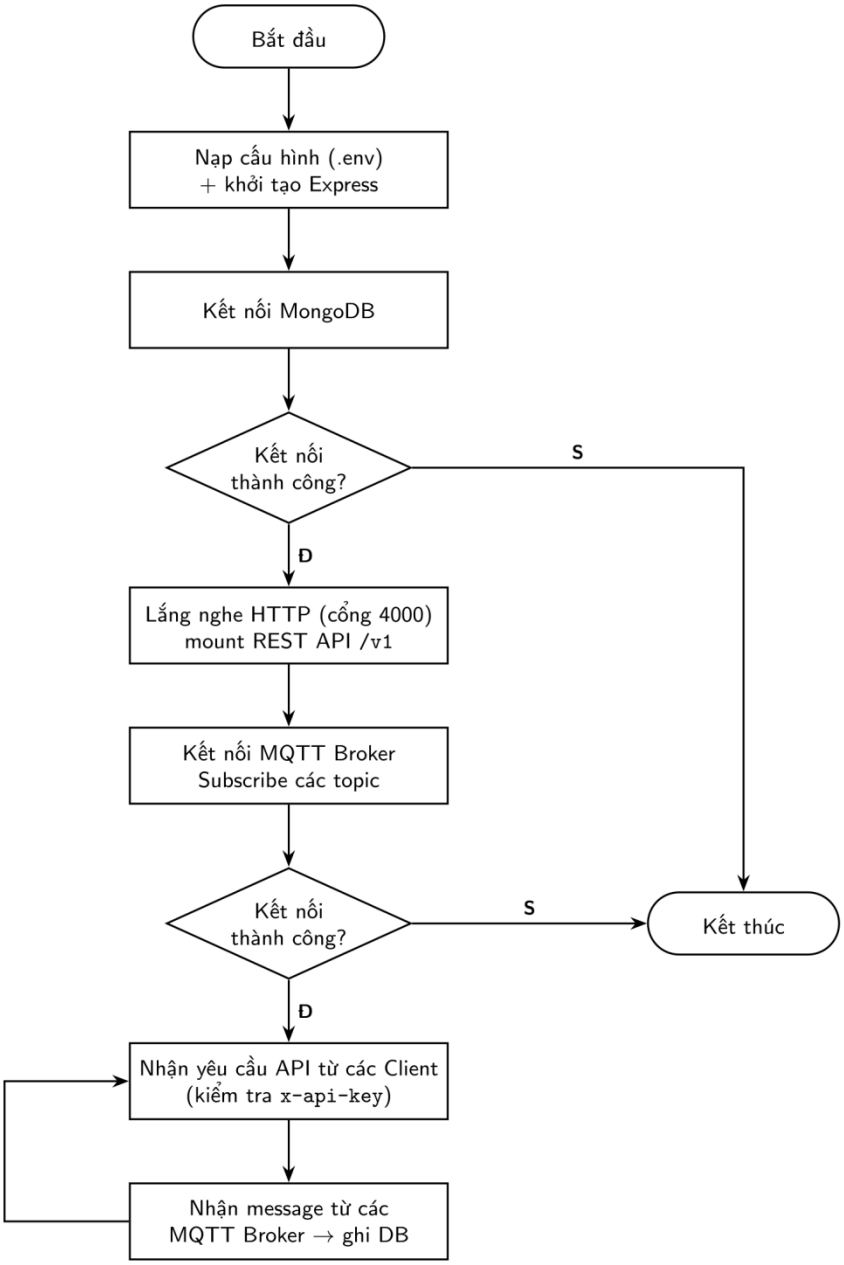
Tập tin server.js là chương trình chính, khởi tạo kết nối tới cơ sở dữ liệu MongoDB và máy chủ MQTT HiveMQ, đồng thời nạp các routes để client có thể sử dụng. Bên cạnh đó, tập tin mqttHandler.js đảm nhận vai trò cầu nối giữa MQTT và MongoDB: lắng nghe dữ liệu cảm biến và trạng thái mà Gateway gửi lên để ghi vào cơ sở dữ liệu, và phát các lệnh điều khiển xuống Gateway. Thư mục lib/ chứa các hàm mã hóa gói tin nhị phân (encodeControl.js, encodeConfig.js) phục vụ việc đóng gói lệnh điều khiển và cấu

hình gửi tới phần cứng. Thư mục scripts/ chứa các tiện ích nạp dữ liệu mẫu vào cơ sở dữ liệu (seed.js, seedMongo.js).

Tập tin .env dùng để khai báo đường dẫn kết nối MongoDB (MONGODB_URI), khóa bảo mật API (API_KEY) và thông tin máy chủ MQTT HiveMQ (HOST, PORT, USERNAME, PASSWORD). Tập tin .gitignore đảm bảo các tập tin quan trọng như .env không bị đăng công khai lên GitHub, đồng thời loại trừ thư viện đã cài đặt (node_modules) khỏi mã nguồn được lưu trữ.

3.3.2.3.2 Thiết kế luồng hoạt động của Backend

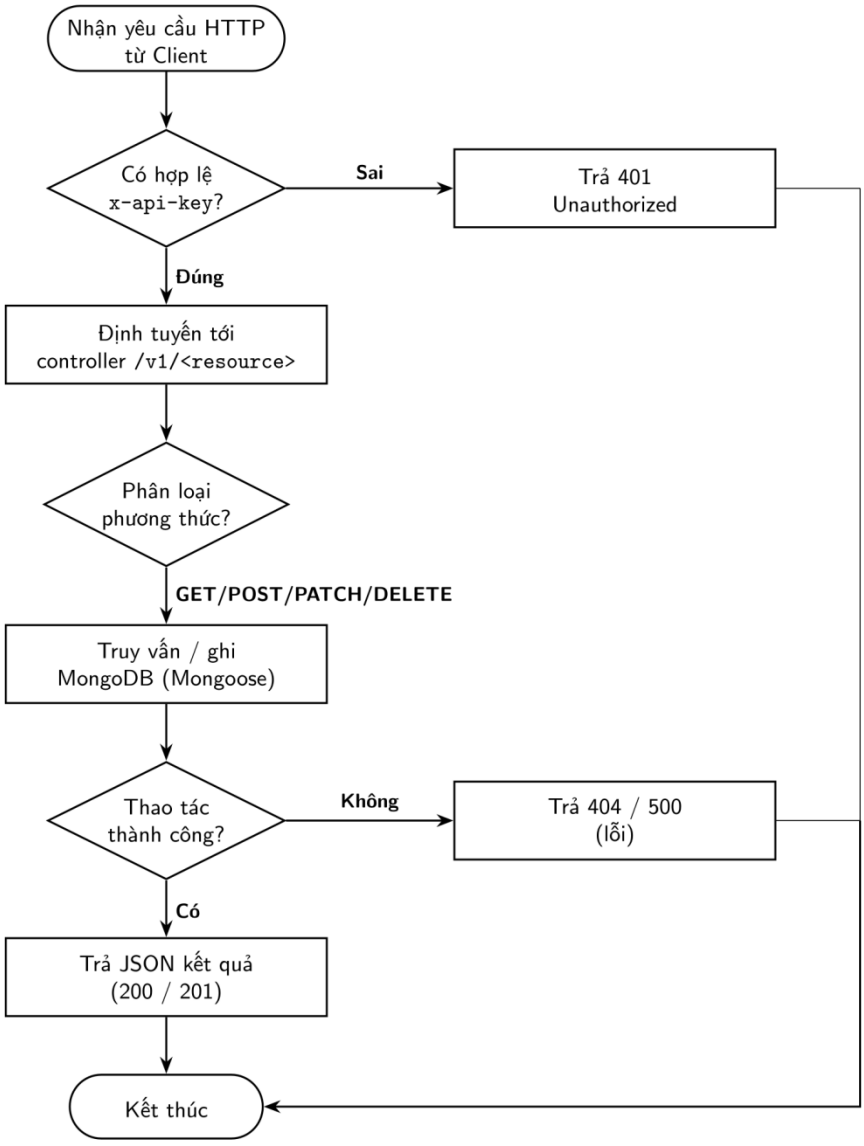
Sau khi xây dựng kiến trúc thư mục và phân tách các chức năng theo mô hình MVC, nhóm nghiên cứu tiến hành thiết kế luồng hoạt động chính nhằm thể hiện cách các thành phần trong hệ thống phối hợp với nhau để thực hiện các chức năng cốt lõi. Việc xác định rõ luồng hoạt động giúp hệ thống dễ mở rộng, thuận tiện bảo trì và tối ưu hiệu năng.



Hình 3.21: Lưu đồ hoạt động tổng quan của Backend.

Hình 3.21 mô tả luồng hoạt động tổng quan của Backend. Khi khởi động, chương trình nạp cấu hình từ tập tin .env và khởi tạo ứng dụng Express, sau đó tiến hành kết nối tới cơ sở dữ liệu MongoDB. Nếu kết nối thất bại, chương trình ghi log lỗi và kết thúc ngay; nếu thành công, Backend mở cổng lắng nghe HTTP (cổng 4000) và gắn các tuyến REST API dưới tiền tố /v1. Tiếp theo, Backend kết nối tới MQTT Broker và đăng ký

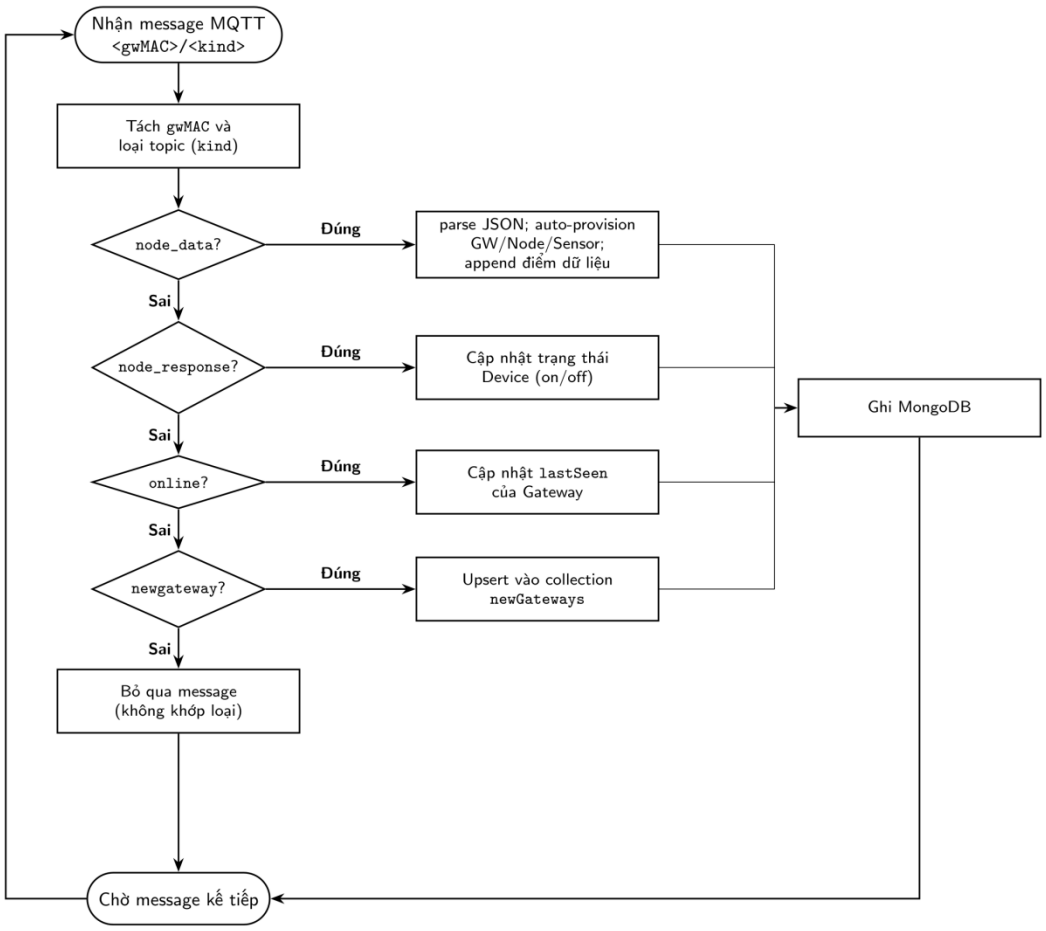
(subscribe) các topic cần thiết; nếu kết nối MQTT thất bại thì kết thúc. Khi cả hai kết nối đã sẵn sàng, Backend bước vào vòng phục vụ liên tục: vừa tiếp nhận và xử lý các yêu cầu API từ phía Client (có kiểm tra khóa bảo mật x-api-key), vừa nhận các message từ MQTT Broker để ghi vào cơ sở dữ liệu. Hai chức năng này được tách thành hai luồng riêng, mô tả chi tiết ở Hình 3.22 và Hình 3.23.



Hình 3.22: Lưu đồ luồng hoạt động thực hiện API của Backend

Hình 3.22 trình bày luồng xử lý một yêu cầu API. Khi nhận được yêu cầu HTTP từ Client, Backend kiểm tra header x-api-key; nếu thiếu hoặc sai, hệ thống trả về mã 401 (Unauthorized) và kết thúc. Nếu hợp lệ, yêu cầu được định tuyến tới controller tương

ứng theo tài nguyên (/v1/<resource>). Tùy theo phương thức (GET, POST, PATCH, DELETE), controller thực hiện thao tác truy vấn hoặc ghi dữ liệu trên MongoDB thông qua Mongoose. Nếu thao tác thành công, Backend trả về dữ liệu dạng JSON kèm mã 200 hoặc 201; nếu không tìm thấy bản ghi hoặc phát sinh lỗi, hệ thống trả về mã 404 hoặc 500 và kết thúc.



Hình 3.23: Lưu đồ luồng hoạt động thực hiện MQTT của Backend.

Hình 3.23 mô tả luồng xử lý dữ liệu MQTT. Mỗi khi nhận được một message từ Broker, Backend tách phần địa chỉ MAC của Gateway (gwMAC) và loại topic (kind) từ tên topic, rồi phân loại để xử lý: với topic node_data, hệ thống phân tích JSON, tự động khởi tạo Gateway/Node/Sensor nếu chưa có và bổ sung điểm dữ liệu cảm biến; với node_response, cập nhật trạng thái bật/tắt của thiết bị (Device); với online, cập nhật thời điểm hoạt động gần nhất (lastSeen) của Gateway; với newgateway, ghi nhận (upsert) Gateway chưa đăng ký vào collection newGateways. Sau khi xử lý, kết quả được ghi

vào MongoDB; nếu message không khớp loại nào thì được bỏ qua. Cuối cùng, Backend quay lại chờ message kế tiếp, tạo thành vòng lặp phục vụ liên tục.

3.3.2.3.3 Cấu hình cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống được tổ chức thành các collection trong MongoDB, mỗi collection tương ứng một schema khai báo bằng Mongoose để ràng buộc kiểu dữ liệu và cấu trúc trường. Dưới đây trình bày khai báo của năm collection cốt lõi (User, Gateways, Node, Devices, Sensors), tiếp đó là danh sách API và cơ chế bảo mật bằng khóa x-api-key.

```
const userSchema = new mongoose.Schema({
    _id:       {type: String},
    name:      {type: String},
    birth:     {type: String},
    phone:     {type: String},
    email:     {type: String},
    username:  {type: String, required: true},
    password:  {type: String, required: true},
    avatar:    {type: String},
    home:      {type: String},
    gatewayId: [{type: String}]
});
```

Hình 3.24: Khai báo các đối tượng có trong User.

Collection User lưu thông tin tài khoản người dùng, gồm khóa chính _id dạng chuỗi, các trường thông tin cá nhân (name, birth, phone, email, avatar, home), cặp username và password bắt buộc, cùng gatewayId dạng mảng chứa các gateway mà người dùng quản lý.

```
const gatewaySchema = new mongoose.Schema({
    _id:               {type: String},
    userId:            {type: String},
    projectName:       {type: String},
    gatewayName:       {type: String},
    gatewayAddress:    {type: String},
    gatewayDesc:       {type: String},
    gatewayCreatedDay: {type: String},
    nodeId:            [{type: String}],
    mac:               {type: String},
    registered:        {type: Boolean, default: false},
    lastSeen:          {type: String}
});
```

Hình 3.25: Khai báo các đối tượng có trong Gateways.

Collection Gateways lưu thông tin gateway: khóa chính _id, userId (chủ sở hữu), các trường dự án (projectName, gatewayName, gatewayAddress, gatewayDesc, gatewayCreatedDay), mảng nodeId, và các trường phần cứng mac, registered, lastSeen.

```
const nodeSchema = new mongoose.Schema({
    _id:            {type: String},
    name:           {type: String, default: ""},
    mac:            {type: String},
    registered:     {type: Boolean, default: false},
    online:         {type: Boolean, default: false},
    lastSeen:       {type: String},
    gatewayId:      {type: String},
    nodePosition:   {type: String},
    nodeCreatedDay: {type: String},
    nodeAddh:       {type: String},
    nodeAddl:       {type: String},
    nodeDesc:       {type: String},
    nodeAddress:    {type: String}
});
```

Hình 3.26: Khai báo các đối tượng có trong Nodes.

Collection Node lưu thông tin mỗi node (nhà kính): khóa chính _id, name, mac, các cờ registered và online, lastSeen, gatewayId quản lý cùng các trường vị trí và địa chỉ LoRa (nodePosition, nodeCreatedDay, nodeAddh, nodeAddl, nodeDesc, nodeAddress).

```
const deviceSchema = new mongoose.Schema({
    _id:         {type: String},
    gatewayId:   {type: String},
    nodeId:      {type: String},
    name:        {type: String},
    pin:         {type: String},
    description: {type: String},
    status:      {type: Boolean, default: false}
});
```

Hình 3.27: Khai báo các đối tượng có trong Devices.

Collection Devices lưu thiết bị điện điều khiển được (relay): khóa chính _id, gatewayId, nodeId, name, pin (định danh chân relay), description và status (trạng thái bật/tắt).

```
const sensorSchema = new mongoose.Schema({
    _id:        {type: String},
    name:       {type: String},
    gatewayId:  {type: String},
    nodeId:     {type: String},
    sensorType: {type: Number},
    data: {
        type: mongoose.Schema.Types.Mixed,
        default: {}
        // { today:   { dataMinute:[{time,value}],
        //              dataHour:  [{time,value}] },
        //   pastDay: [{ date, dataMinute:[...],
        //               dataHour:[...] }] }
    }
});
```

Hình 3.28: Khai báo các đối tượng có trong Sensors.

Collection Sensors lưu dữ liệu cảm biến: khóa chính _id, name, gatewayId, nodeId, sensorType (1=nhiệt độ, 2=độ ẩm, 3=ánh sáng) và data kiểu Mixed. Trường data gồm today (dữ liệu hiện tại theo phút và giờ) và pastDay (dữ liệu các ngày trước) phục vụ vẽ biểu đồ.

Tài nguyên	Endpoint (tiền tố /v1)	Phương thức
Người dùng	/v1/users	GET, POST, PATCH, DELETE
	/v1/users/register	POST
	/v1/users/login	POST
Gateway	/v1/gateways	GET, POST, PATCH, DELETE
Node	/v1/nodes	GET, POST, PATCH, DELETE
Cảm biến	/v1/sensors	GET, POST, PATCH, DELETE
Thiết bị	/v1/devices	GET, POST, PATCH, DELETE
Lịch hẹn giờ	/v1/schedules	GET, POST, PATCH, DELETE
Điều kiện tự động	/v1/conditions	GET, POST, PATCH, DELETE
Gateway mới	/v1/newGateways	GET, POST, PATCH, DELETE
Thông báo	/v1/notifies	GET, POST, PATCH, DELETE

Hình 3.29: Danh sách các API cung cấp cho Website.

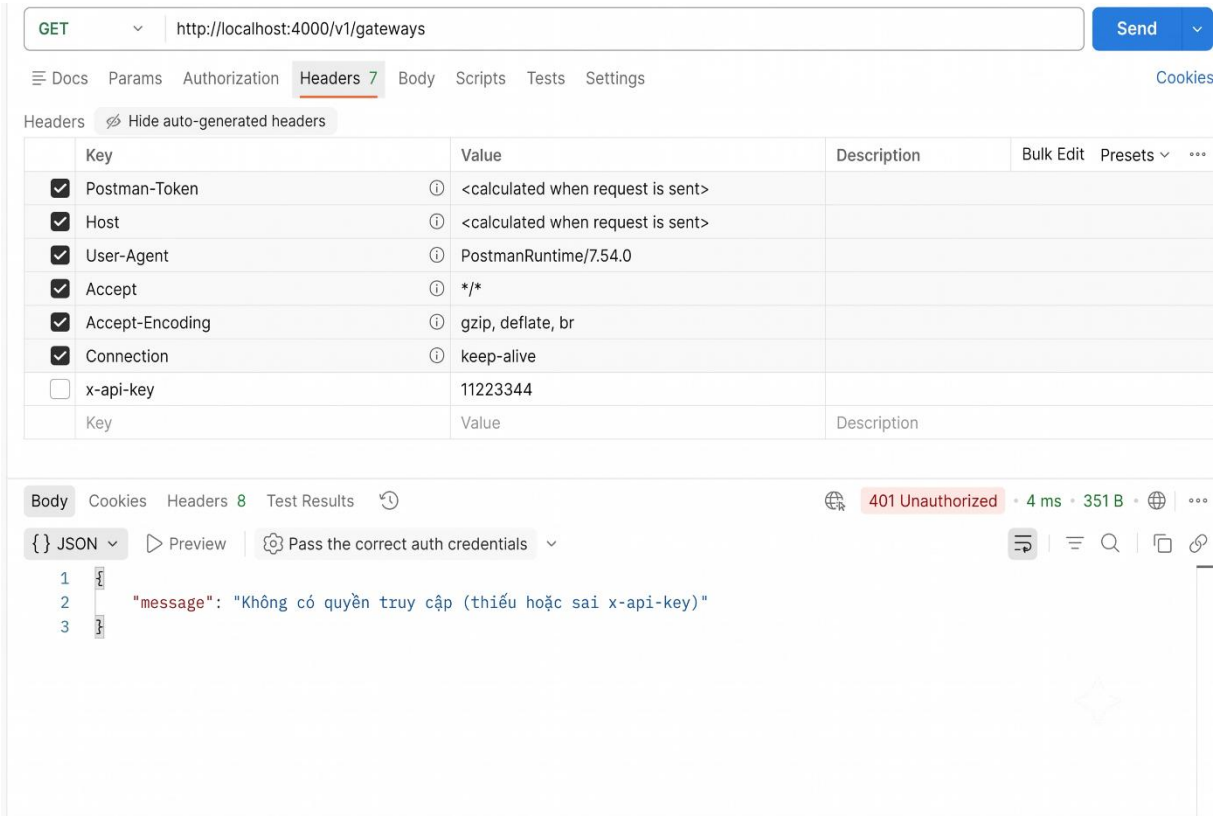
Mỗi collection được expose thành các endpoint REST dưới tiền tố /v1 với đầy đủ phương thức GET, POST, PATCH, DELETE. Riêng tài nguyên người dùng có thêm hai endpoint POST /v1/users/register và POST /v1/users/login phục vụ đăng ký và đăng nhập.

```
PORT=4000
MONGODB_URI=mongodb://localhost:27017/greenhouse_datn
API_KEY=greenhouse-dev-key-2026

MQTT_HOST=********.s1.eu.hivemq.cloud
MQTT_PORT=8883
MQTT_USERNAME=********
MQTT_PASSWORD=********
```

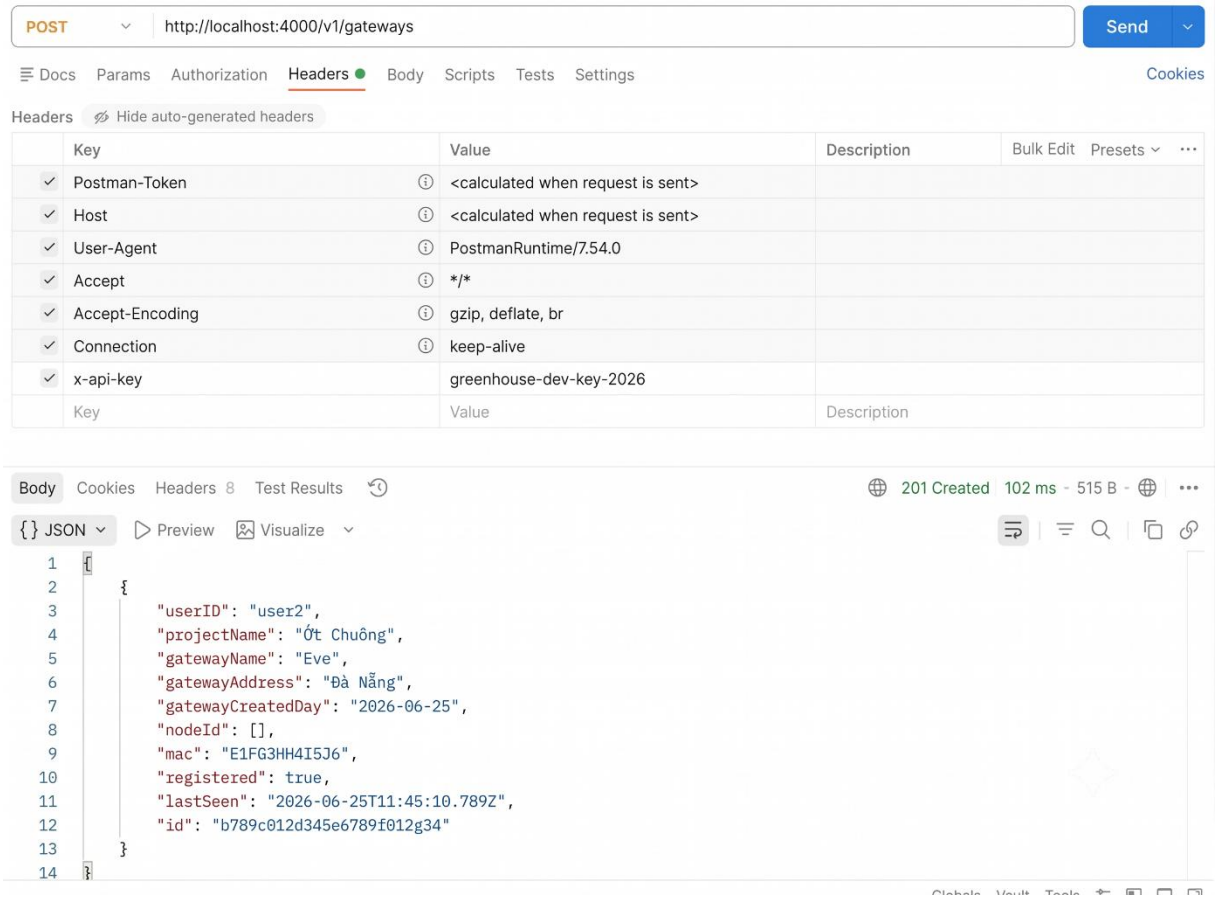
Hình 3.30: Mã Key để bảo mật API.

Khóa bảo mật API_KEY được khai báo trong tập tin .env và không được đẩy lên GitHub. Mọi yêu cầu tới các endpoint /v1 đều phải kèm header x-api-key trùng với giá trị này thì mới được phục vụ.



Hình 3.31: Khi không nhập key hoặc key sai sẽ không phản hồi.

Khi gửi yêu cầu mà thiếu hoặc sai header x-api-key, máy chủ trả về mã 401 (Unauthorized) kèm thông báo "Không có quyền truy cập", không trả dữ liệu.



Hình 3.32: Nhập API key đúng.

Khi header x-api-key đúng với khóa đã khai báo, máy chủ trả về mã 200 cùng dữ liệu JSON tương ứng — ví dụ danh sách gateway của người dùng.

3.3.3 Giao diện người dùng

Trong thực tế, người dùng thường xuyên cần truy cập Internet từ nhiều thiết bị khác nhau như máy tính văn phòng, điện thoại cá nhân hay máy tính bảng khi di chuyển. Việc cài đặt ứng dụng riêng trên mỗi thiết bị không những mất thời gian mà còn gây khó khăn trong việc bảo trì và cập nhật. Bên cạnh đó, nhu cầu truy cập nhanh chóng, mọi lúc mọi nơi và khả năng chia sẻ thông tin dễ dàng giữa các người dùng là rất quan trọng. Vì thế, việc lựa chọn phát triển giao diện người dùng dưới dạng Website là hợp lý, vì nó cho phép truy cập linh hoạt, không yêu cầu cài đặt, dễ bảo trì và dễ mở rộng về sau.

3.3.3.1 Yêu cầu chức năng Website

Mỗi trang web được thiết kế sẽ tích hợp rất nhiều chức năng để phục vụ cho nhu cầu của người dùng, vì thế việc chia website thành nhiều trang khác nhau là cần thiết để tổ chức nội dung một cách rõ ràng, khoa học và dễ sử dụng cho người truy cập. Mỗi trang đảm nhiệm một chức năng hoặc nội dung riêng biệt như trang chủ, trang điều khiển thiết bị, trang giám sát dữ liệu, trang cài đặt hệ thống,... giúp người dùng dễ dàng định hướng và truy cập đúng thông tin mình cần. Đồng thời, cách tổ chức này còn giúp cải thiện hiệu năng hệ thống, giảm tải dữ liệu cần xử lý trên mỗi trang, từ đó nâng cao trải nghiệm người dùng và đảm bảo tính mở rộng trong tương lai khi hệ thống phát triển thêm nhiều tính năng mới.

Với 'Hệ thống quản lý nông trại nhà kính thông minh', mỗi người dùng sẽ quản lý ít nhất một Gateway. Mỗi Gateway này chịu trách nhiệm quản lý, điều phối nhiều nhà kính độc lập đặt phân tán tại các vị trí khác nhau trong nông trại. Tại mỗi nhà kính, hệ thống sẽ tích hợp các thiết bị bao gồm các hệ thống thiết bị điều khiển (như hệ thống tưới nước, quạt thông gió, đèn chiếu sáng) và các cảm biến chuyên dụng (như cảm biến nhiệt độ, độ ẩm đất, ánh sáng) riêng biệt để phục vụ cho việc giám sát đặc thù từng khu vực canh tác. Vì thế, nhóm nghiên cứu đã đưa ra danh sách các trang cần có trên Website để phục vụ người dùng bao gồm:

- **Trang đăng ký:** Hỗ trợ tạo tài khoản người dùng mới trên hệ thống để bắt đầu quản lý nông trại.
- **Trang đăng nhập:** Cung cấp chức năng xác thực quyền truy cập cho người dùng đã có tài khoản thông qua tên đăng nhập và mật khẩu.
- **Trang chủ:** Giới thiệu tổng quan về các tính năng của hệ thống quản lý nhà kính, số lượng cảm biến, thiết bị, thông tin đơn vị phát triển và thông tin liên hệ.
- **Trang thông tin người dùng:** Nơi hiển thị, cập nhật thông tin cá nhân của người quản lý nông trại và cấu hình các cài đặt tài khoản bảo mật.
- **Trang quản lý nông trại:** Cung cấp cái nhìn toàn diện về các mô hình nông trại/dự án mà người dùng đã khởi tạo, bao gồm: danh sách các khu vực dự án, tổng số lượng Gateway, số lượng nhà kính, số lượng thiết bị điều khiển và các

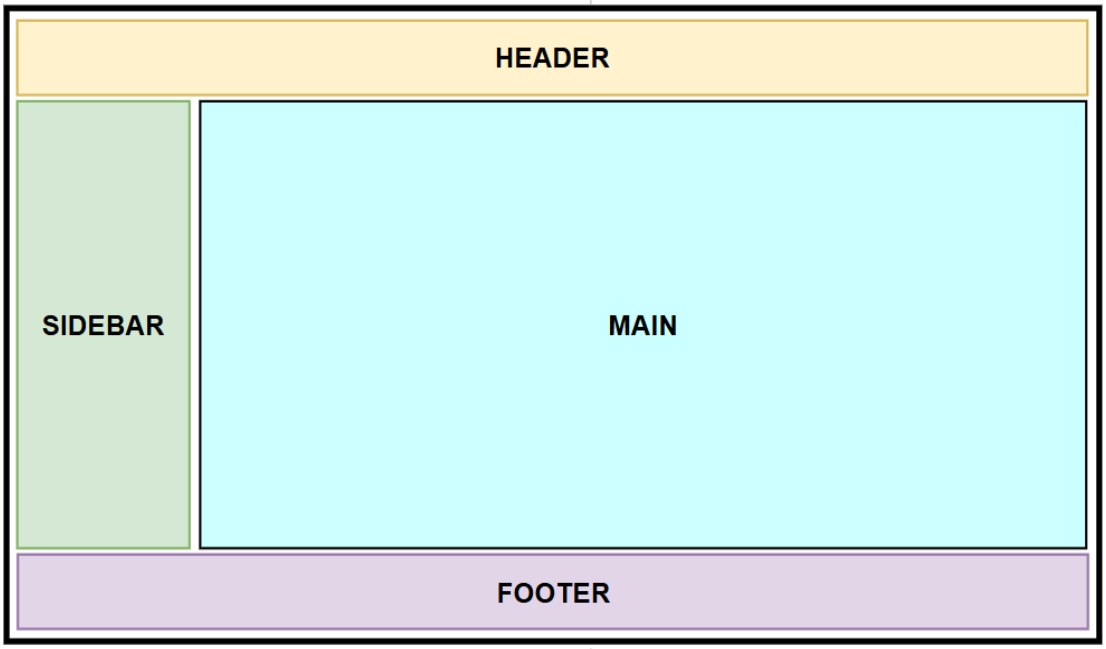
đầu cảm biến đang hoạt động. Trang này cũng tích hợp các chức năng thêm, chỉnh sửa hoặc xóa các thực thể quản lý.

- **Trang quản lý Gateway:** Được thiết kế nhằm mục đích quản lý tập trung một bộ điều khiển trung tâm (Gateway) bao gồm tất cả các nhà kính thuộc quyền điều phối của Gateway đó. Trang hỗ trợ so sánh dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng) giữa các nhà kính và cung cấp giao diện điều khiển đồng loạt các thiết bị chấp hành trong phân khu.
- **Trang quản lý nhà kính:** Cho phép người dùng đi sâu vào phân tích chi tiết dữ liệu thời gian thực và lịch sử của duy nhất một nhà kính cụ thể. Tại đây, người dùng có thể theo dõi sát sao biểu đồ cảm biến và trực tiếp tương tác với các thiết bị điều khiển (hệ thống tưới, quạt thông gió) tại nhà kính đó.

3.3.3.2 Thiết kế giao diện

Để xây dựng một hệ thống thân thiện với người dùng, việc thiết kế giao diện đóng vai trò rất quan trọng. Trong đó, hai yếu tố cơ bản cần được chú trọng là UI (User Interface) và UX (User Experience)[12]. UI là phần giao diện mà người dùng nhìn thấy và tương tác trực tiếp, bao gồm bố cục, màu sắc, nút bấm và biểu tượng. Trong khi đó, UX liên quan đến cảm nhận và trải nghiệm tổng thể của người dùng khi sử dụng hệ thống, đảm bảo hệ thống dễ hiểu, dễ sử dụng và đáp ứng đúng nhu cầu. Việc kết hợp hài hòa giữa UI và UX sẽ giúp hệ thống hoạt động hiệu quả và nâng cao sự hài lòng và thích thú của người dùng.

Hệ thống sẽ sử dụng bố cục đơn giản bao gồm header, footer, sidebar và main (Hình 3.33).



Hình 3.33: Cấu trúc Website cơ bản.

Trong đó, header nằm ở đầu trang, chứa logo, tiêu đề hệ thống và thanh điều hướng chính, giúp người dùng dễ dàng truy cập các khu vực quan trọng. Footer được đặt ở cuối trang, thường hiển thị thông tin liên hệ, bản quyền và liên kết hỗ trợ. Sidebar là thanh bên, đóng vai trò điều hướng phụ hoặc hiển thị các chức năng, danh mục bổ sung theo từng vai trò người dùng. Cuối cùng, khu vực main là phần nội dung trung tâm, nơi hiển thị thông tin chính của từng trang, như dữ liệu, biểu mẫu hay bảng điều khiển. Việc bố trí các thành phần này một cách hợp lý giúp giao diện trở nên dễ sử dụng, trực quan và thuận tiện cho người dùng trong quá trình tương tác với hệ thống nhằm tăng giá trị UX.

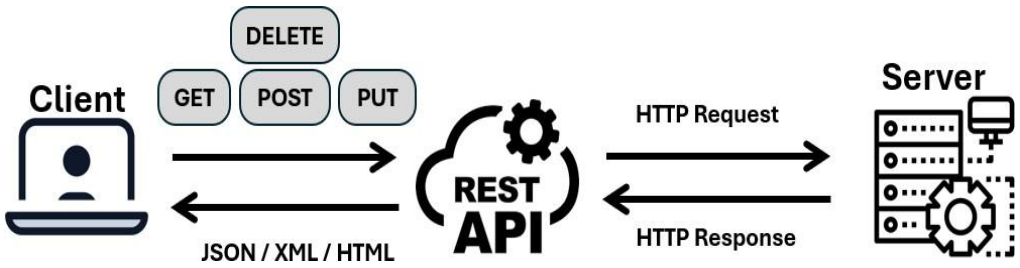
Ngoài ra giao diện còn được thiết kế theo nguyên tắc nhất quán về màu sắc, kiểu chữ và cách bố trí thành phần nhằm tạo cảm giác chuyên nghiệp và dễ nhận diện thương hiệu. Hệ thống ưu tiên thiết kế phản hồi (responsive) để đảm bảo hiển thị tốt trên nhiều thiết bị khác nhau như máy tính, máy tính bảng và điện thoại di động vì yêu cầu phải có sự linh hoạt trong quá trình quản lý, người dùng có thể ở bất cứ đâu, bằng bất cứ thiết bị gì thì vẫn có khả năng giám sát và điều khiển được hệ thống.

Thêm vào đó, để nâng cao trải nghiệm người dùng, hệ thống sẽ lưu trữ thông tin đăng nhập của người dùng trong phiên làm việc trước đó bằng cách sử dụng cơ chế cookie hoặc local storage. Nhờ vậy, khi người dùng quay lại website, họ không cần phải đăng nhập lại từ đầu, giúp tiết kiệm thời gian và tạo cảm giác tiện lợi, liên tục trong quá trình sử dụng. Việc lưu trữ này vẫn đảm bảo các yếu tố bảo mật thông tin cá nhân, đặc biệt trong các chức năng liên quan đến xác thực và quyền truy cập.

Cuối cùng, việc phản hồi thao tác thành công hay thất bại (ví dụ: đăng nhập thành công) cũng là điều cực kỳ cần thiết để hệ thống giao tiếp được với người dùng, từ đó rõ ràng, minh bạch và dễ sử dụng hơn, nâng cao trải nghiệm và sự hài lòng của người dùng.

3.3.3.3 Giao tiếp giữa Website và cơ sở dữ liệu

Trong một hệ thống Website hoàn chỉnh, việc giao tiếp giữa giao diện người dùng (Front-end) và cơ sở dữ liệu không diễn ra trực tiếp mà được thực hiện trung gian thông qua một tầng API – cụ thể là RESTful API [13] (Hình 3.34). Cách tiếp cận này giúp đảm bảo tính phân tách rõ ràng giữa Front-end (giao diện hiển thị) và Back-end (xử lý logic nghiệp vụ và dữ liệu), từ đó tối ưu hóa tính linh hoạt, giúp hệ thống dễ dàng bảo trì và mở rộng hiệu quả trong tương lai.



Hình 3.34: Mô hình giao tiếp giữa Client và Server thông qua REST API.

Khi người dùng thực hiện một thao tác trên giao diện Website (chẳng hạn như điền thông tin đăng nhập hoặc gửi biểu mẫu đăng ký), hệ thống Front-end sẽ gửi một yêu cầu HTTP (HTTP Request) đến tầng REST API. Yêu cầu này được thực hiện thông qua các phương thức chuẩn bao gồm: GET (lấy dữ liệu), POST (tạo dữ liệu mới), PUT (cập nhật dữ liệu) hoặc DELETE (xóa dữ liệu).

Các ví dụ cụ thể về định tuyến API trong hệ thống bao gồm:

- **GET /api/users/123:** Lấy thông tin chi tiết của người dùng có định danh id là 123.
- **POST /api/users:** Khởi tạo một tài khoản người dùng mới vào hệ thống.
- **PUT /api/users/123:** Cập nhật thông tin của người dùng có định danh id là 123.
- **DELETE /api/users/123:** Xóa hoàn toàn người dùng có định danh id là 123 khỏi hệ thống.

Tại phía Back-end, máy chủ sẽ tiếp nhận yêu cầu, xử lý các logic nghiệp vụ liên quan (ví dụ: xác thực tên đăng nhập và mã hóa mật khẩu, kiểm tra điều kiện dữ liệu), sau đó tiến hành truy vấn hoặc cập nhật thông tin vào cơ sở dữ liệu. Kết quả xử lý thành công sẽ được phản hồi về phía Client dưới định dạng dữ liệu cấu trúc JSON. Nhận được phản hồi này, Front-end sẽ tiếp tục xử lý để cập nhật lại giao diện người dùng (hiển thị thông báo, vẽ biểu đồ trực quan, lưu trữ trạng thái đăng nhập vào Local Storage hoặc Cookies).

Bên cạnh đó, nếu xảy ra lỗi trong quá trình giao tiếp, phía Server sẽ phản hồi lại các mã trạng thái HTTP (HTTP Status Code) tương ứng để thông báo tình trạng hệ thống:

- **400 Bad Request:** Yêu cầu từ Client có cú pháp không hợp lệ hoặc thiếu các trường thông tin bắt buộc.
- **401 Unauthorized:** Người dùng chưa thực hiện xác thực hoặc chuỗi mã định danh (Token) đã hết hạn/không hợp lệ.
- **403 Forbidden:** Người dùng đã xác thực nhưng không có quyền hạn truy cập vào tài nguyên được yêu cầu.
- **404 Not Found:** Không tìm thấy tài nguyên hoặc đường dẫn URL yêu cầu trên hệ thống.
- **409 Conflict:** Yêu cầu bị xung đột dữ liệu (ví dụ: đăng ký tài khoản với email đã tồn tại).
- **422 Unprocessable Entity:** Yêu cầu đúng cú pháp nhưng dữ liệu gửi lên vi phạm quy tắc logic (ví dụ: sai định dạng dữ liệu cảm biến).
- **500 Internal Server Error:** Lỗi phát sinh từ hệ thống máy chủ hoặc gặp sự cố kết nối cơ sở dữ liệu.

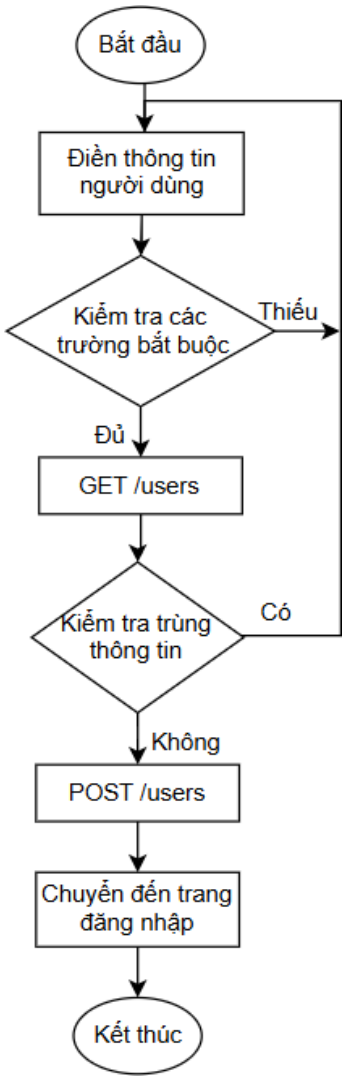
Nhờ việc tổ chức kiến trúc theo mô hình RESTful này, đội ngũ phát triển Front-end và Back-end có thể làm việc độc lập mà không bị phụ thuộc hoàn toàn vào tiến độ của nhau. Các lập trình viên có thể sử dụng các công cụ hỗ trợ như JSON Server (để giả lập cơ sở dữ liệu và API) hoặc Postman (ứng dụng kiểm thử và kiểm tra chức năng API độc lập). Kiến trúc này không chỉ giúp toàn bộ hệ thống trở nên tường minh, dễ quản lý, bảo trì mà còn mở ra khả năng mở rộng linh hoạt trong tương lai, cho phép tích hợp thêm ứng dụng di động (Mobile App), phần mềm quản lý ngoại vi hoặc giao tiếp với các dịch vụ của bên thứ ba.

3.4 THUẬT TOÁN VÀ PHÂN LUỒNG DỮ LIỆU

3.4.1 Quản lý dữ liệu người dùng

3.4.1.1 Chức năng đăng ký người dùng mới

Khi bắt đầu, người dùng chưa có tài khoản phải tiến hành đăng ký người dùng mới.



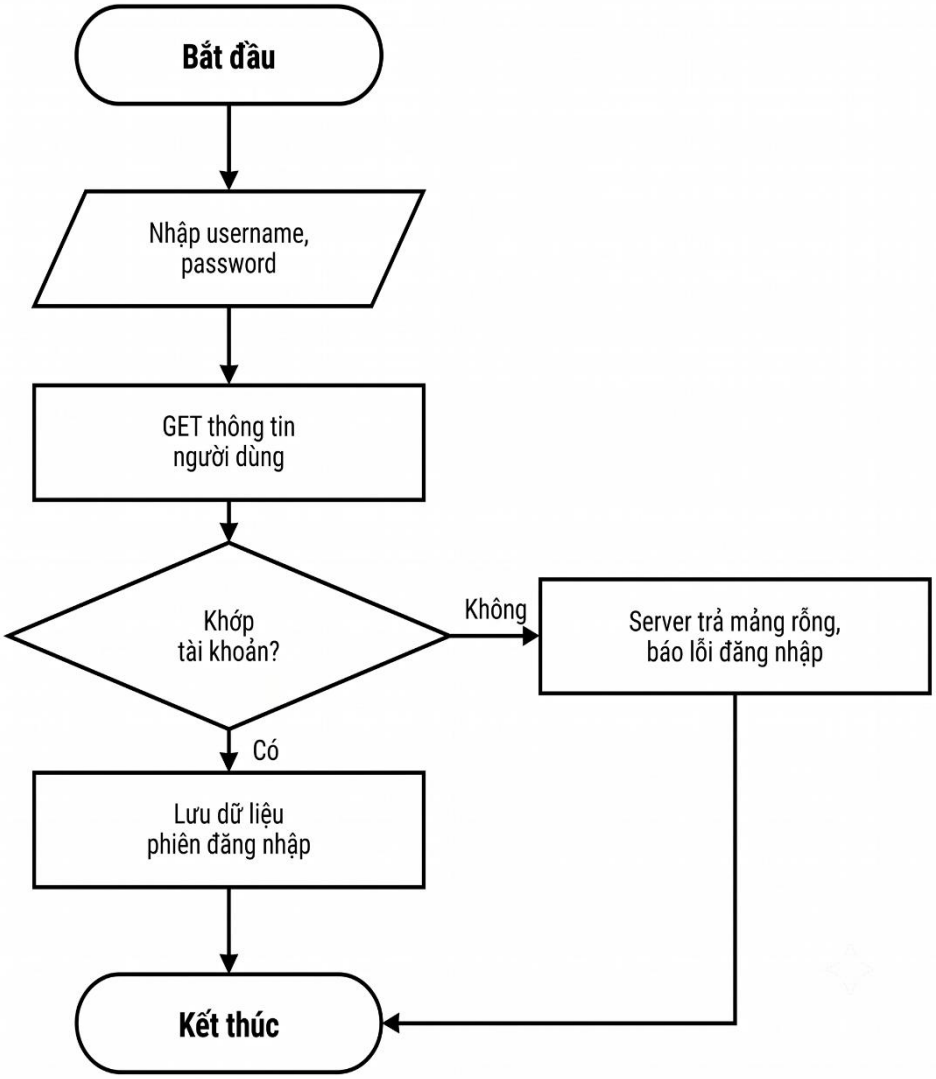
Hình 3.35: Lưu đồ thuật toán đăng ký người dùng.

Chức năng đăng ký người dùng mới được mô tả trên lưu đồ Hình 3.35. Đầu tiên, người dùng sẽ nhập thông tin cá nhân bao gồm một vài trường bắt buộc như: Tên, tên đăng nhập, số điện thoại, tài khoản, mật khẩu, sau đó nhấn đăng ký. Lúc này, Front-end sẽ kiểm tra các trường bắt buộc đã được điền hay chưa, nếu chưa sẽ gửi thông báo đến người dùng để tiếp tục điền thông tin, nếu đã đầy đủ thì sẽ gửi lệnh GET đến server Back-end để kiểm tra thông tin trùng lặp với những tài khoản đã có sẵn và tiếp tục chờ phản hồi bên phía server, trong quá trình này, nếu Server trả về mã lỗi thì sẽ thông báo đến người dùng. Trong trường hợp nhận thành công dữ liệu và không có trùng lặp thông tin thì sẽ tiếp tục gửi đến Server lệnh POST để tạo người dùng mới

và tiếp tục chờ phản hồi. Nếu thành công thì sẽ thông báo cho người dùng và chuyển đến trang đăng nhập.

3.4.1.2 Chức năng đăng nhập

Đối với người dùng đã đăng ký tài khoản từ trước, đây sẽ là luồng đăng nhập dành cho người dùng.



Hình 3.36: Lưu đồ thuật toán đăng nhập.

Thuật toán về chức năng đăng nhập người dùng được mô tả tại Hình 3.36. Trước tiên, người dùng sẽ đăng nhập vào hệ thống thông qua username và password đã đăng ký từ trước, sau đó hệ thống sẽ kiểm tra các trường trùng lặp tương tự chức năng đăng

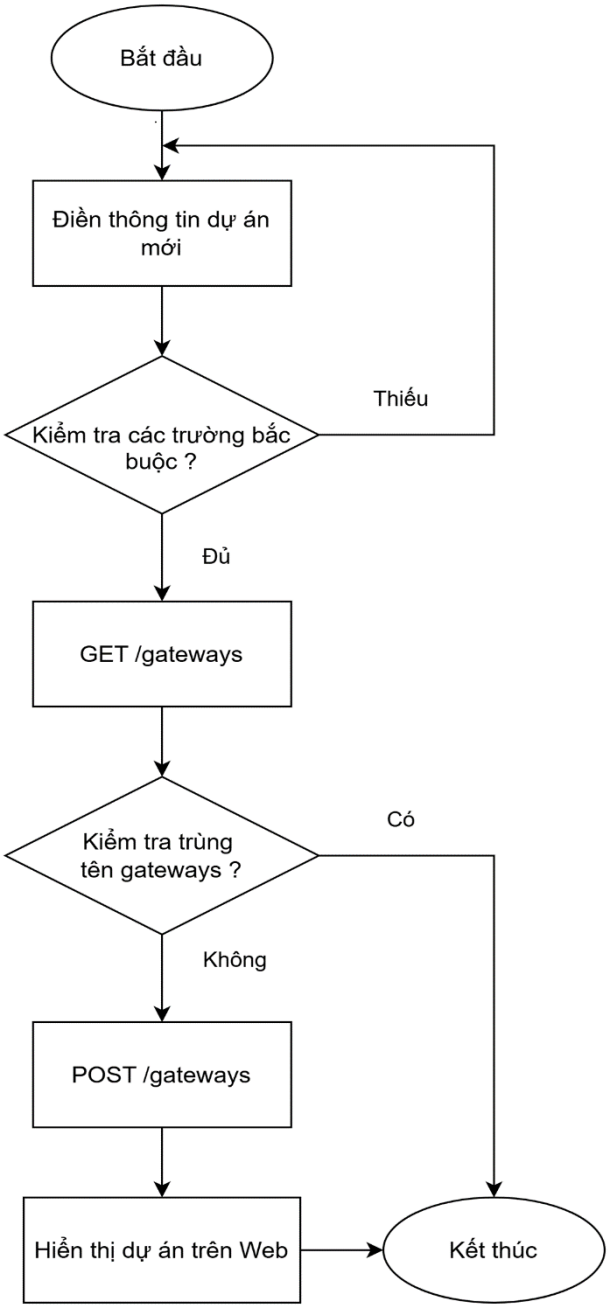
ký, sau đó gửi lệnh GET đến server để lấy thông tin người dùng. Nếu cả hai trường đều khớp, Server sẽ trả về một mảng gồm Struct của người dùng. Sau đó, Website sẽ lưu dữ liệu lại cho những lần đăng nhập sau. Nếu không trùng dữ liệu với người dùng đã đăng ký thì Server sẽ trả về mảng rỗng và báo lỗi.

3.4.2 Quản lý dữ liệu dự án

3.4.2.1 Thao tác với Gateway

- **Tạo mới dự án và Gateway**

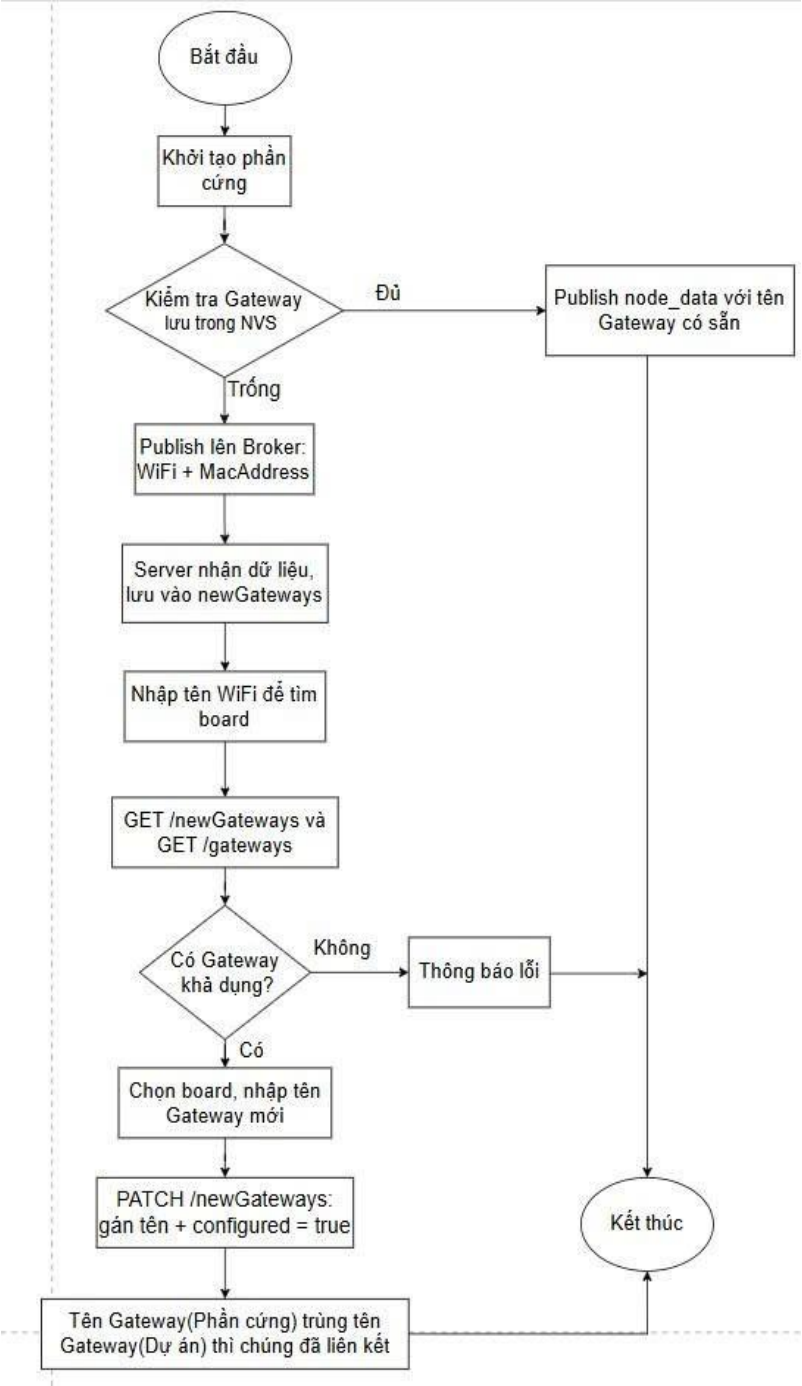
Đối với người dùng lần đầu đăng nhập vào, trang dự án sẽ trống. Người dùng sẽ tiến hành tạo dự án mới.



Hình 3.37: Lưu đồ thuật toán thêm dự án mới

Việc tạo Gateway cũng tương tự với tạo người dùng mới, đều có các bước kiểm tra các trường bắt buộc và các trường trùng thông tin (Hình 3.37). Nhưng với dự án mới, trường duy nhất không được trùng đó chính là tên Gateway - là tên để phân biệt các Topic đăng ký MQTT tránh việc xung đột các gói tin.

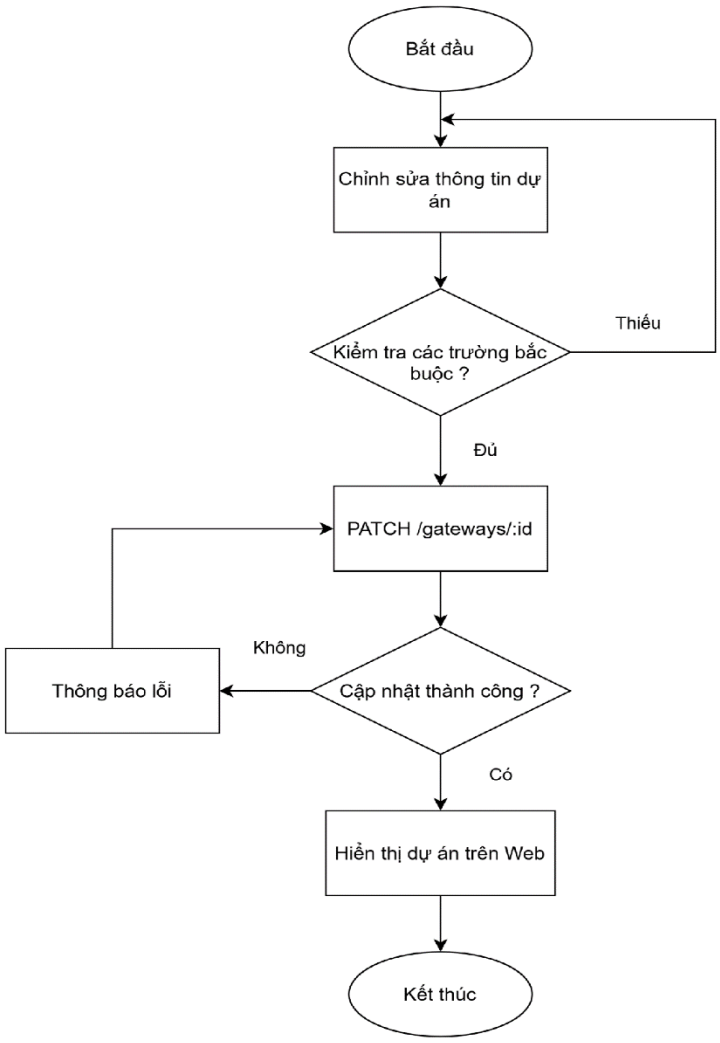
Tuy nhiên, đây chỉ là công đoạn để tạo một Dự án với tên Gateway trên cơ sở dữ liệu chứ chưa hề cấu hình phần cứng. Để kết nối phần cứng với dự án này, chúng ta phải đổi tên Gateway (Phần cứng) trùng với tên Gateway được lưu trên Server.



Hình 3.38: Lưu đồ thuật toán cài đặt phần cứng mới

Lưu đồ Hình 3.38 mô tả việc đưa một Gateway phần cứng mới vào hệ thống theo đúng cách hệ thống vận hành. Khi khởi tạo, Gateway kiểm tra tên đã lưu trong bộ nhớ trong; nếu đã có tên thì nó đăng ký các topic và phát dữ liệu với tên sẵn có. Nếu chưa có tên, Gateway publish lên Broker thông điệp gồm tên WiFi và địa chỉ MAC, Server nhận và lưu vào danh sách Gateway chưa cấu hình. Người dùng nhập tên WiFi để tìm; hệ thống dùng GET /newGateways và GET /gateways để lọc ra các Gateway khả dụng (WiFi khớp và tên chưa bị dự án nào dùng). Khi chọn được board, người dùng đặt tên Gateway mới trùng với tên Gateway của dự án rồi PATCH /newGateways để đánh dấu đã cấu hình.

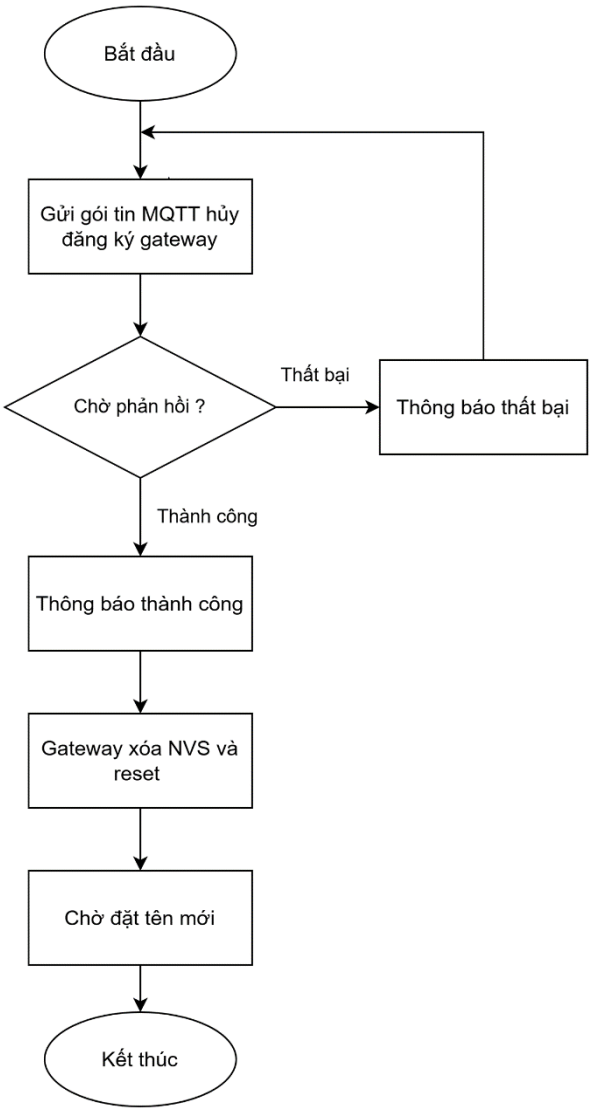
- **Chỉnh sửa dự án và Gateway**



Hình 3.39: Lưu đồ thuật toán chỉnh sửa dự án

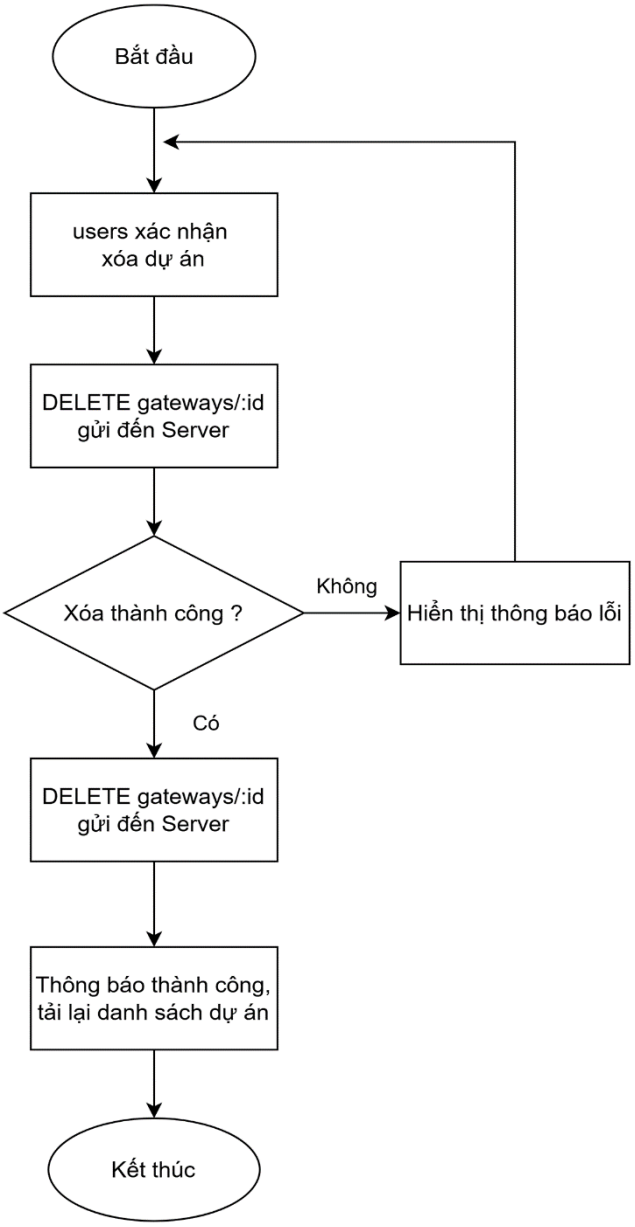
Hình 3.39 mô tả quá trình chỉnh sửa thông tin dự án. Người dùng thay đổi các trường như tên dự án, tên Gateway, địa chỉ; hệ thống kiểm tra các trường bắt buộc rồi gửi lệnh PATCH /gateways/:id để lưu trực tiếp dữ liệu mới lên Server và thông báo kết quả.

- **Xóa dự án và Gateway**



Hình 3.40: Lưu đồ thuật toán xóa tên Gateway (Phần cứng)

Hình 3.40 mô tả thao tác đưa Gateway phần cứng về trạng thái ban đầu. Hệ thống gửi gói tin MQTT để hủy đăng ký xuống Gateway và chờ phản hồi; nếu không nhận được thì thông báo thất bại. Khi thành công, Gateway xóa cấu hình trong bộ nhớ trong (NVS) và tự Reset, trở về trạng thái chưa cấu hình để chờ cài đặt tên mới.

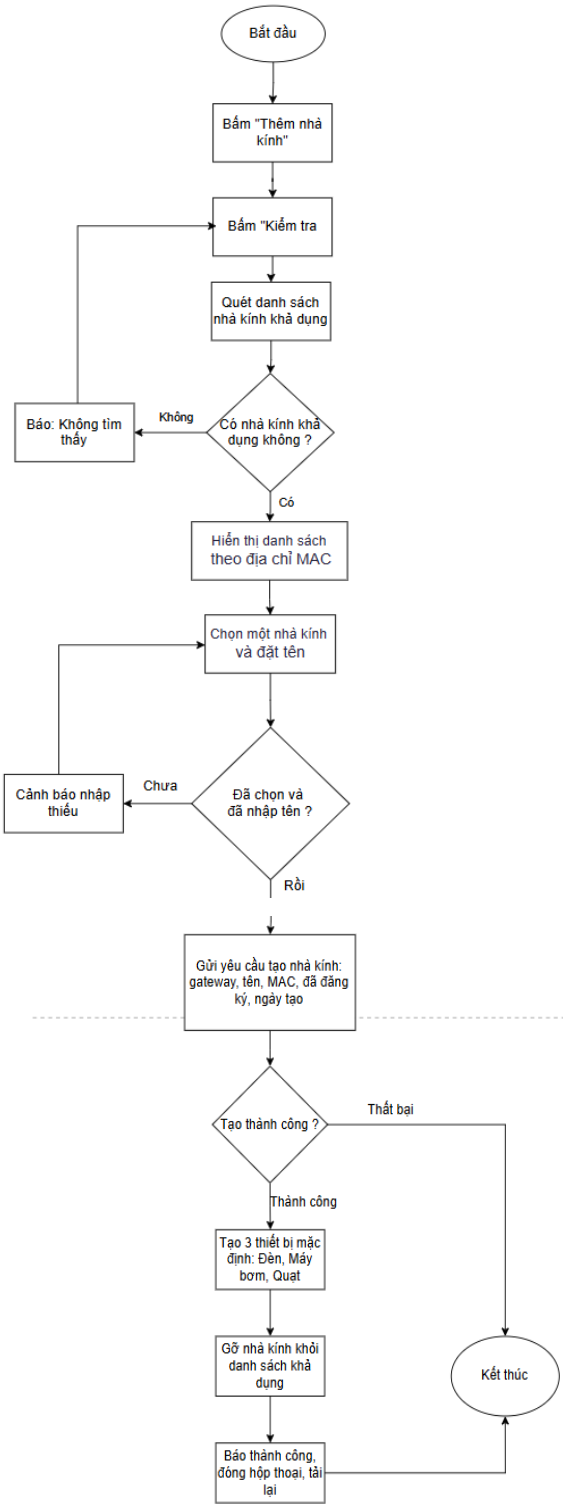


Hình 3.41: Lưu đồ thuật toán xóa dự án

Quá trình xóa dự án được diễn ra như sau: Đầu tiên, người dùng phải xác nhận thao tác xóa thông qua hộp thoại thông báo nhằm tránh xóa nhầm, vì hành động này không thể hoàn tác. Sau khi xác nhận, Website gửi yêu cầu DELETE /gateways/:id đến Server để xóa bản ghi Gateway tương ứng với dự án. Cuối cùng, Website chờ nhận phản hồi từ Server; nếu thành công thì thông báo đến người dùng và tải lại danh sách dự án, còn nếu thất bại thì hiển thị thông báo lỗi.

3.4.2.2 Thao tác với Node

- **Tạo mới node cho Gateway**



Hình 3.42: Lưu đồ thuật toán tạo mới node cho gateway

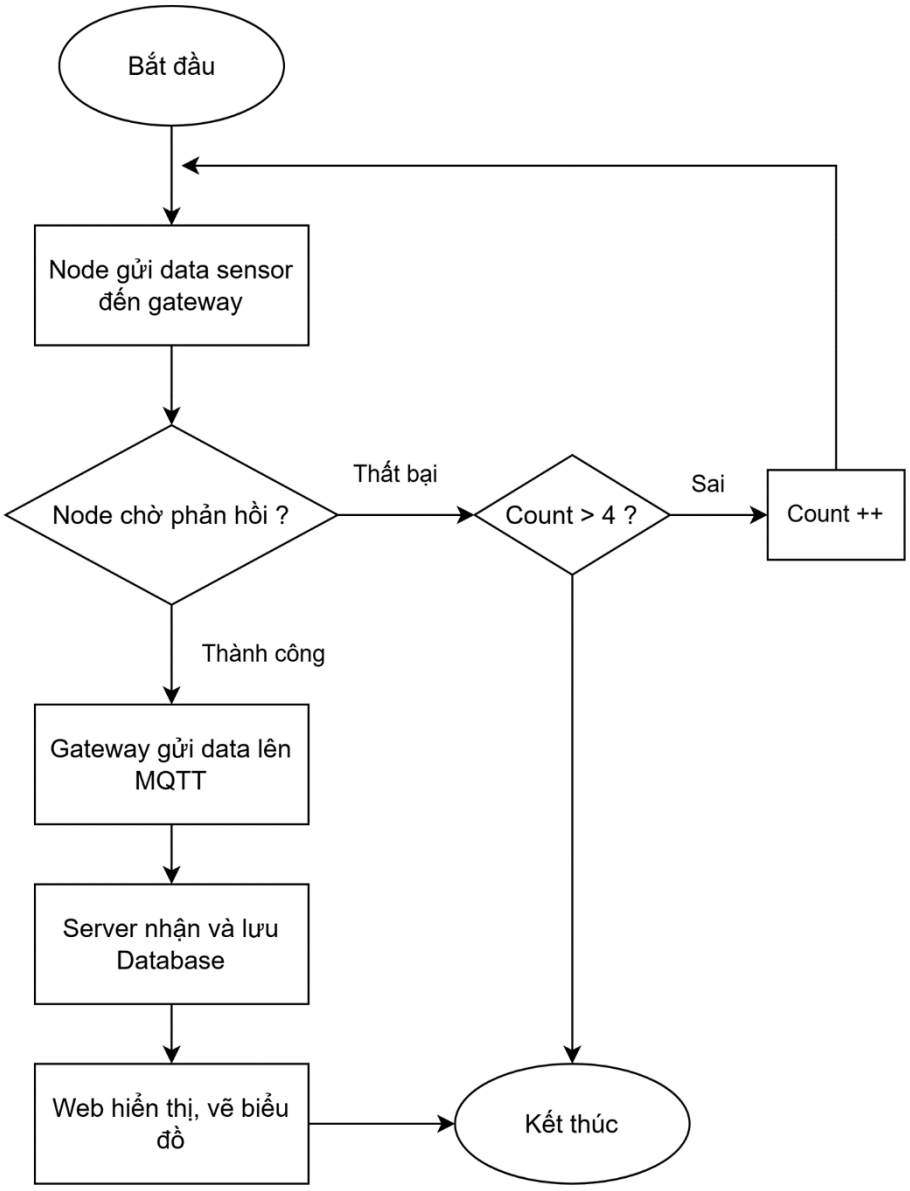
Khi người dùng chọn chức năng "Thêm nhà kính", hệ thống sẽ quét và hiển thị danh sách các nhà kính khả dụng, tức các thiết bị phần cứng đang hoạt động nhưng chưa được gán cho bất kỳ dự án nào. Nếu không tìm thấy nhà kính khả dụng, hệ thống sẽ thông báo để người dùng kiểm tra lại sau.

Người dùng chọn một nhà kính từ danh sách và nhập tên cho nhà kính đó. Trước khi tạo mới, hệ thống kiểm tra tính hợp lệ của dữ liệu đầu vào, bao gồm việc đã chọn nhà kính và đã nhập tên hay chưa. Nếu thông tin chưa đầy đủ, hệ thống sẽ hiển thị cảnh báo tương ứng.

Khi dữ liệu hợp lệ, yêu cầu tạo nhà kính sẽ được gửi đến máy chủ cùng các thông tin như mã dự án, tên nhà kính, địa chỉ MAC, trạng thái đăng ký và thời gian tạo. Sau khi tạo thành công, hệ thống tự động khởi tạo các thiết bị mặc định (Đèn, Máy bơm và Quạt), cập nhật lại danh sách nhà kính khả dụng, đồng thời tải lại dữ liệu để hiển thị nhà kính mới trên giao diện.

Trong trường hợp xảy ra lỗi trong quá trình giao tiếp với máy chủ, hệ thống sẽ thông báo lỗi và dừng thao tác nhằm đảm bảo tính nhất quán của dữ liệu.

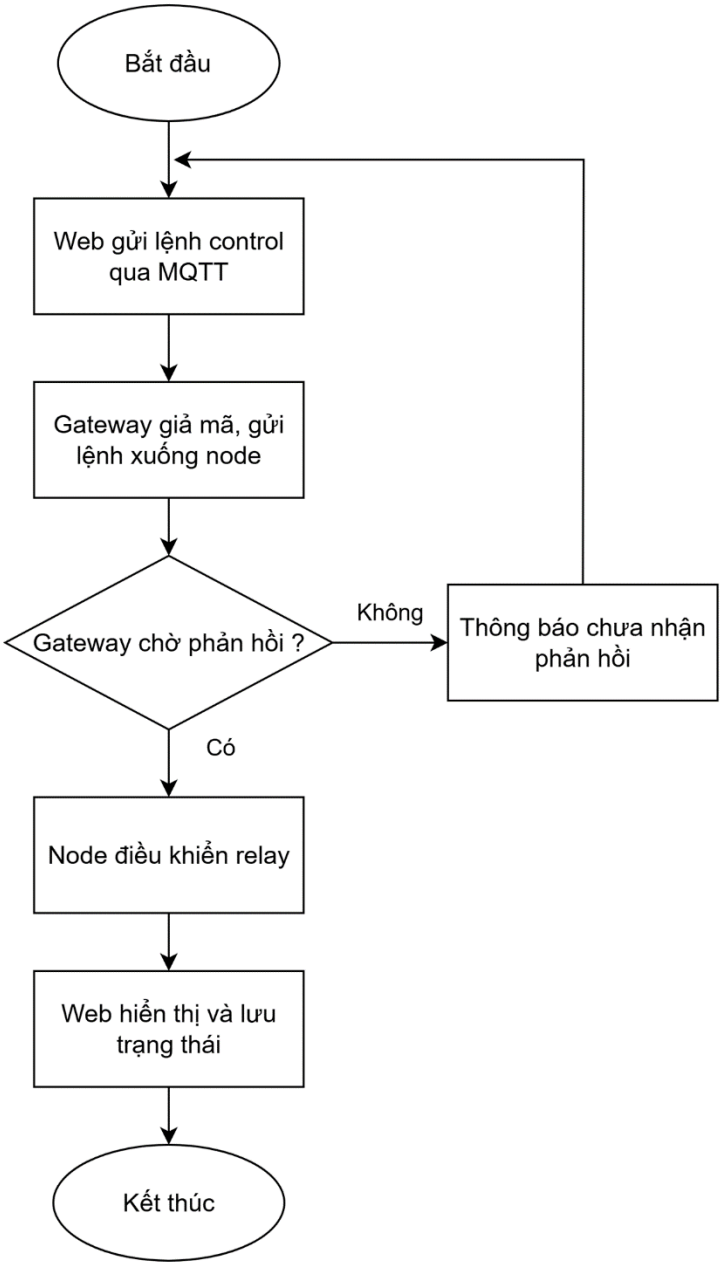
3.4.2.3 Thao tác với cảm biến



Hình 3.43: Lưu đồ thuật toán đọc dữ liệu cảm biến

Thuật toán đọc dữ liệu cảm biến được mô tả trên lưu đồ Hình 3.43. Đầu tiên, Node - nơi chứa các cảm biến môi trường sẽ đọc dữ liệu và gửi đến Gateway chủ, Node sẽ cố gửi 3 lần nếu không nhận được phản hồi. Gateway sau khi nhận được gói tin sẽ phản hồi lại Node và tiếp tục gửi dữ liệu này lên MQTT Broker để Server nhận và lưu vào cơ sở dữ liệu để Website có thể truy suất và vẽ biểu đồ trực quan.

3.4.2.4 Điều khiển thiết bị



Hình 3.44: Lưu đồ thuật toán điều khiển thiết bị

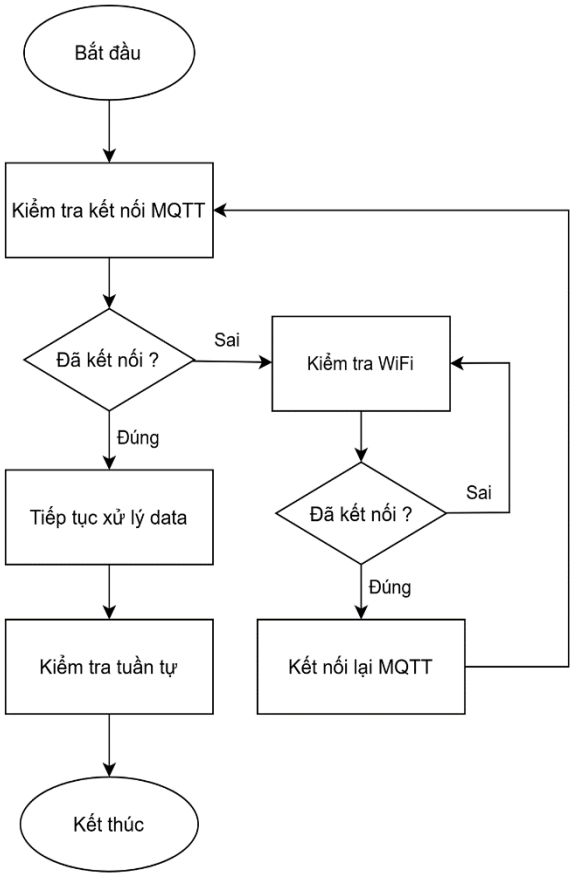
Quy trình điều khiển thiết bị được thực hiện theo cơ chế khép kín từ Web đến phần cứng và ngược lại: khi người dùng bật/tắt công tắc trên Website, một lệnh chứa trạng thái 1/0 cùng định danh chính xác (Gateway – Khu – Nút – Relay) sẽ được gửi qua giao thức MQTT đến Gateway, rồi tiếp tục truyền qua sóng LoRa để Node điều khiển

đóng/ngắt Relay vật lý của thiết bị (đèn, bơm, quạt). Sau đó, hệ thống sẽ chờ phản hồi từ Node để quyết định kết quả: nếu mất kết nối (thất bại), quy trình kết thúc và giữ nguyên giao diện để tránh báo sai thực tế; nếu nhận được xác nhận (thành công), tín hiệu sẽ truyền ngược qua MQTT về Website để cập nhật lại công tắc trên giao diện, đồng thời hệ thống gửi lệnh PATCH cập nhật trạng thái mới vào cơ sở dữ liệu và hiển thị thông báo hoàn tất cho người dùng.

3.4.2.5 Các cơ chế dự phòng và phục hồi khi xảy ra vấn đề

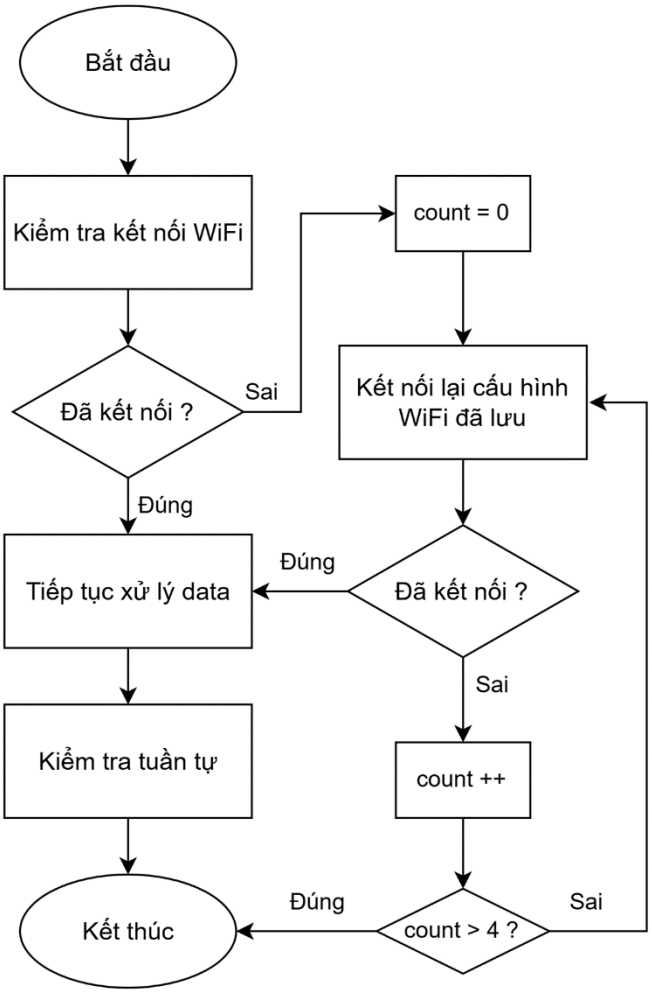
- **KHI MẤT KẾT NỐI SERVER MQTT**

Khi kết nối với máy chủ MQTT bị gián đoạn, hệ thống kiểm tra trạng thái kết nối WiFi trước khi thử kết nối lại. Nếu WiFi hoạt động, hệ thống sẽ cố gắng tái kết nối với máy chủ MQTT. Đồng thời, trong thời gian mất kết nối, dữ liệu nhận được từ LoRa được lưu vào hàng đợi để xử lý sau khi kết nối được khôi phục, hàng đợi này sẽ hoạt động theo vòng, khi đầy thì dữ liệu cảm biến mới sẽ thay thế dữ liệu cảm biến cũ, đồng thời vì dung lượng vi điều khiển hạn chế, nên chỉ cấu hình tối đa 100 gói tin được lưu trong hàng đợi. Âm thanh báo lỗi được phát để thông báo tình trạng mất kết nối (Hình 3.45).



Hình 3.45: Lưu đồ thuật toán sửa chữa kết nối MQTT

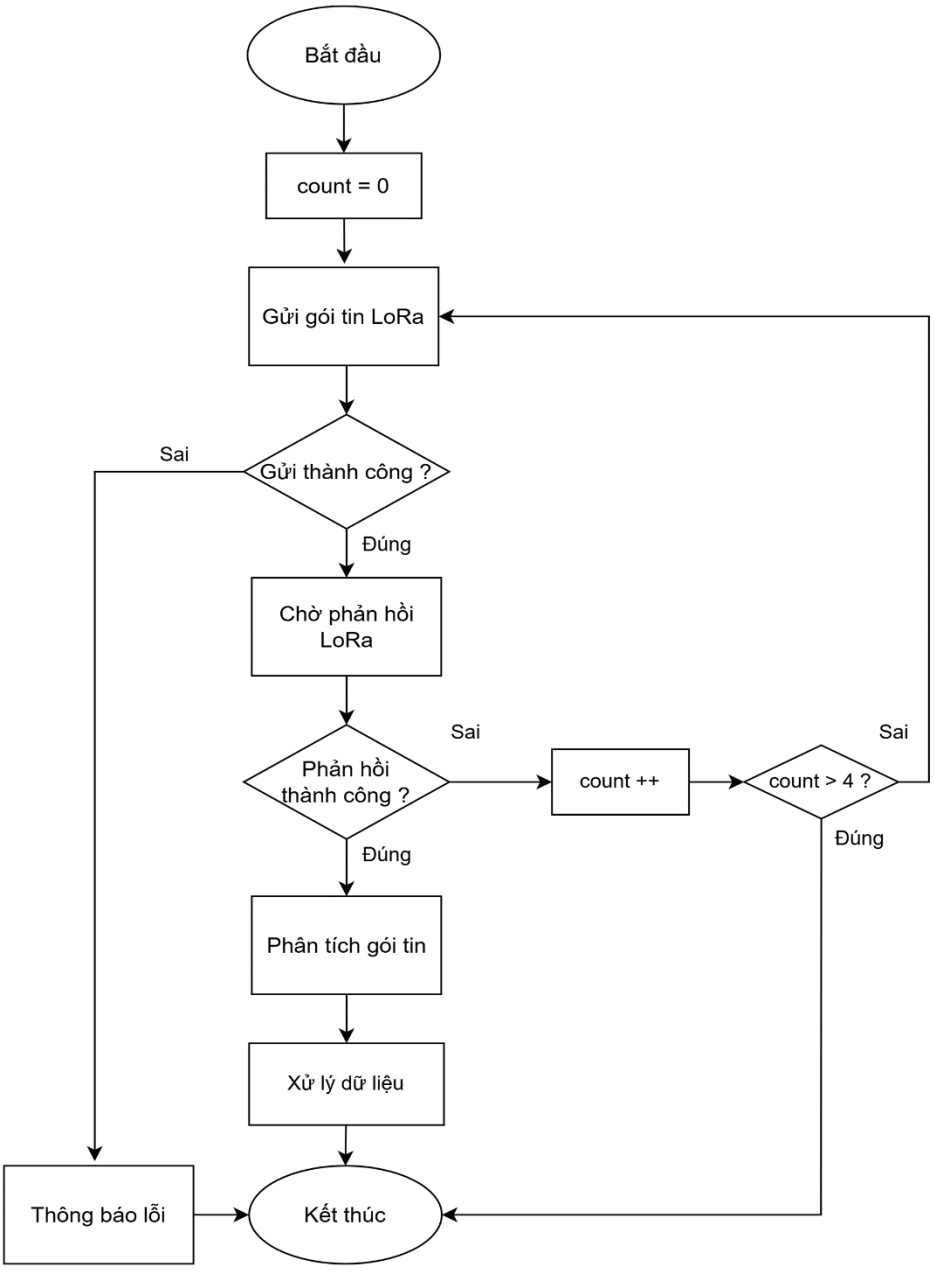
Nếu WiFi bị mất kết nối, hệ thống sẽ tạm hoãn việc thử kết nối lại MQTT và tập trung vào việc khôi phục WiFi. Một tác vụ riêng biệt được chạy để kiểm tra và thử kết nối lại WiFi. Trong thời gian này, dữ liệu LoRa tiếp tục được lưu vào hàng đợi, đảm bảo không mất dữ liệu khi WiFi hoặc MQTT không khả dụng. Nếu quá số lần kết nối lại WiFi thì hệ thống sẽ Reset WiFi và phát ra một WiFi từ ESP32 để người dùng kết nối và cấu hình thông số WiFi mới (Hình 3.46).



Hình 3.46: Lưu đồ thuật toán sửa chữa kết nối WiFi

- **XỬ LÝ GỬI GÓI TIN LORA**

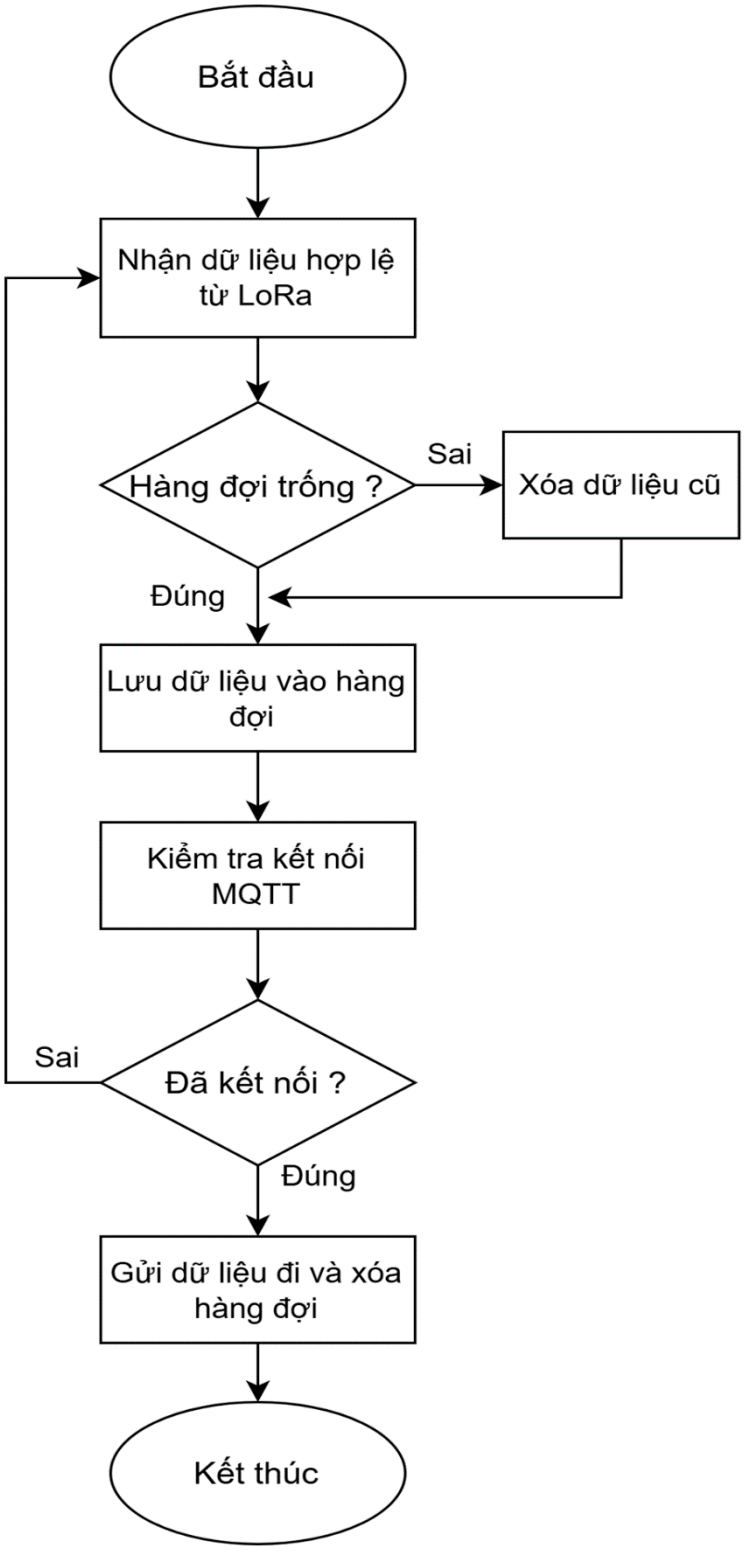
Quy trình gửi LoRa với cơ chế gửi gói tin, chờ phản hồi và thử lại nếu không thành công. Quá trình bắt đầu bằng việc gửi gói tin, sau đó kiểm tra xem gửi có thành công không. Nếu thất bại, hệ thống ghi lỗi và dừng; nếu thành công, hệ thống chờ phản hồi trong một khoảng thời gian nhất định. Khi nhận phản hồi, nếu thành công, gói tin được phân tích và xử lý tiếp; nếu thất bại, hệ thống kiểm tra đã đủ số lần tối đa thử lại hay chưa và hệ thống thử gửi lại gói tin với khoảng chờ ngẫu nhiên để tránh xung đột, sau đó tiếp tục chờ phản hồi. Nếu quá số lần thử lại, lỗi được ghi nhận. Cơ chế gửi nhiều lần và chờ phản hồi này đảm bảo kiên trì trong việc truyền dữ liệu, đồng thời giới hạn số lần thử để tránh lặp vô hạn (Hình 3.47).



Hình 3.47: Lưu đồ thuật toán xử lý gói tin LoRa

- **XỬ LÝ GÓI TIN NẾU MẤT KẾT NỐI MQTT HOẶC WIFI**

Dữ liệu nhận từ LoRa được lưu vào hàng đợi có kích thước cố định (100 phần tử). Nếu hàng đợi đầy do mất kết nối MQTT hoặc WiFi, hệ thống sẽ ghi đè dữ liệu cũ nhất để lưu dữ liệu mới, đảm bảo không mất thông tin gần nhất. Khi kết nối MQTT được khôi phục, dữ liệu trong hàng đợi được xử lý và gửi đi. Điều này giúp hệ thống duy trì hoạt động liên tục ngay cả khi kết nối không ổn định (Hình 3.48).



Hình 3.48: Lưu đồ thuật toán gửi và chờ phản hồi LoRa

CHƯƠNG 4

KẾT QUẢ

4.1 TỔNG QUAN KẾT QUẢ ĐẠT ĐƯỢC

Sau quá trình nghiên cứu và triển khai, nhóm đã xây dựng thành công một hệ thống hoàn chỉnh bao gồm cả phần cứng và phần mềm, đáp ứng mục tiêu giám sát nhà kính và điều khiển thiết bị tại nhiều khu vực khác nhau.

Về phần cứng:

- Thiết kế và chế tạo các Node (nhà kính) cảm biến có khả năng thu thập dữ liệu môi trường (nhiệt độ, độ ẩm, ánh sáng...) và điều khiển thiết bị điện.
- Phát triển mạch Gateway đóng vai trò trung tâm điều phối, thu nhận dữ liệu từ nhiều Node (nhà kính) thông qua mạng LoRa và truyền tải dữ liệu đến Server qua giao thức MQTT.

Về phần mềm:

- Xây dựng Web Server sử dụng Node.js, kết nối với cơ sở dữ liệu MongoDB để lưu trữ và xử lý dữ liệu cảm biến.
- Thiết kế giao diện Website hỗ trợ người dùng đăng nhập, tạo dự án, giám sát dữ liệu theo thời gian thực và điều khiển thiết bị điện từ xa.

Phân tích và định tuyến luồng dữ liệu:

Ngoài các chức năng thu thập và điều khiển, nhóm đã xây dựng được một mô hình định tuyến dữ liệu tối ưu giữa các Node và Gateway nhằm đảm bảo tính ổn định, độ tin cậy cao và giảm thiểu mất mát dữ liệu trong môi trường mạng không dây.

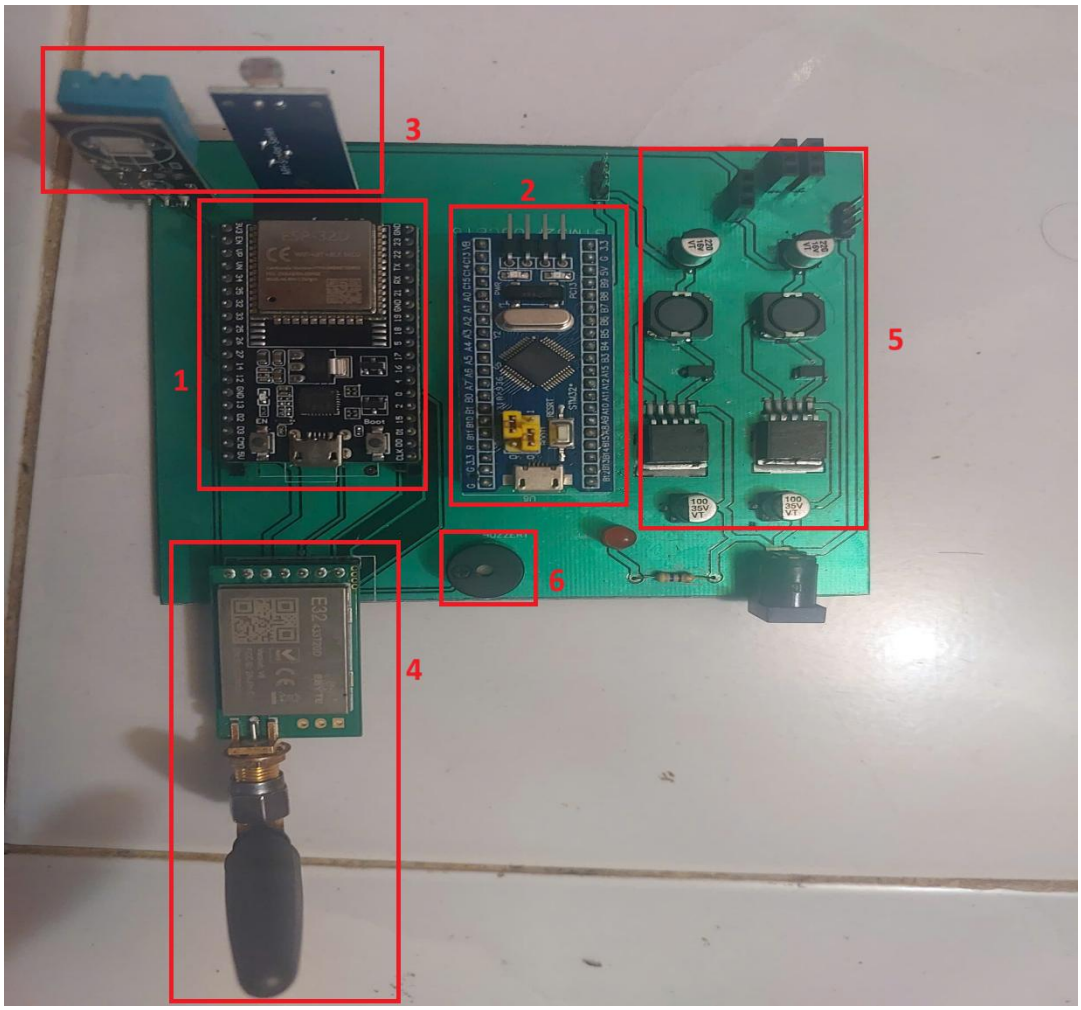
Hệ thống xử lý dữ liệu được tổ chức theo mô hình phân tán, cho phép mở rộng quy mô mạng lưới trong tương lai và đáp ứng tốt các yêu cầu của một hệ thống quản lý thiết bị điện thông minh đa khu vực.

4.2 KẾT QUẢ PHẦN CỨNG

4.2.1 Phần cứng Node

Với mỗi Node có khả năng thu thập được 3 loại dữ liệu cảm biến: ánh sáng, nhiệt độ, độ ẩm. Ngoài ra, để thuận tiện cho việc quản lý, phần cứng sẽ có tích hợp Buzzer để phát thông báo trong quá trình thực thi.

Mặt khác, sử dụng nguồn 12V được cấp bởi Adapter và đưa ra nhiều chân nguồn với mức điện áp khác nhau để tạo ra khả năng mở rộng hệ thống, tích hợp đa dạng cảm biến và thiết bị điện.



Hình 4.1: Hình ảnh thực tế board phần cứng Node (nhà kính).

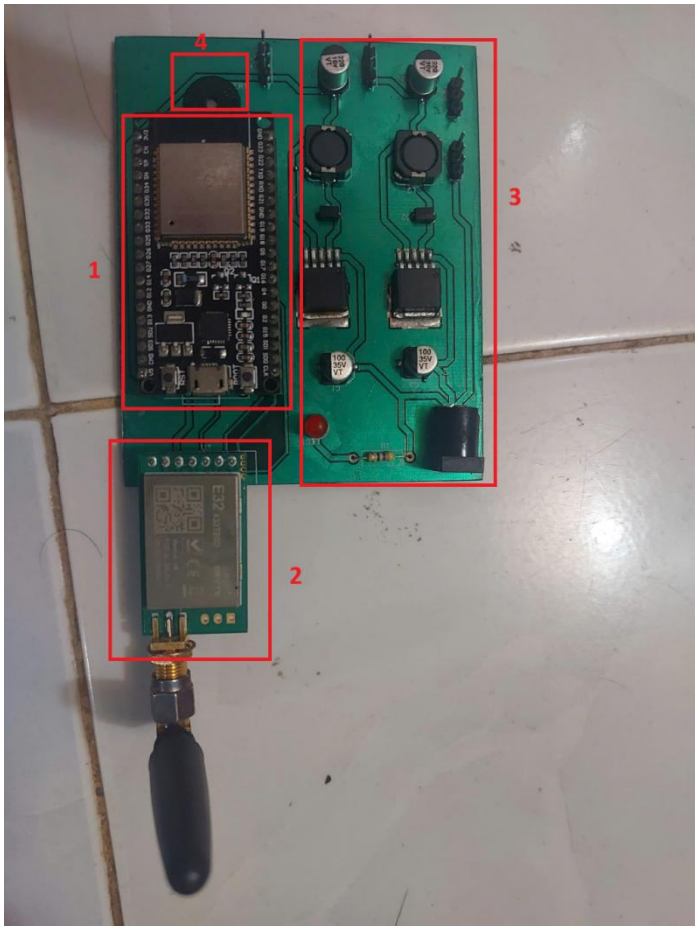
Hình 4.1 bao gồm các linh kiện: Vị trí số 1 là ESP32; vị trí số 2 STM32F103C8T6; vị trí số 3 gồm cảm biến ánh sáng và cảm biến nhiệt độ, độ ẩm; vị trí số 4 là module

LoRa E32 433T30D; vị trí số 5 gồm bộ nguồn 12V, hạ áp 3V3 5V và có led thông báo khi có nguồn; vị trí số 6 buzzer cảnh báo.

4.2.2 Phần cứng Gateway

Board Gateway sử dụng vi điều khiển ESP32 có khả năng kết nối Internet để làm trung gian giữa mạng cục bộ và Server thông qua giao thức MQTT và kết nối với các Node thông qua LoRa. Tương tự với Node, Gateway cũng tích hợp Buzzer để thông báo cho người dùng.

Thêm vào đó, board Gateway còn tích hợp các chân nguồn từ 3.3V - 12V, cung cấp khả năng linh hoạt để giao tiếp thông qua những chuẩn truyền thông khác tùy vào nhu cầu thực tế của người dùng.



Hình 4.2: Hình ảnh thực tế board phần cứng Gateway.

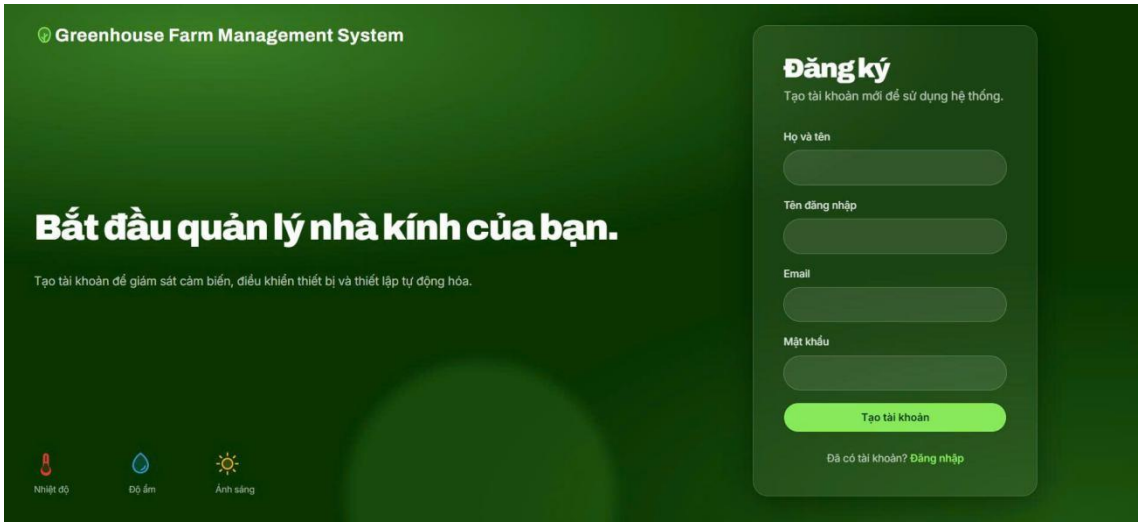
Hình 4.2 bao gồm các linh kiện: Vị trí số 1 là ESP32; vị trí số 2 là module LoRa E32 433T30D; vị trí số 3 gồm bộ nguồn 12V hạ áp 3V3,5V và có led thông báo nguồn; vị trí số 4 buzzer thông báo.

4.3 KẾT QUẢ PHẦN MỀM

4.3.1 Giao diện Website

Hệ thống phần mềm bao gồm một giao diện Website thân thiện với người dùng, được thiết kế nhằm hỗ trợ việc giám sát và điều khiển thiết bị điện một cách trực quan và hiệu quả. Giao diện Website cho phép người dùng dễ dàng thực hiện các thao tác như đăng ký, đăng nhập, tạo và quản lý các dự án giám sát tại nhiều khu vực khác nhau.

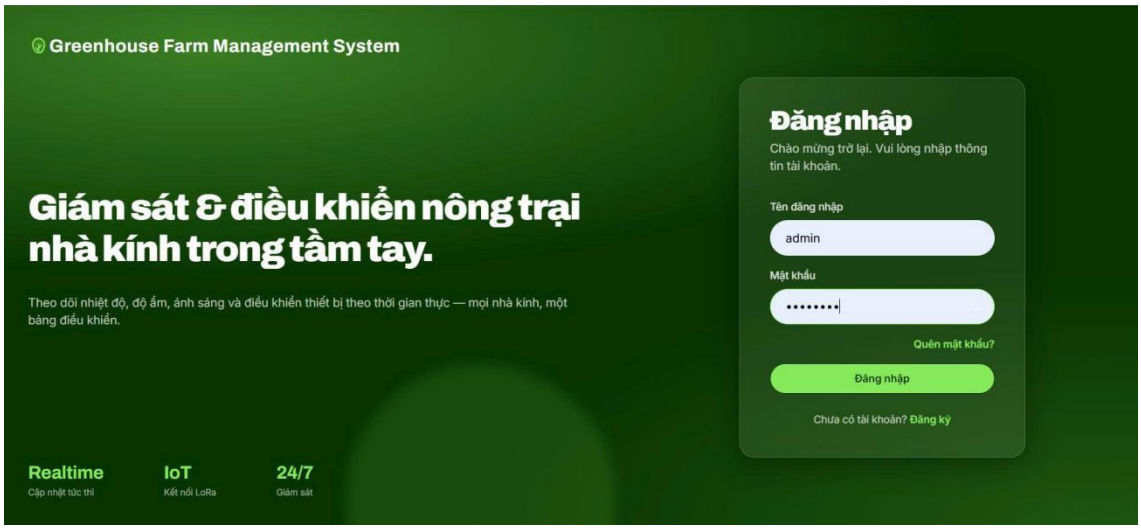
4.3.1.1 Giao diện đăng ký người dùng



Hình 4.3: Giao diện đăng ký tài khoản người dùng.

Hình 4.3 là trang đăng ký người dùng mới của hệ thống, bao gồm các trường thông tin của người dùng như tên, email,...Việc đăng ký cần xác thực các trường bắt buộc như tên, email, tên đăng nhập và mật khẩu không được để trống.

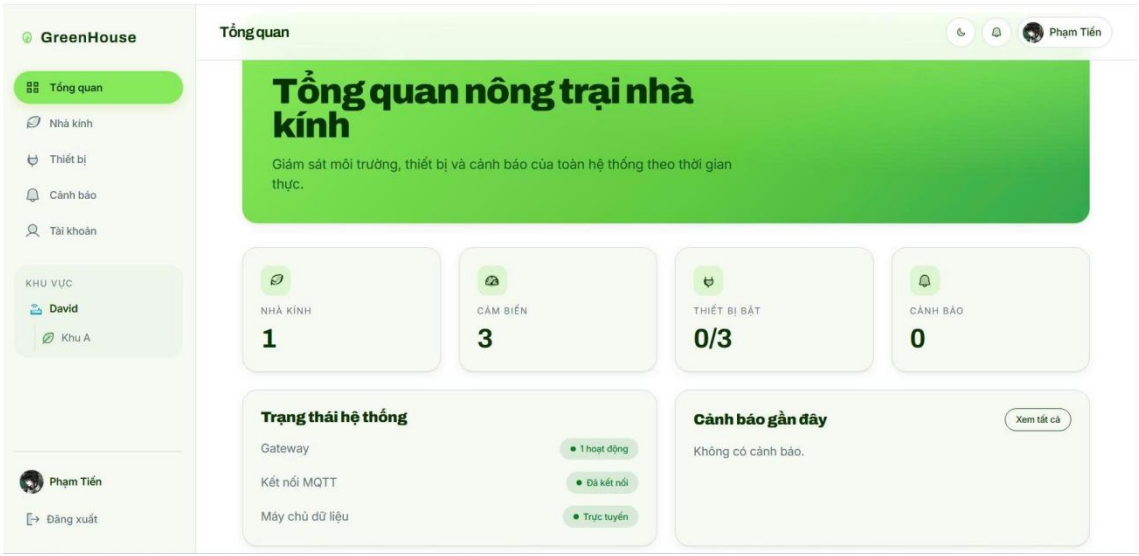
4.3.1.2 Giao diện đăng nhập



Hình 4.4: Giao diện đăng nhập người dùng.

Hình 4.4 là trang đăng nhập với tài khoản đã được đăng ký từ trước, yêu cầu xác minh chính xác cả username và password. Thêm vào đó, trang còn có chức năng lưu thông tin người dùng vào Local Storage để sử dụng cho lần đăng nhập.

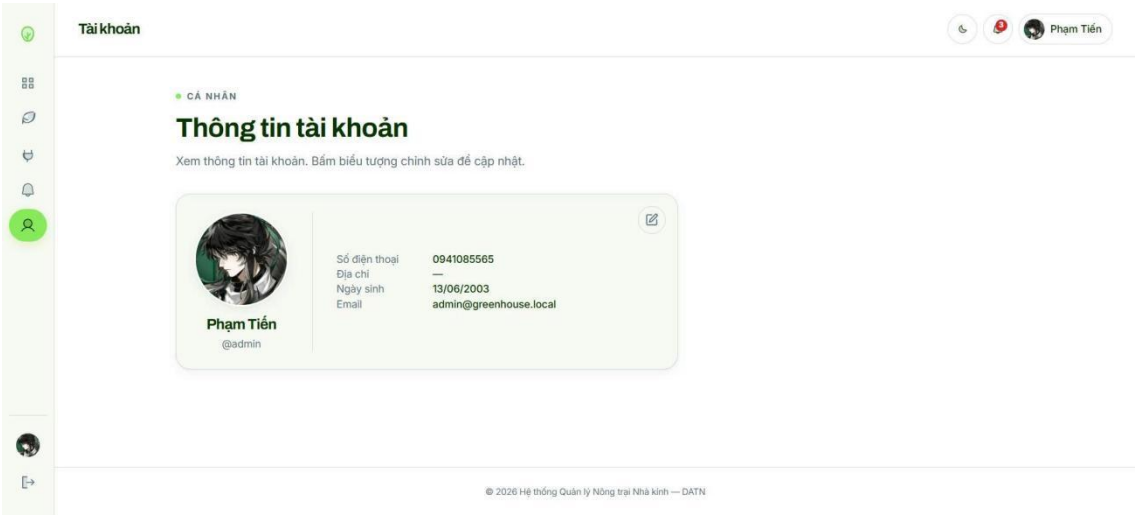
4.3.1.3 Giao diện trang chủ



Hình 4.5: Giao diện trang chủ hệ thống

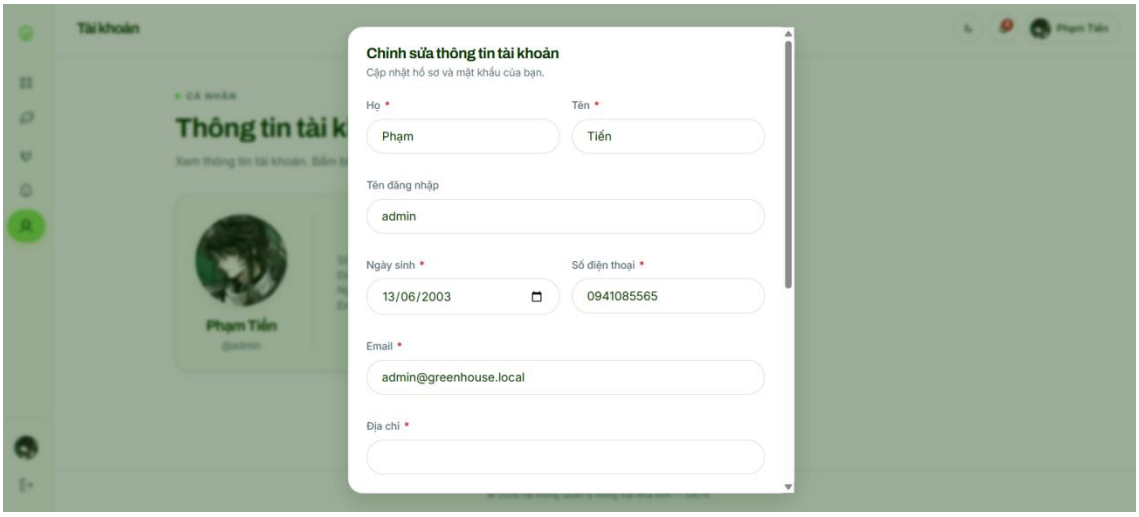
Trang chủ là nơi cung cấp cái nhìn tổng quan về toàn bộ hệ thống cần được thiết kế đẹp mắt và linh hoạt trong việc điều hướng đến các trang khác (Hình 4.5).

4.3.1.4 Giao diện trang thông tin người dùng

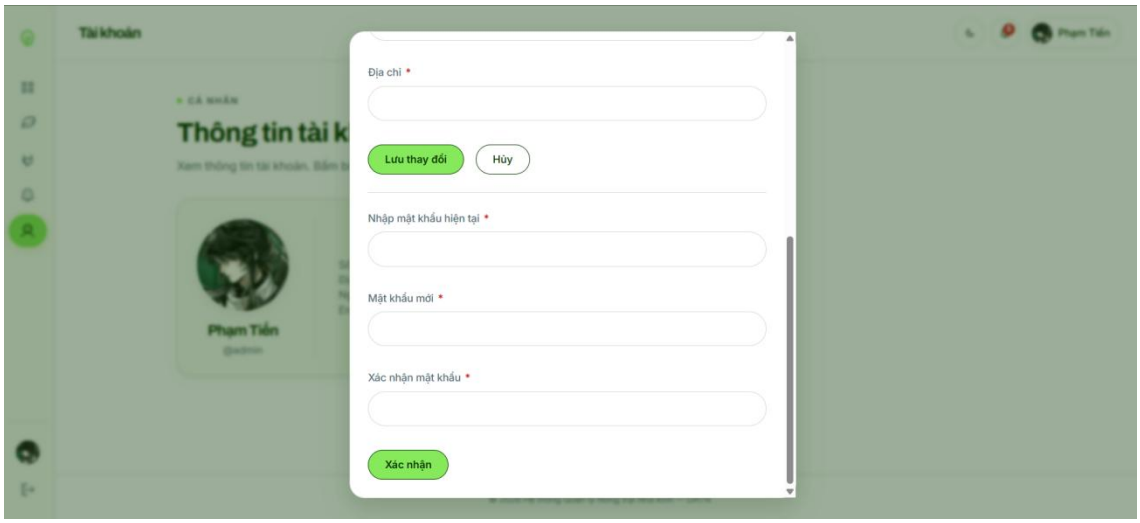


Hình 4.6: Giao diện trang thông tin người dùng.

Giao diện quản lý thông tin người dùng Hình 4.6 là nơi để người dùng có thể thay đổi thông tin cá nhân (Hình 4.7) hoặc cập nhật mật khẩu (Hình 4.8).

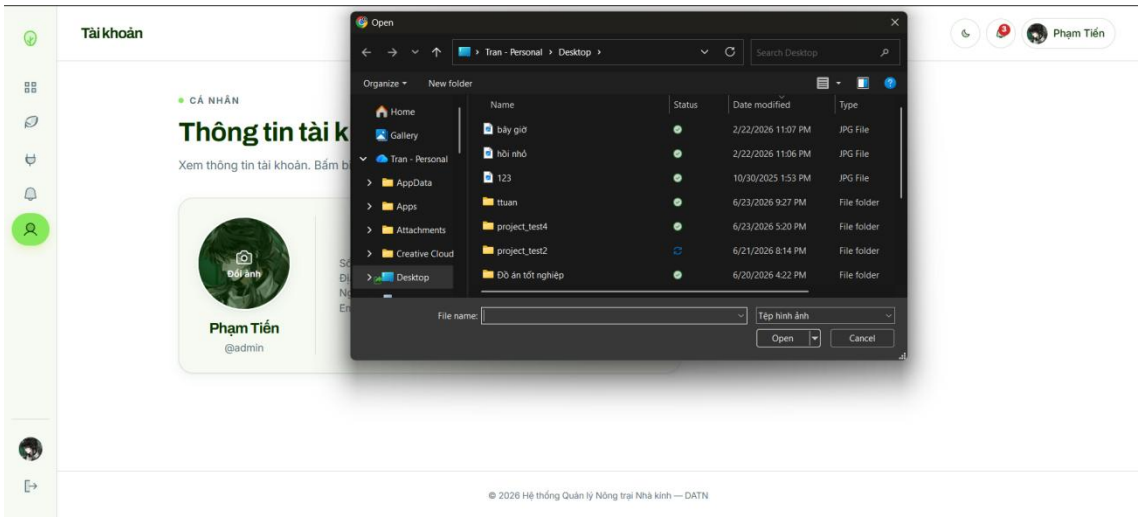


Hình 4.7: Form cập nhật thông tin cá nhân.



Hình 4.8: Form thay đổi mật khẩu người dùng.

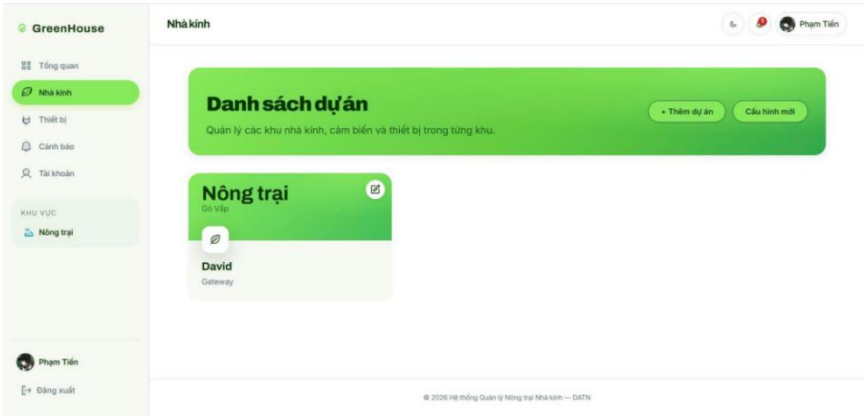
Cuối cùng, người dùng cũng có khả năng thay đổi ảnh cá nhân thay vì xuất hiện chữ cái đầu tiên của tên (Hình 4.9).



Hình 4.9: Form thay đổi avatar người dùng.

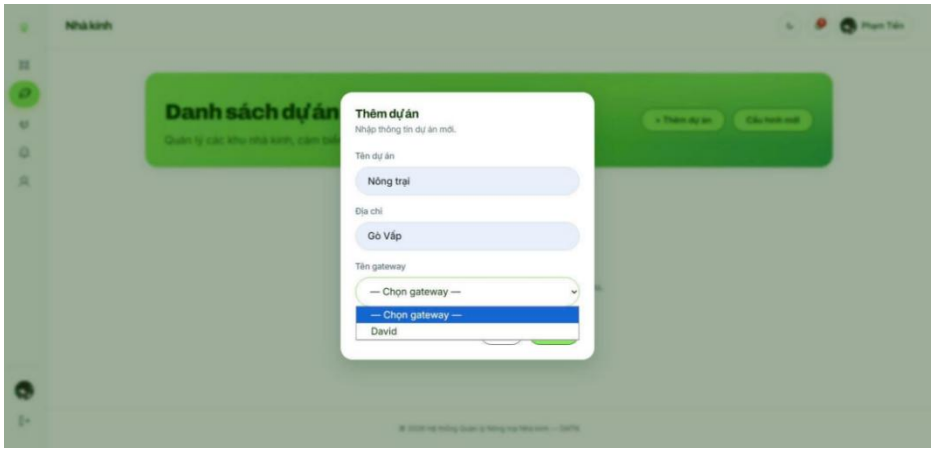
4.3.1.5 Giao diện trang quản lý dự án

Trang nhà kính được tạo ra với mục đích quản lý tất cả dự án một cách tổng quan nhất về số lượng thiết bị, cảm biến, điều khiển thiết bị. Đồng thời có các chức năng khởi tạo, chỉnh sửa và xóa các dự án không cần thiết.



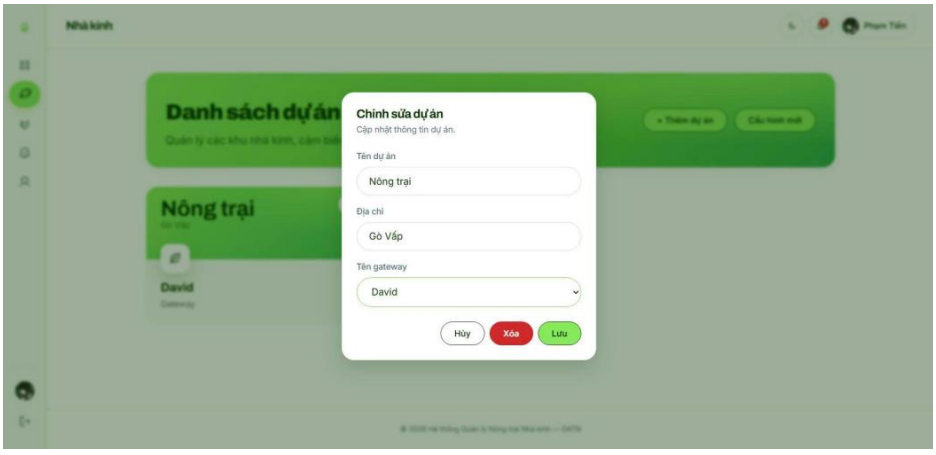
Hình 4.10: Giao diện quản lý dự án.

Dưới đây là một vài giao diện tương tác với nhà kính và gateway:



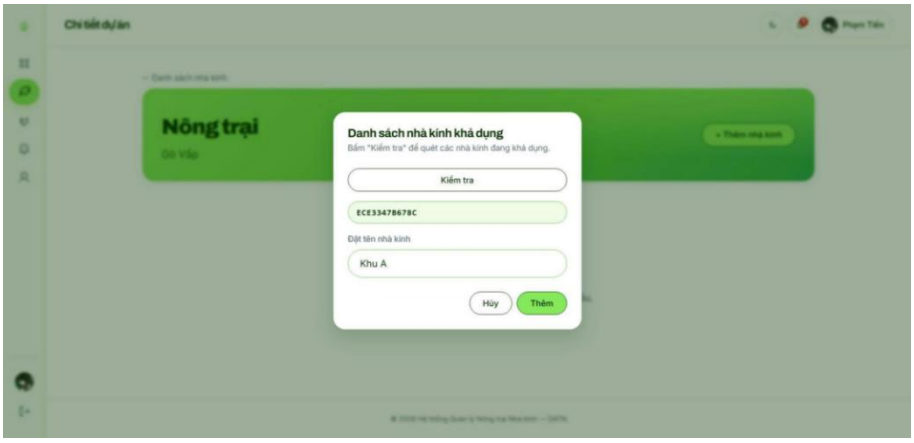
Hình 4.11: Form thêm dự án mới.

Hình hiển thị form dùng để tạo dự án mới gồm các trường tên dự án, địa chỉ, tên gateway.

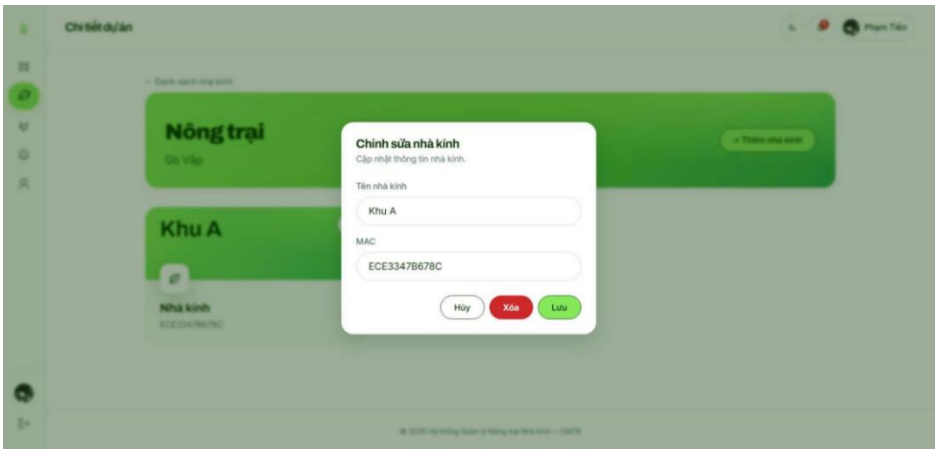


Hình 4.12: Form chỉnh sửa thông tin dự án đã có.

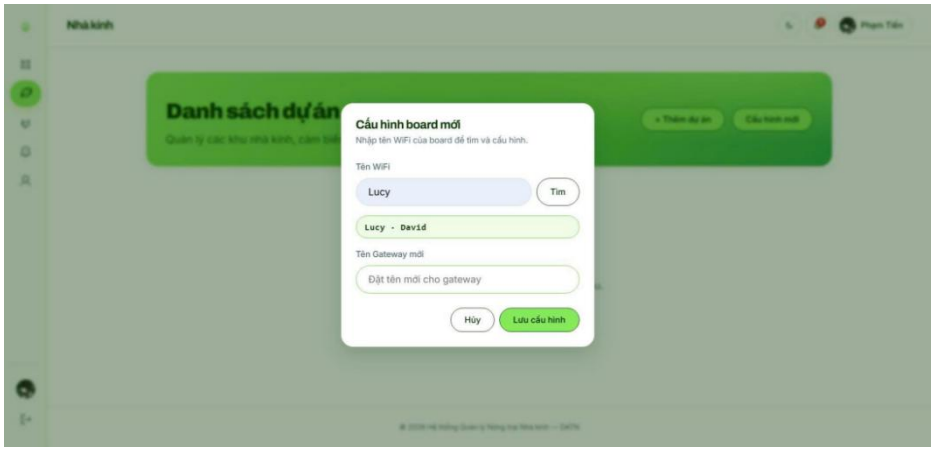
Với form cập nhật thông tin dự án Hình 4.12 các trường thông tin được cập nhật thông qua giao thức GET để đồng bộ với dữ liệu trước đó, tuy nhiên Form tích hợp cả việc xóa dự án.



Hình 4.13: Form thêm nhà kính mới vào dự án.



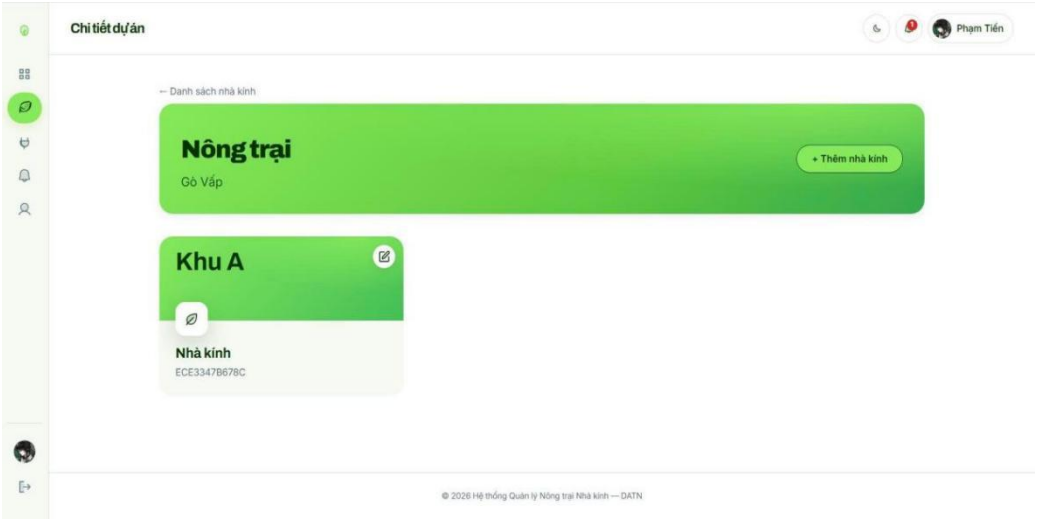
Hình 4.14: Form chỉnh sửa nhà kính đã có.



Hình 4.15: Form cấu hình Gateway mới.

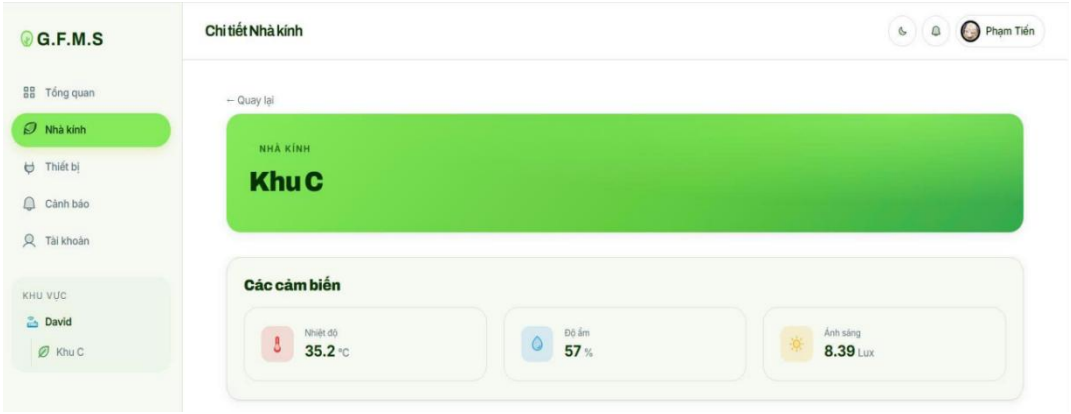
Ngoài ra, việc cấu hình một board gateway mới cũng được tích hợp trong trang này để sau khi người dùng tạo dự án thì có thể đổi tên Gateway (phần cứng) một cách nhanh chóng.

Với trang quản lý dự án, sẽ được điều hướng chính đến hai trang con là quản lý Gateway và quản lý Node (nhà kính). Ngoài ra, giao diện còn có tích hợp chức năng chỉnh sửa nhà kính để dễ dàng quản lý cũng như đổi tên thuận tiện cho việc quản lý và ghi nhớ.

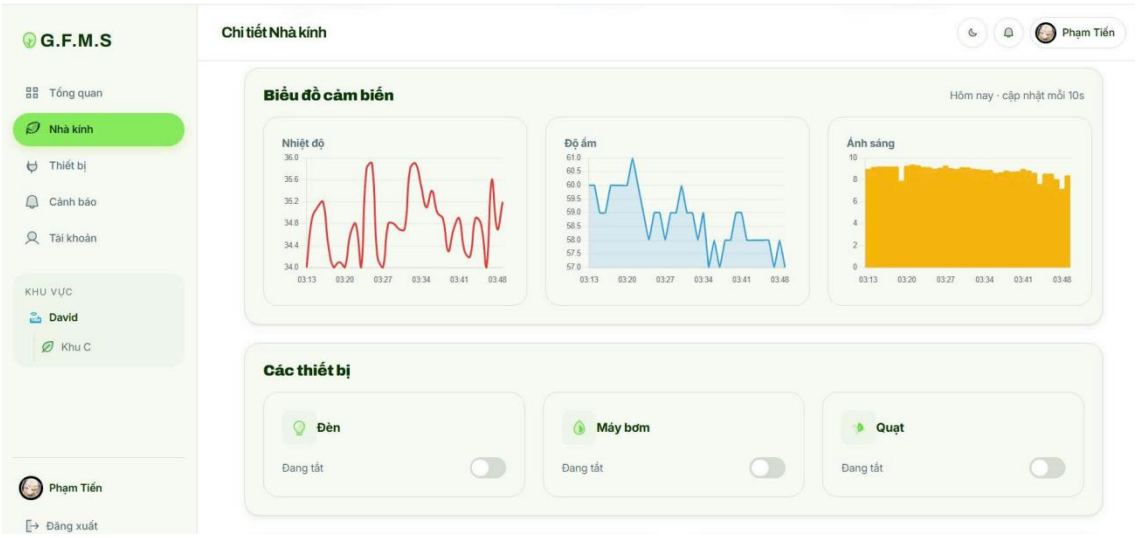


Hình 4.16: Giao diện quản lý nhà kính (node).

Về trang quản lý cảm biến và các thiết bị , có cái nhìn tổng quan về thông số 3 cảm biến nhiệt độ, độ ẩm, ánh sáng dưới dạng thông số và biểu đồ, kết hợp điều khiển thiết bị từ xa (đèn chiếu sáng, quạt, máy bơm) thông qua web.



Hình 4.17: Giao diện hiển thị thông số cảm biến.



Hình 4.18: Giao diện hiển thị biểu đồ cảm biến và điều khiển thiết bị.

4.3.2 Cơ sở dữ liệu

4.3.2.1 Dữ liệu của User


```
_id: "user1"
name : "Phạm Tiến"
birth : "2003-06-13"
phone : "0941085565"
email : "admin@greenhouse.local"
username : "admin"
password : "admin123"
avatar : "…"
home : ""
▸ gatewayId : Array (empty)
```


Hình 4.19: Dữ liệu của User được lưu trên cơ sở dữ liệu.

Sau khi người dùng thực hiện thao tác đăng ký tài khoản trên hệ thống thông qua giao diện hoặc API, thông tin đăng ký như họ tên, email, tên đăng nhập, mật khẩu (đã mã hoá), thời gian tạo, v.v... sẽ được hệ thống kiểm tra hợp lệ và sau đó lưu trữ vào MongoDB. Cụ thể, dữ liệu người dùng sẽ được lưu trong collection có tên là users, nằm trong cơ sở dữ liệu chính của hệ thống. Mỗi bản ghi (document) đại diện cho một tài khoản người dùng vừa đăng ký ở website.

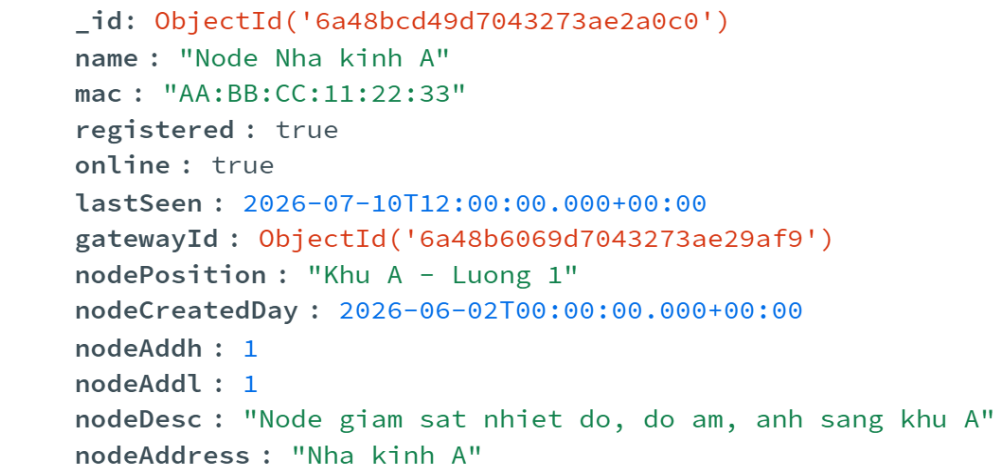
4.3.2.2 Dữ liệu của Gateways



Hình 4.20: Dữ liệu của Gateways được lưu trên cơ sở dữ liệu.

Sau khi người dùng đăng ký tài khoản thành công và đăng nhập vào hệ thống, người dùng có thể bắt đầu khởi tạo một dự án mới bằng cách đăng ký Gateway – đây là bước đầu tiên để cấu hình hệ thống thu thập và giám sát dữ liệu từ các node cảm biến. Mỗi Gateway được lưu dưới dạng một document riêng biệt trong MongoDB, bao gồm các thông tin chi tiết về tên, vị trí, thời gian tạo và người sở hữu. Việc liên kết giữa tài khoản người dùng và các Gateway tạo ra cấu trúc quản lý dạng một người dùng – nhiều Gateway, phù hợp với các hệ thống IoT có tính phân cấp theo vùng hoặc thiết bị.

4.3.2.3 Dữ liệu của Nhà kính



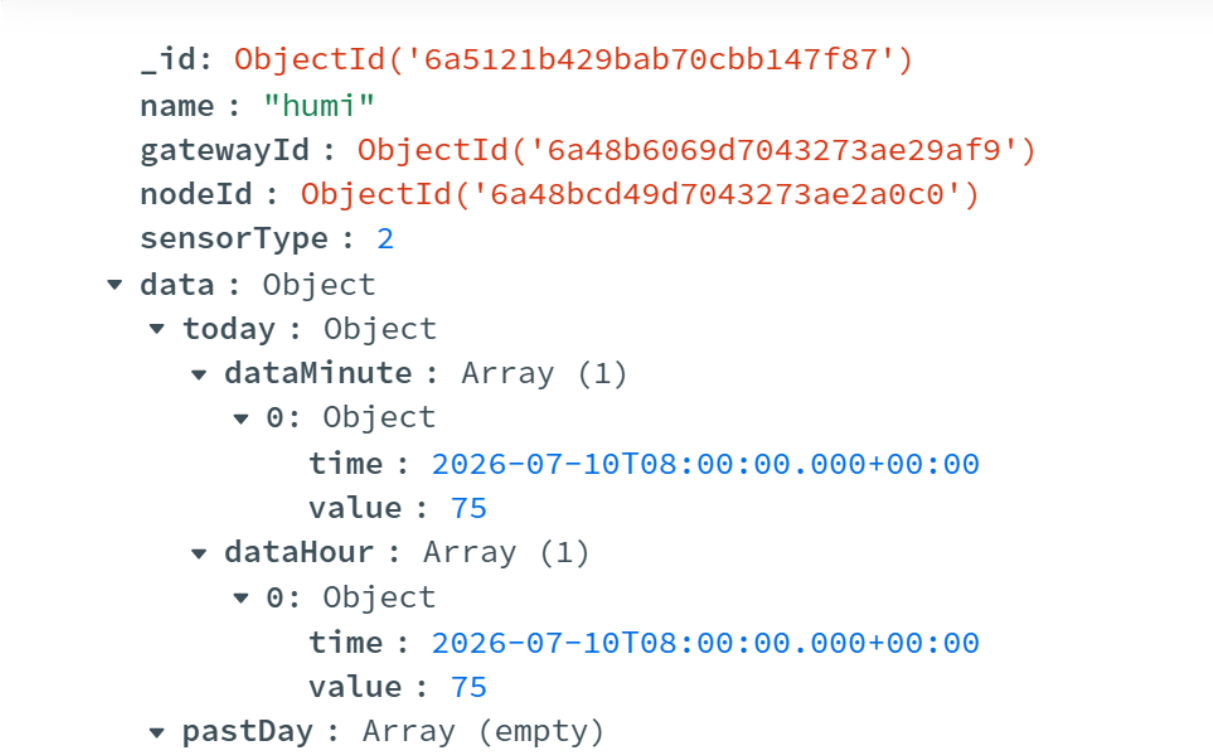
Hình 4.21: Dữ liệu của Node được lưu trên cơ sở dữ liệu.

Sau khi người dùng đã khởi tạo thành công Gateway, bước tiếp theo trong quá trình cấu hình hệ thống là đăng ký các Node cảm biến. Node là phần tử trung gian giữa Gateway và các cảm biến hoặc thiết bị điều khiển, đóng vai trò thu thập dữ liệu và gửi về Gateway thông qua giao tiếp không dây. Người dùng nhấn lưu lại thì dữ liệu sẽ được lưu trên MongoDB.

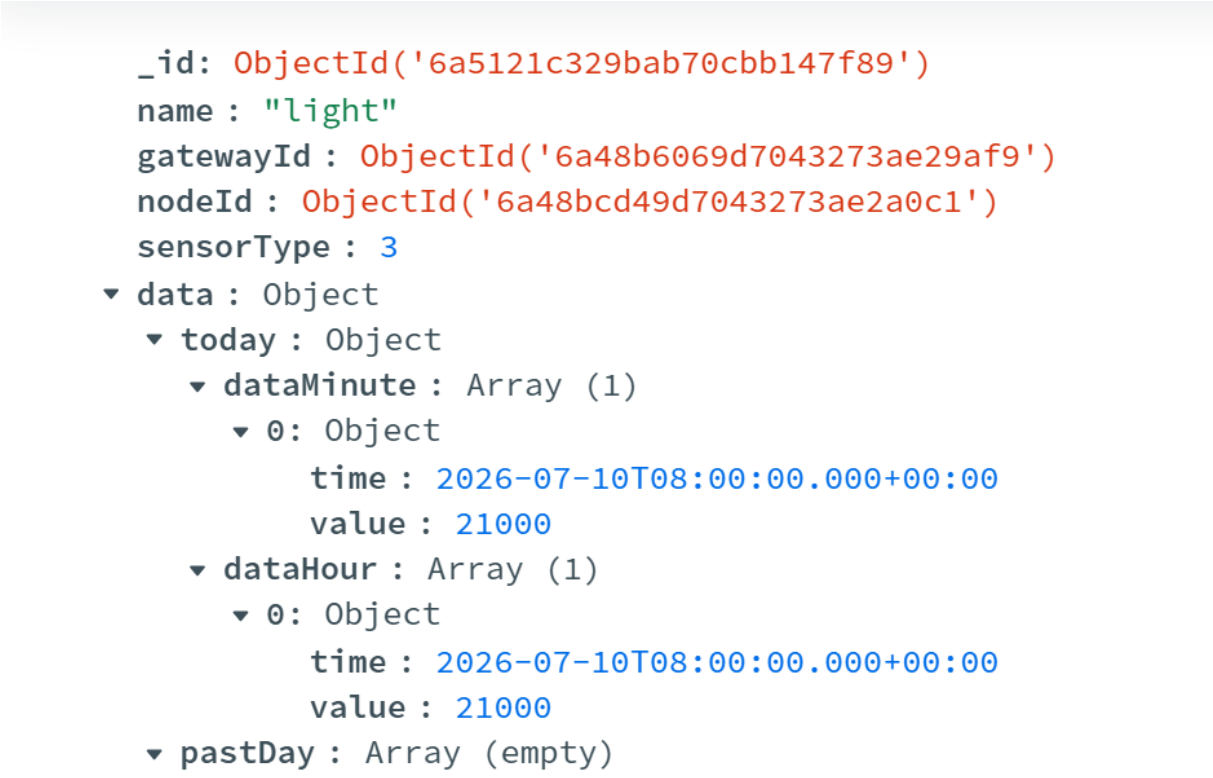
4.3.2.4 Dữ liệu của Sensors



Hình 4.22: Dữ liệu cảm biến nhiệt độ thuộc Sensors được lưu trên cơ sở dữ liệu.



Hình 4.23: Dữ liệu cảm biến độ ẩm thuộc Sensors được lưu trên cơ sở dữ liệu.



Hình 4.24: Dữ liệu cảm biến ánh sáng thuộc Sensors được lưu trên cơ sở dữ liệu.

Sau khi Node đã được đăng ký và cấu hình, người dùng có thể gán các cảm biến (sensor) tương ứng để thu thập dữ liệu môi trường hoặc trạng thái thiết bị. Các sensor được cấu hình tại mỗi Node sẽ định nghĩa các loại dữ liệu mà Node sẽ thu thập và gửi về Gateway, sau đó được truyền lên Server để lưu trữ và xử lý. Sau khi xử lý xong thì server lưu lại loại sensors và giá trị thuộc về node quản lý. Phù hợp cho việc phân cấp trong hệ thống này.

4.3.2.5 Dữ liệu của Devices

```
_id: ObjectId('6a5122af29bab70cbb147f8c')
gatewayId : ObjectId('6a48b6069d7043273ae29af9')
nodeId : ObjectId('6a48bcd49d7043273ae2a0c0')
name : "Quat thong gio A1"
pin : 1
description : "Relay dieu khien quat thong gio khu A"
status : true
```

Hình 4.25: Dữ liệu thiết bị quạt gió thuộc Devices được lưu trên cơ sở dữ liệu.

```
_id: ObjectId('6a5122b829bab70cbb147f8e')
gatewayId : ObjectId('6a48b6069d7043273ae29af9')
nodeId : ObjectId('6a48bcd49d7043273ae2a0c0')
name : "Den chieu sang A1"
pin : 2
description : "Relay dieu khien den chieu sang khu A"
status : false
```

Hình 4.26: Dữ liệu thiết bị đèn chiếu thuộc Devices được lưu trên cơ sở dữ liệu.

```
_id: ObjectId('6a5122c229bab70cbb147f90')
gatewayId : ObjectId('6a48b6069d7043273ae29af9')
nodeId : ObjectId('6a48bcd49d7043273ae2a0c1')
name : "May bom tuoi B1"
pin : 1
description : "Relay dieu khien may bom tuoi nuoc khu B"
status : true
```


Hình 4.27: Dữ liệu thiết bị máy bơm thuộc Devices được lưu trên cơ sở dữ liệu.

Người truy cập vào trang nhà kính (hay trang điều khiển cảm biến và nhiệt độ), ứng với mỗi thiết bị điện sẽ được định dạng trên MongoDB với cấu trúc id, gatewayid, nodeid, tên thiết bị, số lượng (pin), và trạng thái (status).

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và triển khai, đề tài "Hệ thống quản lý nông trại nhà kính" đã hoàn thành một hệ thống hoàn chỉnh gồm cả phần cứng và phần mềm, hoạt động ổn định trong điều kiện thử nghiệm. Hệ thống có khả năng giám sát điều kiện môi trường trong nhà kính tại nhiều khu vực, truyền dữ liệu về trung tâm thông qua công nghệ LoRa kết hợp giao thức MQTT, lưu trữ vào cơ sở dữ liệu và cho phép người dùng giám sát, điều khiển thiết bị điện từ xa thông qua giao diện Website.

5.1.1 Các vấn đề đã giải quyết được

Đối chiếu với các mục tiêu đã đề ra ở Chương 1, đề tài đã đạt được những kết quả sau:

- Giám sát môi trường nhà kính: Hệ thống thu thập được ba thông số môi trường là nhiệt độ, độ ẩm và ánh sáng, phục vụ việc theo dõi và đánh giá điều kiện sinh trưởng trong từng khu vực nhà kính.
- Điều khiển thiết bị điện từ xa: Người dùng có thể bật/tắt các thiết bị trong nhà kính (đèn, quạt, máy bơm…) thông qua giao diện Website một cách trực tiếp (thủ công), theo lịch trình định sẵn, hoặc tự động theo ngưỡng giá trị cảm biến.
- Truyền dữ liệu không dây tầm xa: Việc sử dụng LoRa cho giao tiếp giữa Node và Gateway giúp hệ thống hoạt động hiệu quả tại các khu vực nhà kính phân tán, không phụ thuộc vào hạ tầng mạng WiFi tại từng Node.
- Xây dựng Server Backend và cơ sở dữ liệu: Đã phát triển backend bằng Node.js theo mô hình MVC, kết nối với cơ sở dữ liệu MongoDB và cung cấp REST API có bảo mật, đảm bảo việc lưu trữ và truy xuất dữ liệu theo thời gian thực.

- Giao diện người dùng trực quan: Website cho phép người dùng đăng ký, đăng nhập, quản lý dự án/nhà kính, giám sát dữ liệu cảm biến qua biểu đồ và điều khiển thiết bị, đồng thời hỗ trợ hiển thị trên nhiều loại thiết bị (đa nền tảng).
- Cơ chế tự động hóa: Hệ thống hỗ trợ đặt lịch và thiết lập điều kiện tự động bật/tắt thiết bị dựa trên giá trị cảm biến (ví dụ: tự động bật quạt khi nhiệt độ vượt ngưỡng), giúp giảm thao tác thủ công và nâng cao hiệu quả canh tác.

5.1.2 Ưu điểm của hệ thống

- Tính linh hoạt và khả năng mở rộng cao: Kiến trúc cho phép thêm nhiều Node tại các khu vực nhà kính khác nhau và triển khai nhiều Gateway độc lập kết nối về cùng một server mà không phải thay đổi cấu trúc phần cứng hay phần mềm cốt lõi.
- Kiến trúc Node hai vi điều khiển (ESP32 + STM32) hợp lý: Việc phân tách nhiệm vụ giữa hai vi điều khiển giúp tối ưu hiệu năng và độ ổn định. ESP32 đảm nhận giao tiếp LoRa và thu thập dữ liệu từ các cảm biến, trong khi STM32 phụ trách việc bật/tắt và điều khiển các thiết bị điện. Cách tổ chức này cho phép xử lý song song giữa truyền thông và điều khiển, giúp hệ thống hoạt động ổn định và thuận lợi cho việc mở rộng về sau.
- Tiết kiệm chi phí và năng lượng: Sử dụng các linh kiện phổ biến, giá thành hợp lý; công nghệ LoRa tiêu thụ điện năng thấp, phù hợp với hệ thống IoT nông nghiệp phân tán trên diện rộng.
- Có cơ chế dự phòng: Hệ thống được trang bị các cơ chế khôi phục kết nối WiFi/MQTT và lưu tạm dữ liệu khi mất kết nối, giúp hạn chế mất mát dữ liệu trong quá trình vận hành.

5.1.3 Hạn chế của hệ thống

- Số loại cảm biến còn hạn chế: Hệ thống hiện chỉ hỗ trợ ba loại cảm biến (nhiệt độ, độ ẩm, ánh sáng), chưa mở rộng sang các loại cảm biến chuyên biệt cho canh tác nhà kính như độ ẩm đất, nồng độ CO_2 hay dinh dưỡng.

- Phạm vi và độ ổn định của LoRa bị ảnh hưởng bởi môi trường: Trong khu vực có nhiều vật cản, khoảng cách truyền thực tế giảm đáng kể so với thông số lý thuyết, đồng thời độ trễ phản hồi tăng lên khi khoảng cách xa hoặc số lượng Node tăng.
- Giao tiếp LoRa bán song công: Do LoRa không truyền và nhận đồng thời, vẫn tồn tại nguy cơ xung đột khi Node đang gửi dữ liệu mà Gateway gửi lệnh điều khiển, dẫn đến khả năng bỏ lỡ gói tin.
- Bảo mật đường truyền chưa cao: Dữ liệu truyền giữa các thành phần chưa được tích hợp cơ chế mã hóa mạnh, có thể tiềm ẩn rủi ro khi triển khai trong môi trường yêu cầu bảo mật cao.
- Phụ thuộc vào anten và điều kiện lắp đặt: Để đạt khoảng cách và độ ổn định tốt, anten cần được lắp ở vị trí cao và thông thoáng, gây hạn chế nhất định khi lắp đặt thực tế trong nhà kính.

5.2 HƯỚNG PHÁT TRIỂN

Nhằm khắc phục những hạn chế còn tồn tại và nâng cao khả năng ứng dụng của hệ thống trong canh tác nhà kính, nhóm đề xuất một số hướng phát triển trong tương lai như sau:

- Mở rộng và đa dạng hóa cảm biến: Bổ sung các loại cảm biến phù hợp với nông nghiệp nhà kính như độ ẩm đất, nồng độ CO_2, cường độ pH/dinh dưỡng, lượng mưa…, đồng thời xây dựng cơ chế cấu hình linh hoạt để người dùng dễ dàng thay đổi loại cảm biến tùy theo loại cây trồng.
- Tối ưu hóa định tuyến và xử lý dữ liệu: Cải thiện thuật toán truyền nhận và quản lý đa Node trong mạng LoRa nhằm giảm độ trễ, hạn chế xung đột và nâng cao độ tin cậy khi số lượng nhà kính tăng lên.
- Thiết kế thiết bị Repeater cho LoRa: Nghiên cứu bổ sung các thiết bị trung gian (Repeater) để mở rộng vùng phủ sóng, giúp chuyển tiếp dữ liệu từ các Node ở xa hoặc bị vật cản về Gateway một cách ổn định hơn.

- Tăng cường bảo mật dữ liệu: Tích hợp các cơ chế mã hóa nâng cao (AES, TLS/SSL) cho đường truyền giữa Gateway và Server, đồng thời nâng mức QoS của MQTT để đảm bảo độ tin cậy cho các lệnh điều khiển và cảnh báo quan trọng.
- Phát triển ứng dụng di động: Bên cạnh giao diện Website, xây dựng ứng dụng di động (Android/iOS) cho phép người nông dân giám sát, điều khiển và nhận thông báo cảnh báo theo thời gian thực, tăng tính tiện lợi trong quá trình canh tác.
- Cải thiện độ bền phần cứng: Thiết kế vỏ bọc chống nước, chống bụi và đảm bảo cách điện cho các Node, phù hợp với môi trường nhà kính có độ ẩm cao và nhiệt độ biến động.
- Ứng dụng trí tuệ nhân tạo (AI) trong phân tích dữ liệu: Tích hợp các mô hình học máy để phát hiện bất thường trong môi trường nhà kính, dự đoán xu hướng (như nhu cầu tưới tiêu, thông gió) và tối ưu hóa việc điều khiển thiết bị tự động, hướng tới mô hình nông nghiệp chính xác (precision agriculture).

TÀI LIỆU THAM KHẢO

[1] T. M. Thái và D. N. Long, "Ứng dụng công nghệ IoT và mạng cảm biến trong giải pháp quản lý môi trường và nâng cao hiệu quả mô hình lúa tôm ở huyện An Biên tỉnh Kiên Giang," Tạp chí Khoa học Đại học Cần Thơ, tập 58, số chuyên đề SDMD, tr. 31–41, 2022.

[2] N. Đ. Thanh, F. Ferrero, L. C. V. Khải, N. H. N. Thương, P. V. Tuấn, và L. Q. Huy, "Mạng công nghiệp LoRaWAN cho thành phố Đà Nẵng: Giải pháp cho ứng dụng IoT tầm xa tiêu thụ năng lượng thấp," Tạp chí Khoa học và Công nghệ – Đại học Đà Nẵng, tập 18, số 1, tr. 7–10, 2020.

[3] U. Hunkeler, H. L. Truong, and A. Stanford-Clark, "MQTT-S – A publish/subscribe protocol for wireless sensor networks," in Proc. 3rd Int. Conf. Communication Systems Software and Middleware and Workshops (COMSWARE), IEEE, 2008.

[4] T. V. Hiền, "Đánh giá hiệu năng giao thức MQTT và HTTP trong một hệ thống IoT thời gian thực," Tạp chí Khoa học và Công nghệ – Đại học Đà Nẵng, tập 20, số 3, tr. 51–55, 2022.

[5] B. P. Crow, I. Widjaja, J. G. Kim, and P. T. Sakai, "IEEE 802.11 Wireless Local Area Networks," IEEE Communications Magazine, vol. 35, no. 9, pp. 116–126, 1997.

[6] V. Sharma and M. Dave, "SQL and NoSQL Databases," International Journal of Advanced Research in Computer Science and Software Engineering, vol. 2, no. 8, pp. 20–27, 2012.

[7] N. V. Định, "Đánh giá hiệu năng một số hệ quản trị cơ sở dữ liệu NoSQL," Luận văn Thạc sĩ, Trường Đại học Công nghệ – Đại học Quốc gia Hà Nội, 2020.

[8] STMicroelectronics, "STM32F103x8, STM32F103xB Datasheet," Sep. 2023.

[9] Z. Parker, S. Poe, and S. V. Vrbsky, "Comparing NoSQL MongoDB to an SQL DB," in Proc. 51st ACM Southeast Conference (ACMSE), 2013.

[10] A. Leff and J. T. Rayfield, “Web-application development using the Model/View/Controller design pattern,” in Proc. 5th IEEE Int. Enterprise Distributed Object Computing Conf. (EDOC), 2001, pp. 118–127.
[11] N. A. Tuấn, “Luận bàn về hiệu năng của việc sử dụng Restful Web Services trong phát triển ứng dụng di động,” Tạp chí Khoa học và Công nghệ – Đại học Đà Nẵng, tập 12, số 2(73), tr. 123–128, 2013.
[12] J. Nielsen, Usability Engineering. San Francisco, CA: Morgan Kaufmann, 1993.
[13] M. Masse, REST API Design Rulebook. Sebastopol, CA: O’Reilly Media, 2011.